



**UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH**

Centre de la Imatge i la Tecnologia Multimèdia

BACHELOR THESIS

Mario Dorado Martínez

TerrainServer - A Procedural Terrain Generation Plugin for Godot



TerrainServer

Procedural Terrain Generation for Godot

Supervisor of the Thesis [Mónica Martín Mínguez](#)
Study Programme [Game Design and Development](#)

Barcelona 2026

A mis padres y mi hermano, fueron ellos quienes me incitaron y apoyaron a empezar este viaje. Sin ellos, sencillamente, no estaría aquí.

A mis amigos, por acompañarme durante el camino.

A mi pareja, por su paciencia y por aguantarme durante todos estos años.

A mi tutora, por su apoyo, dedicación, y por soportar los dolores de cabeza que le he causado con este trabajo.

Sin vosotros esto no sería posible, gracias por hacerme ser quien soy.

To my parents and my brother, it was they who encouraged and supported me to begin this journey. Without them, I simply would not be here.

To my friends, for accompanying me along the way.

To my partner, for their patience and for putting up with me all these years.

To my supervisor, for her support, dedication, and for enduring the headaches I have caused her with this work.

Without you, this would not have been possible. Thank you for making me who I am.

Mario Dorado Martínez

Information

Title	TerrainServer - A Procedural Terrain Generation Plugin for Godot
Author	Mario Dorado Martínez
Institute	Centre de la Imatge i la Tecnologia Multimèdia
Supervisor	Mónica Martín Mínguez — Centre de la Imatge i la Tecnologia Multimèdia
Keywords	Procedural Generation, Terrain Rendering, Godot Engine, GDExtension, GPU Noise, Biome Generation, Level of Detail
Links	https://github.com/mdoradom/TerrainServer https://github.com/users/mdoradom/projects/7 https://github.com/mdoradom/TerrainServer/releases https://github.com/mdoradom/TerrainServer/milestones https://www.youtube.com/watch?v=2KfAA36HRqc

Abstract

Unlike many commercial alternatives, the Godot Engine lacks robust built-in tools for generating large-scale procedural terrain and dynamic biomes, limiting its viability for open-world game development. This thesis presents TerrainServer, an open-source GDExtension plugin that closes this gap with a fully GPU-driven architecture: terrain is rendered in real time through a Geometry Clipmap of nested grids displaced in the vertex shader, while an independently evaluated CPU heightmap stays bit-for-bit identical to it via a portable integer-hash noise function, letting an asynchronous, distance-bounded physics grid regrid collision without blocking the main thread. Biomes are assigned continuously from temperature and moisture fields classified against a Whittaker scheme, blended per fragment into soft ecotones, and textured through triplanar projection, parallax occlusion mapping, and a slope-aware rock overlay, all exposed to developers through a custom editor plugin, gizmo set, and live climate dock. A demo project built on Quixel Megascan textures configures a full five-biome, rock-layered terrain to showcase these features end to end. Benchmarking this demo across four desktop and laptop device/OS combinations confirms real-time, cross-platform performance and CPU/GPU noise parity. These results show that a fully GPU-driven, GDExtension-based architecture can deliver real-time procedural terrain and biome generation for Godot, closing a concrete capability gap in the engine.

Contents

Information & Abstract	3
Contents	4
List of Figures	7
List of Tables	10
Glossary	11
Acronyms	14
1 Introduction	15
1.1 Motivation	15
1.2 Problem formulation	16
1.3 General objectives	17
1.4 Specific objectives.	17
1.5 Scope of the project	19
2 State of Art	20
2.1 Existing tools and Solutions	20
2.2 Terrain	23
Heightmaps 23 • Meshes 24 • Voxel-based terrain 25 • Comparison 27	
2.3 Modern Terrain Rendering Techniques	28
Clipmaps 28 • Level of Detail (LOD) 31 • Tessellation 32 • Occlusion Culling 34	
2.4 Procedural Terrain Generation Methods	35
Noise Basis Functions 35 • Subdivision Methods 42 • Noise Layering and Manipulation 44 • Erosion Simulation 47 • Data-Driven Terrain Synthesis 50	
2.5 Biome Assignment Techniques.	52
Noise-Based Biome Generation 52 • Voronoi-Based Biome Assignment 53 • Comparison and Use Cases 54	
2.6 How does Godot handle terrain generation?	54
GDExtension 55 • EditorPlugin 56 • ArrayMesh 56 • MultiMeshInstance3D 57 • HeightMapShape3D 58 • FastNoiseLite 59	
3 Project Management	60
3.1 Organizational Tools	60
3.2 Development Tools	62
3.3 Validation Tools.	62
3.4 SWOT	63
3.5 Risk and Contingency Plan	64

3.6	Cost Analysis	64
	Personnel Costs 65 • Hardware and Software Amortization 65 • Total Budget 66	
3.7	Environmental Impact and Social Responsibility	66
	Development Phase 66 • Usage Phase 68 • End of Life Phase 68 • Social Responsibility 69	
3.8	Adjustments to the Project Plan	69
4	Methodology	71
4.1	Phase 0: Core Architecture & Setup	71
4.2	Phase 1: Infinite Terrain Rendering.	71
4.3	Phase 2: Async Data Generation	72
4.4	Phase 3: Physics & Collisions	72
4.5	Phase 4: Biomes & Environmental Variety.	72
4.6	Phase 5: Editor Integration & UX	73
4.7	Phase 6: Release & Documentation.	73
5	Project Development	74
5.1	Phase 0: Core Architecture & Setup	74
	File Structure 74 • CI/CD Setup 76 • Proof of Concept 76 • Result 77	
5.2	Phase 1: Infinite Terrain Rendering.	78
	Architecture Improvements 78 • Challenges Encountered & Solutions 80 • Result 81	
5.3	Phase 2: GPU-Based Procedural Generation	82
	The shader implementation 83	
5.4	Phase 3: Physics & Collision	87
	Decoupling Collision from the Render Mesh 88 • Achieving Deterministic CPU/GPU Noise Parity 88 • Physics-Specific Level of Detail 90 • Asynchronous Regrid Pipeline 90 • Height Queries and the focus_path Property 91 • Challenges Encountered & Solutions 92 • Result 93	
5.5	Phase 4: Biomes & Environmental Variety.	94
	Modelling Climate: Temperature and Moisture Fields 94 • Whittaker Classification as a Soft Per-Fragment Blend 96 • The TerrainBiomeLayer Resource 98 • Uploading Layers as Texture Arrays 100 • Triplanar Projection 101 • Parallax Occlusion Mapping over a Blended Height Field 103 • Slope-Aware Rock Overlay 106 • CPU-Side Biome Queries 107 • Challenges Encountered & Solutions 108 • Result 109	
5.6	Phase 5: Editor Integration & Tooling	111
	Editor Initialization & Introspection API 111 • Editor Preview & Camera Integration 111 • Gizmos & Visual Debugging 112 • Terrain Dock & Climate Chart 112 • Result 113	
5.7	Phase 6: Release & Documentation.	113
	Documentation & Code Cleanup 114 • Testing & Validation 114 • Distribution & Future Work 114 • Result 115	
6	Project Validation	119
6.1	Methodology	119

6.2	Cross-Platform Baseline	121
6.3	Clipmap Level-of-Detail Scaling	121
6.4	Mesh Resolution Scaling	122
6.5	Procedural Noise Cost	123
6.6	Physics Range and the Asynchronous Regrid Pipeline	124
6.7	Parallax Occlusion Mapping Cost	125
6.8	Stress Test	126
6.9	Discussion	127
7	Conclusions	129
7.1	Summary of Achievements	129
7.2	Challenges and Lessons Learned	129
7.3	Future Work and Recommendations	130
	References	132

List of Figures

2.2	(a) Example of a heightmap. (b) Mesh rendered from the heightmap.	24
2.3	Visual representation of Voxel data structures.	26
2.5	How Geometry Clipmaps Work [37].	29
2.6	Terrain Rendering Using a Coarse Geometry Clipmap [37]. . .	30
2.7	Longest Edge Bisection subdivision example [39]	33
2.8	Example of recursive non-uniform adaptive subdivision [39] . .	33
2.9	Representation of the link to Binary Tree [39]	34
2.10	(a) Example of white noise. (b) Example of perlin noise.	36
2.11	A two-dimensional grid of gradient vectors [41]	36
2.12	The dot product of a point with its nearest grid node gradient value. The dot product determines the ramp intensity relative to that corner [42]	37
2.13	The final interpolated result showing the smooth gradient noise distribution [43]	37
2.14	(a) Example of perlin noise. (b) Example of simplex noise. . . .	38
2.16	(a) Euclidean distance. (b) Manhattan distance.	40
2.17	Example of Worley noise.	42
2.18	Visualization of diamond-square algorithm on a 5×5 array [48].	43
2.19	Example of diamond-square generated heightmap [49].	44
2.20	Standard fBm implementation as a loop over octaves.	45
2.22	Two-stage domain warp pattern.	47
2.23	Single-iteration thermal erosion kernel.	49
2.24	Whittaker Biome Classification diagram [53].	53
3.1	GitHub Project Board used for task management.	60
3.2	Gantt Chart view of the project timeline.	61
3.3	GitHub Milestones tracking project deadlines.	61
3.4	SWOT Analysis Diagram.	63
5.1	Project file structure.	75
5.2	(a) Terrain3D node. (b) TerrainConfiguration resource.	77
5.3	Preliminary PoC of the terrain generation system running within the Godot editor.	78
5.4	TerrainConfiguration parameters.	79
5.5	Vertex Displacement Shader.	80
5.6	Infinite terrain using Geometry Clipmaps, Vertex Displacement Shader, and CPU-based noise.	82
5.7	Visualization of the Geometry Clipmap levels. 4 clipmap levels, terrain size of 64, mesh resolution of 64.	82
5.8	Vertex Displacement Shader with GPU-based procedural noise generation.	84

5.9	Pseudo-random hash and Perlin Noise functions implementation.	85
5.10	Combining the domain warp, the ridged relief and the redistribution curve under the province mask into the final height.	87
5.11	Integer-hash replacement shared between the GLSL shader and the C++ <code>TerrainNoise</code> port.	89
5.12	New Physics property group added to <code>TerrainConfiguration</code> . .	90
5.13	Grid-snap margin guarding regrid requests.	91
5.14	Multiple <code>RigidBody3D</code> objects interacting with the terrain. . . .	93
5.15	Temperature and moisture fields, shared between <code>terrain.gdshader</code> and <code>terrain_noise.h</code>	95
5.16	Per-fragment Whittaker weights: four <code>smoothstep</code> edges per biome layer.	97
5.17	Shared texture-slot setter.	99
5.18	Triplanar weighting and sampling, with per-axis early-out and explicit derivatives.	102
5.19	World-space parallax sweep and adaptive linear search; the refinement that follows is described below.	104
5.20	Triplanar projection and parallax occlusion mapping, disabled (a) and enabled (b), with the profiler overlay reporting CPU time, GPU time and frame rate for each.	106
5.21	The demo scene textured by biome, with the slope-aware rock overlay on the ridge.	110
5.22	Top-down view of the biome distribution produced by the temperature and moisture fields.	110
5.23	Close-up of a biome transition: grass blending into loose pebbles across an ecotone, each keeping its own surface relief.	111
5.24	The custom Terrain Dock displaying real-time statistics and the Whittaker biome chart.	113
5.25	Editor viewport showing the terrain with clipmap and collision gizmos enabled.	113
5.26	Top-down view of a highland province: ridged, snow-capped terrain with the snow biome's own rock exposed on its slopes.	116
5.27	Ground-level view inside the mountain range: sharp ridgelines without mesh-scale creasing.	117
5.28	The mountain province seen from across the lowland: flat, grassy Fractal Brownian Motion (fBm) in the foreground, ridged terrain on the horizon.	117
5.29	Approaching the province boundary.	118
5.30	Crossing into the highland province: grass gives way to the ridged, rock-covered slopes beyond it.	118
6.2	GPU and CPU frame time at the baseline configuration, median of 240 sampled frames, across the four measured device/OS combinations.	121

6.3	GPU frame time against clipmap ring count, median of 240 sampled frames, one line per device.	122
6.4	GPU frame time against mesh resolution, median of 240 sampled frames, one line per device.	123
6.5	Collision regrid latency against noise octave count, mean of the regrid observed per run, one line per device.	124
6.6	Collision regrid latency against physics range, mean of the regrid observed per run, one line per device. <code>physics_range = 8</code> is omitted: at that setting the grid clamps to its 8-cell minimum and recomputes within a single frame, so no device recorded a single observable regrid.	125
6.7	GPU frame time against Parallax Occlusion Mapping (POM) step configuration, median of 240 sampled frames, one line per device.	126

List of Tables

2.1	Comparison of popular terrain tools.	21
2.4	Comparison of terrain representations in videogames.	27
3.5	Risks and Contingency Plan.	64
3.6	Breakdown of personnel costs.	65
3.7	Hardware amortization costs.	65
3.8	Final Cost Analysis.	66
3.9	Estimated CO ₂ emissions during the development phase.	68
6.1	Devices used for benchmarking.	120
6.8	Stress scenario results, median and 90th percentile of 240 sampled frames.	127

Glossary

Biome	A distinct ecological zone (e.g., desert, forest) defined by climatic factors such as temperature and precipitation.
CDLOD	Continuous Distance-Dependent Level of Detail, an algorithm that selects mesh complexity based on the 3D distance to the camera to avoid popping artifacts.
Chunk	A discrete segment of the terrain (mesh or voxel data) used to manage memory and rendering by loading or unloading sections based on player distance.
Clipmap	A technique for efficiently caching and rendering large terrains using a set of nested regular grids centered around the viewer.
Domain warping	A technique that increases noise complexity by distorting the input coordinate space of a noise function with another noise function.
Ecotone	The transition zone between two adjacent ecological communities, where the characteristics of both biomes are mixed.
Gain	A parameter (also called persistence) that controls how quickly the amplitude decreases for each successive octave.
GDExtension	GDExtension, a system for creating native modules and plugins for the Godot game engine.
Gradient noise	A type of coherent noise (like Perlin or Simplex) generated by interpolating between pseudorandom gradients at lattice points.
Heightmap	A 2D representation of terrain elevation where each pixel value represents the height at that point.

Hydraulic erosion	A simulation algorithm that models the transport of terrain material by flowing water, creating river channels and valleys.
Instancing	A rendering technique (Hardware Instancing) that draws multiple copies of the same mesh in a single draw call, essential for dense vegetation.
Lacunarity	A parameter that determines how quickly the frequency increases for each successive octave in fractal noise generation.
Mesh	A collection of vertices, edges, and faces that define the shape of a polyhedral object in 3D computer graphics.
Mipmap	A precomputed chain of progressively down-scaled copies of a texture, selected by the GPU according to sampling density.
Occlusion culling	An optimization feature that prevents the GPU from rendering objects that are hidden behind other objects.
Octave	Each distinct layer of noise added together in Fractal Brownian Motion, typically doubling in frequency relative to the previous one.
Procedural generation	Algorithmic creation of content using mathematical functions rather than manual creation.
Pseudorandomness	The property of a sequence of numbers that appears random but is generated by a deterministic process, ensuring reproducibility given a specific seed.
SCons	An open-source software construction tool used to configure and build the C++ bindings for Godot GDExtension.
Seed	An initial value used to initialize a pseudo-random number generator, ensuring that the same sequence of numbers (and thus the same terrain) is produced every time.

Terrain	The 3D surface representing the ground or landscape in a virtual environment.
Tessellation	A technique used to dynamically increase the geometric detail of 3D models by subdividing their surfaces into smaller polygons.
Texture array	A single GPU texture object holding several equally sized image slices that can be addressed by an index at sample time, allowing a shader to select between them dynamically.
Thermal erosion	A simulation of mechanical weathering where material slides down steep slopes due to gravity until it reaches a stable angle of repose.
Triplanar mapping	A texturing technique that projects a texture along the three world-space axis planes and blends the three samples by the surface normal, removing the need for explicit UV coordinates.
UV	The two-dimensional coordinate pair that maps a point on a 3D surface to a position within a texture image.
Voronoi diagram	A partitioning of a plane into regions based on distance to points in a specific subset of the plane, often used for biome distribution.
Voxel	A volumetric unit of information representing a value on a regular grid in three-dimensional space, analogous to a pixel in 2D.

Acronyms

AABB	Axis-Aligned Bounding Box.
AO	Ambient Occlusion.
BVH	Bounding Volume Hierarchy.
CBT	Concurrent Binary Trees.
CI/CD	Continuous Integration/Continuous Deployment.
DEM	Digital Elevation Model.
fBm	Fractal Brownian Motion.
GAN	Generative Adversarial Network.
GLSL	OpenGL Shading Language.
LEB	Longest Edge Bisection.
LOD	Level of Detail.
PBR	Physically Based Rendering.
PoC	Proof of Concept.
POM	Parallax Occlusion Mapping.
SSAO	Screen Space Ambient Occlusion.
SSR	Screen Space Reflections.
VTF	Vertex Texture Fetch.

1 Introduction

This project aims to develop a plugin for the **Godot Engine** to allow the [procedural generation](#) of terrains and biomes.

Godot is an open-source game engine that has gained a lot of popularity in recent years thanks to its accessibility, flexibility, and active community. However, despite its many strengths, it still lacks robust built-in tools for large-scale [terrain](#) and [biome](#) generation, which puts it at a disadvantage compared to other commercial engines that already include such features.

Creating a plugin to fill this gap not only expands the creative possibilities for developers using Godot, but also contributes to the growth and adoption of the engine within the wider game development community. It is a way to make Godot even more versatile and to support its ongoing mission of democratizing game development.

Godot's modular architecture makes it an ideal platform for experimentation and extension. It allows developers to easily add new functionality through plugins without modifying the engine's core. For this project, we will use its modern [GDExtension](#) system [\[2\]](#), the successor to **GDNative**, which provides a clean and powerful way to integrate C++ code and create high-performance extensions for the engine.

1.1 Motivation

The motivation for this project is both personal and technical.

As part of Generation Z, I grew up with *Minecraft*. I have played it since its earliest versions, and even back then, I was amazed by how its worlds seemed infinite and different every time. That sense of wonder has stayed with me through the years with games like *Valheim*, *Astroneer* and *Deep Rock Galactic*, all titles I have really enjoyed. They all share something that fascinates me; their worlds are procedurally generated. This approach allows for endless variety and gives rise to emergent stories and different ways of playing, which really resonate with me.

As a programmer with few artistic skills but a strong appreciation for natural landscapes and environments, procedural generation has always felt like the perfect middle ground. It lets me create rich and diverse worlds without having to design every element by hand, combining my love for exploration with my technical side.

Right now, Godot does not have strong tools for procedural terrain and biome generation, which puts it at a disadvantage compared to other engines that already include these features. Creating a plugin that brings this functionality to Godot would not only expand what its users can create but also help the engine grow

and reach more developers.

Another strong motivation for me is Godot's open-source nature. In contrast to the growing number of questionable practices in the game industry, Godot stands out as a community-driven project that prioritizes accessibility and creative freedom. Many of these commercial tools lock essential features behind paywalls or restrictive licenses and hide their source code, which restricts the freedom of developers and limits the possibilities for independent creators. Projects like Godot are essential to democratize game development and make powerful tools available to everyone.

With this project, I want to contribute, even in a small way, to that vision. It is both a personal exploration of one of my favorite areas of game development and an effort to support the open-source community and the democratization of game creation that Godot represents.

1.2 Problem formulation

Terrain generation is a complex task, both from a technical and artistic point of view. Procedural terrain generation has been widely studied, and there are many algorithms and techniques available in academic literature and existing tools. However, integrating these techniques into a user-friendly and engine-native interface is far from trivial, and it remains an area that is often overlooked in many game engines.

In most workflows, terrain generation is carried out using external tools (like World Machine [3], World Creator [4], or Gaia [5]), and the resulting assets are then imported into the game engine. This process can be time-consuming, rigid, and difficult to iterate on, especially for developers working on smaller projects. Alternatively, some developers build custom in-engine solutions from scratch, but this approach requires significant time and technical expertise, often beyond the scope or resources of indie teams.

The goal of this project is to design and develop a plugin for **Godot** that enables users to generate terrains and biomes procedurally within the engine itself. The plugin aims to provide a flexible, modular, and user-friendly interface that allows developers to customize the generation process according to their needs, without relying on external tools.

Publishing the plugin to the Godot Asset Library [6] as open-source will further contribute to the Godot community and support the democratization of game development. By making the project freely available, other developers will be able to learn from, improve, and expand upon it, reinforcing the collaborative spirit that defines the open-source ecosystem.

For me as a developer, this project is also a personal challenge and a learning opportunity. It will allow me to deepen my understanding of procedural generation

techniques and gain insight into the internal architecture of the Godot engine while contributing something meaningful to its community.

1.3 General objectives

The main objective of this Final Degree Project is to **design, implement, and publish a high-performance, open-source plugin for Godot 4** that enables the procedural generation of large-scale terrains and dynamic biomes. This plugin will be developed using **gdextension (C++)** [2] and integrated into the Godot editor with a user-friendly interface.

- (1) Implement a procedural system for terrains and biomes.
- (2) Integrate the system as a plugin for Godot using gdextension.
- (3) Provide a user-friendly interface to configure the generation parameters.
- (4) Validate the quality and performance of the system through testing.
- (5) Document the project and publish it to the Godot Asset Library [6] under the GNU General Public License (GPL) [7].

1.4 Specific objectives

The specific objectives break down the general objectives into concrete, measurable tasks required to build the plugin:

- (1) **To fulfill "Implement a procedural generation system for terrains and biomes":**
 - (a) Do a state-of-the-art review of terrain algorithms (e.g., Perlin/Simplex noise, erosion) and modern rendering techniques (e.g., [clipmap](#), [Level of Detail \(LOD\)](#)).
 - (b) Implement or Integrate robust noise algorithms (like [FastNoiseLite](#) [8]) in C++ to serve as the foundation for [heightmap](#), temperature, and moisture data.
 - (c) Develop the C++ logic to generate [ArrayMesh](#) [9] data (vertices, normals, UVs) from the heightmap data.
 - (d) Design and implement a biome assignment system based on multiple noise parameters (elevation, temperature, rainfall) inspired by existing climate simulation models.
 - (e) Develop an optimized vegetation placement system using GPU [instancing](#) via [MultiMeshInstance3D](#) [10] and implement billboard transitions for distance [LOD](#).

(2) To fulfill "Integrate the system as a plugin for Godot using gdextension":

- (a) Set up the [SCons \[11\]](#) build environment and C++ bindings necessary for a Godot 4 gdextension [\[12\]](#).
- (b) Implement a spatial partitioning system (chunking) to load and unload terrain sections asynchronously based on player proximity.
- (c) Implement a geometric [Level of Detail](#) strategy to reduce [mesh](#) complexity for distant [chunks](#).
- (d) Expose all core C++ functions and custom classes (e.g., `TerrainManager`, `BiomeConfig`) to the Godot API so they can be controlled from the editor or GDScript.
- (e) Generate physics collision bodies (e.g., `HeightMapShape3D` [\[13\]](#)) for each terrain chunk to allow for player interaction.

(3) To fulfill "Provide a user-friendly interface to configure the generation parameters":

- (a) Design and develop a custom `EditorPlugin` [\[14\]](#) that adds a main dock or panel to the Godot editor interface.
- (b) Implement UI components (sliders, curves, color pickers) for interactively modifying all generation parameters, such as noise settings, biome definitions, and [LOD](#) distances.

(4) To fulfill "Validate the quality and performance of the system through testing":

- (a) Analyze and benchmark the plugin's performance (frame rate, chunk loading times, memory usage) using the Godot Profiler and external tools like Tracy [\[15\]](#).
- (b) Conduct a comparative analysis of the performance of the GDExtension (C++) implementation against a baseline GDScript prototype.
- (c) Test the plugin with multiple users to gather feedback on usability, functionality, and identify potential bugs or areas for improvement.

(5) To fulfill "Document the project and publish it to the Godot Asset Library...":

- (a) Write a comprehensive user guide and API documentation explaining the plugin's architecture, classes, and public functions.
- (b) Prepare the plugin for distribution, ensuring it follows all submission guidelines for the Godot Asset Library [\[6\]](#), including a clear license and repository structure.

1.5 Scope of the project

This project is primarily aimed at **game developers**, both independent and professional, who use Godot Engine to create their projects. However, it may also be of interest to developers working with other engines who are curious about procedural terrain and biome generation techniques. The resulting plugin and documentation will serve as a useful reference for anyone exploring similar systems.

Another group of indirect beneficiaries are the **players** of games developed with Godot using this plugin. By enabling developers to create more varied, dynamic, and natural-looking worlds, the project ultimately enhances the player experience, enriching the sense of immersion and exploration in procedurally generated environments.

From an educational and community perspective, this project also aims to **document and explain** the techniques and algorithms used for terrain and biome generation, providing a solid foundation for future developers interested in this field. The open-source nature of the plugin ensures that it can continue to evolve and inspire new contributions long after the initial development phase.

To keep the project focused and achievable, several areas fall outside its scope. It will not cover procedural generation of other game elements such as buildings, characters, or quests, nor will it explore advanced artificial intelligence or physical simulation systems beyond what is required for basic terrain interaction.

Similarly, the plugin will not include manual terrain or biome editing tools typically found in dedicated world editors. Instead, the focus will remain on **automatic procedural generation**, complemented by a set of adjustable parameters that allow users to fine-tune the results to their specific needs.

2 State of Art

In the context of videogame development, **terrain systems** can generally be divided into two main tasks: *creating the terrain data* and *rendering the terrain*. Each of these tasks presents its own set of challenges and requires different techniques and approaches.

Creating terrain data is a broad process that can range from manually sculpting landscapes using tools like *Blender* or *ZBrush*, to complex physical simulations that mimic natural processes such as erosion or sedimentation.

On the other hand, rendering large-scale, detailed terrains in real time is a significant technical challenge. Efficient rendering is mandatory, especially in open-world environments where the goal is to extend the map across kilometers in every direction.

Most importantly, these two aspects, data generation and rendering, are deeply interdependent. One cannot exist meaningfully without the other. If terrain data is generated but cannot be rendered, it remains invisible and useless. Conversely, a powerful rendering system without underlying terrain data has nothing to display beyond a flat, empty surface. This dependency greatly increases the complexity of the problem, as both sides must be addressed together to achieve a *game-ready* solution.

In this chapter, we will review the current state of the art in terrain generation and rendering. This includes a comparison of existing tools on the market (their features, advantages, limitations, and licensing models), modern optimization techniques for large-scale terrain meshes, current real-time rendering approaches, common methods for procedural terrain generation, and finally, the specific tools and capabilities that **Godot** provides to approach these challenges.

2.1 Existing tools and Solutions

Before designing a new solution for procedural terrain generation in Godot, it is essential to review the existing tools and software currently available in the industry. These tools vary widely in their goals, capabilities, and licensing models, and understanding their strengths and limitations provides valuable context for identifying the gap this project aims to address. This section analyses and compares the most widely used terrain-generation and landscape-authoring tools.

Table 2.1: Comparison of popular terrain tools.

Name	Pros	Cons	License
Gaea [16]	Node-based terrain generation; industry standard; advanced erosion + biome tools; strong documentation; perpetual license.	Closed source; not fully procedural; commercial; future licensing uncertain.	Commercial
World Creator [4]	GPU real-time editing; strong procedural tools; bridges to major engines; active modding community.	Expensive; subscription-based; closed source; terrain authoring only.	Commercial
World Machine [3]	Powerful erosion / water simulation; mature graph workflow; strong export formats; free non-commercial tier.	Expensive; CPU thread limitations by tier; closed source; 1 year of updates only.	Commercial
Vue [17]	Advanced atmosphere / volumetric / cloud tools; powerful offline renderer; broad interoperability; VFX-grade.	Discontinued in 2024; heavy; steep learning curve; not for real-time engines.	Formerly commercial; now free / unsupported
Terragen [18]	High-end photoreal rendering; micropolygon displacement; strong atmosphere / cloud tools; node-based.	Offline / render focus; steep learning curve; commercial; not open source.	Free (edu) / subscription / perpetual
True Terrain 5 (Blender) [19]	Integrated into Blender; vegetation scattering; ecosystem tools; erosion-like features.	Blender-only; closed source; terrain authoring, not procedural world generation.	Commercial
A.N.T. Landscape (Blender) [20]	Integrated in Blender; free / open source; great for fast prototyping.	Feature-limited; slow development activity.	Open source (GPLv2)

Continued on next page

Table 2.1: Comparison of popular terrain tools. (Continued)

Name	Pros	Cons	License
BlenderGIS [21]	Imports real-world GIS / elevation datasets; ideal for real terrain workflows; open source.	Blender-dependent; no procedural biome systems.	Open source (GPLv3)
Terrain3D (Godot) [22]	Native terrain for Godot; LOD, heightmaps, grass / instancing; open source; good editor integration.	Limited biome tools; more editing / import than procedural world generation.	Open source (MIT)
TerraBrush (Godot) [23]	Real-time sculpting; paint tools; vegetation placement; open source.	Manual editing focus; lacks biome / procedural systems.	Open source (MIT)
Infinigen [24]	Photoreal procedural generator; great for synthetic datasets; open source research project.	Research-oriented; heavy; lacks engine workflows; no biome assignment system.	Open source (BSD-3-Clause)
TerraForge3D [25]	Open-source, GPU-powered; unlimited resolution; node editor; physically-based erosion; universal exports.	Lacks biome assignment; terrain authoring rather than full world generation.	Open source (MIT)

From the analysis presented in [Table 2.1 $\rightarrow$ p. 21], it is evident that while there are several powerful tools available for terrain generation and authoring, there remains a gap in the market for a fully integrated, open-source solution that combines procedural terrain generation with biome assignment capabilities, specifically tailored for real-time applications in game engines like Godot. Most existing tools either focus on manual terrain editing or lack comprehensive biome systems, highlighting the need for further development in this area.

The main authoring tools right now are Gaea [16], World Creator [4], and World Machine [3]. These tools are widely used in the industry and offer powerful features for terrain generation and editing. However, they are all commercial products with varying licensing models, which may not be accessible to all developers, especially indie developers or hobbyists. Also, they primarily focus on terrain authoring rather than fully procedural world generation with biome assignment. The closed source nature of these tools also limits the ability to customize or extend their functionality for specific project needs.

In the open-source domain, there are several noteworthy projects such as Terrain3D [22] and TerraBrush [23] for Godot, as well as Blender add-ons like A.N.T. Landscape [20] and BlenderGIS [21]. While these tools provide valuable features for terrain creation and editing, they often lack comprehensive biome assignment systems and are more focused on manual editing rather than procedural generation. This is normal in this type of tools, as we can see in other engines like Unity [26] or Unreal Engine [27], where the in-engine terrain tools are more focused on authoring than procedural generation.

2.2 Terrain

There are several types of terrain commonly used in video games. These can be grouped based on how the mesh data is stored. The most common methods are heightmaps, meshes, and *voxel*-based terrain.

These terrain types are compared and contrasted according to the data structure that defines their shape and the geometry used for their representation and rendering.

The following sections compare the main terrain types and geometric structures mentioned above.

Heightmaps

Heightmaps are one of the most commonly used methods for representing terrain in computer graphics and games. A *heightmap* or *heightfield* is a raster image in which each pixel stores a value representing the elevation of a point relative to a base surface. It is typically visualized as a grayscale image, where black represents the minimum height and white represents the maximum height.

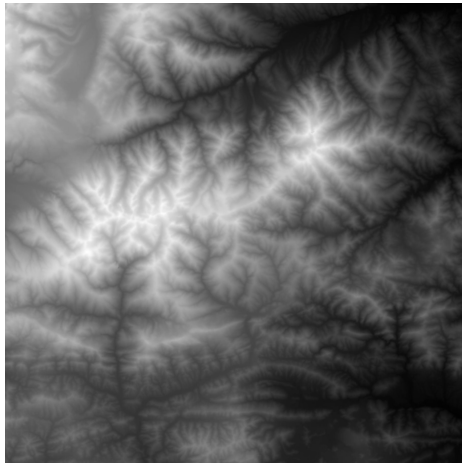
Heightmaps can be directly converted into a 3D mesh for terrain rendering or used in shading techniques such as bump mapping and displacement mapping to modify the geometric position or visual appearance of a surface. The magnitude of displacement can be controlled by the designer, effectively mapping the contrast of the grayscale image to the strength of the height variation.

Although heightmaps are often stored as single-channel grayscale images, their precision can be significantly increased by using additional color channels. A standard RGB 8-bit image can represent only 256 levels of gray and therefore only 256 possible heights. However, by interpreting the three 8-bit channels collectively, a 24-bit image can encode a much wider range: $256^3 = 16,777,216$ possible heights. If the alpha channel is also used (32 bits total), this expands to $256^4 = 4,294,967,296$ possible height values.

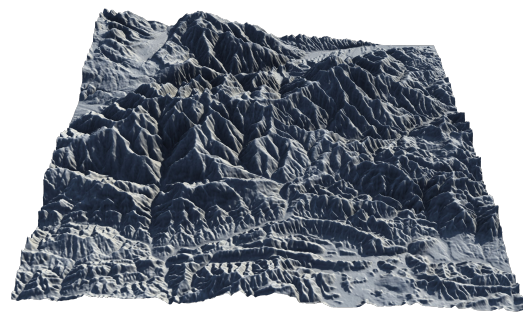
This additional precision is particularly useful for terrains with subtle elevation variations over large areas. When restricted to only 256 grayscale values, height

differences must be quantized into coarse steps, resulting in visible “terracing” artifacts in the rendered terrain.

Heightmaps are widely used in video games, simulation environments, and geographic information systems, where they are known as [Digital Elevation Model \(DEM\)](#).



(a) Example of a heightmap generated using 16 Bit PNG Heightmap Generator [\[28\]](#).



(b) Resulting mesh generated from the heightmap using Gaea.

Figure 2.2: (a) Example of a heightmap. (b) Mesh rendered from the heightmap.

Meshes

A mesh that represents a terrain is not fundamentally different from any other 3D mesh. It is a collection of vertices, edges, and faces that define the shape of the terrain in three-dimensional space. The vertices store information such as their 3D position, normals for shading, UV coordinates for texture mapping, and other useful attributes.

When generating terrain data from an external program, we can use tools that create meshes directly through terrain-sculpting workflows or tools that generate heightmaps using noise algorithms and physical simulations, which are then used to build the mesh. This process was explained earlier in [Section 2.2](#) → p. 23.

In general, meshes are more flexible than heightmaps because they do not suffer from resolution limits and can represent more complex geometry, such as caves.

There are several common ways to obtain terrain meshes:

Manual modeling Using 3D modeling software like Blender, artists can create detailed and highly customized terrains.

Procedural generation Algorithms and techniques such as Perlin noise, Simplex noise, or simulation-based methods can automatically generate terrain. These techniques are discussed in more detail in [Section 2.4](#) → p. 35.

Photogrammetry and laser scanning These techniques allow capturing real-world terrain and converting it into detailed 3D meshes. They have become increasingly popular in the game industry in recent years. Games like *Senua's Saga: Hellblade II* [\[29\]](#) use photogrammetry to capture realistic environments with high fidelity. The process involves taking multiple photos of a location from different angles and using specialized software to reconstruct its 3D geometry. Although this technique falls outside the scope of this project, since the focus here is procedural generation, it is important to acknowledge its relevance in the creation of realistic environments.

Engine-specific terrain systems Some game engines and development tools include built-in systems for creating and editing terrains directly within the editor, simplifying the workflow by removing the need for external software such as those mentioned in [Section 2.1](#) → p. 20. Examples include the terrain systems from Unity [\[26\]](#) and Unreal Engine [\[27\]](#).

Choosing between heightmaps and meshes depends on the specific requirements of the project. Heightmaps are better suited for dynamically adjusting the terrain's topography, while meshes offer more flexibility for complex or highly specific features. Heightmaps also provide advantages such as the ability to encode multiple types of data into different color channels, such as height, texture information, or moisture, which can be useful for procedural biome generation.

In terms of performance, both methods should behave similarly when implemented correctly. Once the meshes are stored in memory, they are essentially the same. The main difference is the cost of generating the mesh from a heightmap, which is usually negligible if done only once at the start of the game.

Voxel-based terrain

A voxel is a value stored on a three-dimensional grid, similar to how a pixel represents information on a two-dimensional image. This concept is commonly defined in the literature as a volumetric unit of information used to represent values in a 3D regular grid. Unlike heightmaps, which are essentially 2D maps with height values, voxel-based terrain operates in full 3D space. This allows it to represent complex structures such as caves, overhangs, tunnels, and destructible environments.

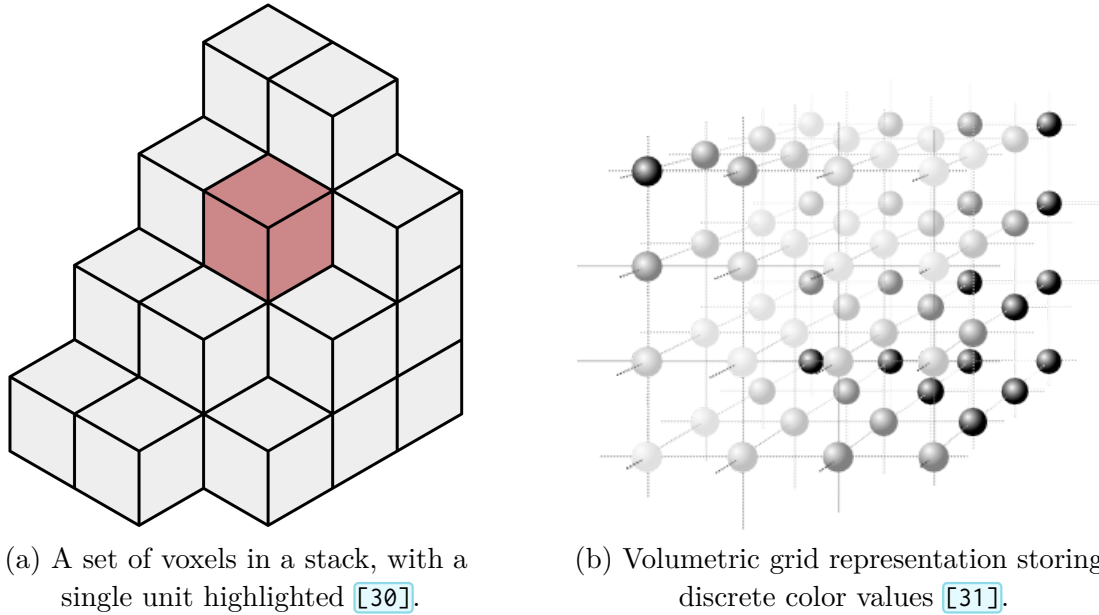


Figure 2.3: Visual representation of Voxel data structures.

To handle the large amount of data involved, voxel terrains are usually divided into smaller sections called “chunks”. This technique makes it possible to load and unload only the necessary parts of the world, improving memory usage and performance. While chunking is not exclusive to voxel systems, it is especially important here due to the volumetric nature of the data.

Voxel terrain is used in several well-known games. *Minecraft* is the most iconic example, featuring an entirely voxel-based world that can be mined, built, and modified in real time. Other titles, such as *No Man’s Sky*, also rely on voxel data internally to support complex terrain deformation and large procedural worlds [32].

Since voxel data cannot be rendered directly, it must be converted into polygons through a process known as *polygonization*. Common algorithms for this task include *Marching Cubes* and *Dual Contouring*, which generate meshes from the voxel grid so the terrain can be displayed efficiently by the GPU.

One of the main advantages of voxel-based terrain is real-time terraforming. Because the world is represented as a 3D grid of volumetric data, modifying the terrain, such as mining, digging, destroying structures, or sculpting the environment, is straightforward and can be done dynamically during gameplay.

A voxel-based terrain system is outside the scope of this project, which focuses on heightmap and mesh-based procedural generation. However, it is important to understand the characteristics and advantages of voxel systems, as they represent a powerful alternative for games that require fully editable or volumetric worlds.

Comparison

Before selecting a terrain representation method for a game or simulation, it is important to understand the trade-offs between the different approaches. Although heightmaps, meshes, and voxel-based terrain can all be used to represent large environments, they differ significantly in flexibility, memory usage, performance characteristics, and the type of geometry they can encode.

Table 2.4^{→ p. 27} summarizes the main advantages and limitations of each method, providing a high-level comparison that can help determine which technique is most appropriate depending on the needs of the project.

Table 2.4: Comparison of terrain representations in videogames.

Name	Pros	Cons	Structure
Heightmaps	Simple to store and generate; efficient for large open worlds; fast rendering and LOD generation; easy to edit and sculpt.	Limited to 2.5D surfaces (no caves or overhangs); requires external techniques for fine detail; resolution-dependent.	2D grid of height values
Meshes	High geometric flexibility; supports overhangs, caves, cliffs; integrates well with modelling tools; suitable for handcrafted or hybrid workflows.	More expensive to generate and render; requires careful LOD and optimization; editing can be complex at scale.	Arbitrary polygon mesh
Voxel Terrain	Fully volumetric; supports destructible worlds, caves, overhangs, mining; easy boolean operations; good for procedural generation.	High memory cost; requires meshing algorithms (e.g., Marching Cubes); performance-heavy for large worlds; Complex to implement.	3D grid of volumetric cells

As shown in Table 2.4^{→ p. 27}, no single technique is universally superior; instead, each terrain representation offers specific strengths that make it more suitable for certain applications. Heightmaps excel in simplicity and efficiency, meshes provide the greatest geometric flexibility, and voxels enable fully editable volumetric worlds. The choice ultimately depends on the requirements of the project, such as the need for real-time editing, support for complex structures, or memory constraints.

2.3 Modern Terrain Rendering Techniques

The rendering of vast, open-world environments stands as one of the most resource-intensive tasks in real-time computer graphics. The primary challenge lies in the dichotomy between the massive scale of the dataset and the finite limits of system memory and processing power. Attempting to render a high-fidelity terrain using a static, brute-force mesh approach is computationally infeasible, as the geometric density required for close-up details would result in an excessive vertex count when applied to the entire landscape.

To achieve photorealistic visuals at interactive frame rates, it is strictly necessary to adopt dynamic, *view-dependent* algorithms. These techniques aim to distribute computational resources intelligently, allocating high fidelity only to the areas immediately visible to the observer while aggressively simplifying distant regions.

This section analyzes the state-of-the-art methodologies designed to solve this scalability problem.

Clipmaps

Clipmaps were introduced by Tanner, Migdal, and Jones at SIGGRAPH 1998 [34]. The original presentation is based on their paper *The Clipmap: A Virtual Mipmap* [33]. As they describe in their paper, “We describe the clipmap, a dynamic texture representation that efficiently caches textures of arbitrarily large size in a finite amount of physical memory for rendering at real-time rates.”

First of all, what is a mipmap? A *mipmap* is a sequence of pre-calculated, optimized textures, each of which is a progressively lower resolution representation of the same image. Mipmaps are used in 3D computer graphics to increase rendering speed and reduce artifacts. When a texture is applied to a 3D surface, the appropriate **Level of Detail** from the mipmap is selected based on the distance from the viewer and the angle of the surface relative to the camera. This helps to minimize aliasing and improve performance by reducing the amount of texture data that needs to be processed. This technique is called mipmaping.

So, what are clipmaps? Tanner, Migdal, and Jones explain, a clipmap is a hierarchical representation of a texture that allows for efficient rendering of large textures by only loading the necessary portions into memory based on the viewer’s position and orientation. Clipmaps are particularly useful for rendering large terrains, where the texture data can be much larger than the available memory on the graphics card.

In the SIGGRAPH 2004, Losasso and Hoppe presented *Geometry Clipmaps: Terrain Rendering Using Nested Regular Grids* [35], which extends the concept of clipmaps to terrain rendering. They describe a method for rendering large terrains using a hierarchical grid structure that allows for efficient **LOD** management and rendering. As they describe in their abstract, “Rendering throughput has

reached a level that enables a novel approach to *Level of Detail* control in terrain rendering. We introduce the geometry clipmap, which caches the terrain in a set of nested regular grids centered about the viewer. The grids are stored as vertex buffers in fast video memory, and are incrementally refilled as the viewpoint moves. This simple framework provides visual continuity, uniform frame rate, complexity throttling, and graceful degradation. Moreover it allows two new exciting real-time functionalities: decompression and synthesis. Our main dataset is a 40GB height map of the United States. A compressed image pyramid reduces the size by a remarkable factor of 100, so that it fits entirely in memory. This compressed data also contributes normal maps for shading. As the viewer approaches the surface, we synthesize grid levels finer than the stored terrain using fractal noise displacement. Decompression, synthesis, and normal-map computations are incremental, thereby allowing interactive flight at 60 frames/sec.”. Also presenting a video demo showing the technique in action [36].

Hoppe’s contribution in *GPU Gems 2* [37] represents a paradigm shift in terrain rendering: moving the workload from the CPU (updating mesh buffers) to the GPU (processing textures). To understand how this works in practice, we can break the pipeline down into three distinct stages: Data Representation, Geometry Structure, and Shader Execution.

Data Representation: The Clipmap Pyramid

The core concept of the Geometry Clipmap is to treat the terrain elevation not as a mesh, but as a 2D signal (an image). The system virtually represents the terrain as a mipmap pyramid of L levels, where Level 0 is the finest resolution and Level $L - 1$ is the coarsest.

However, for massive terrains (e.g., the entire United States), storing the full pyramid in memory is impossible. Instead, the system caches only a fixed-size square window of $N \times N$ samples for each level, centered around the viewer.

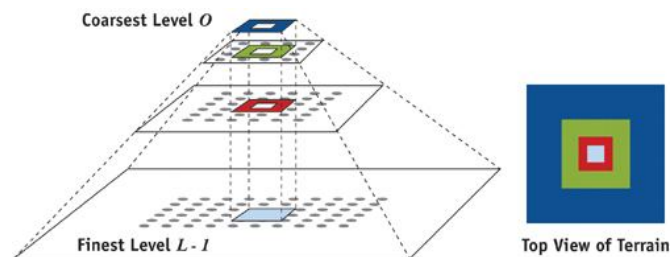


Figure 2.5: How Geometry Clipmaps Work [37].

As illustrated in [Figure 2.5](#), as the viewer moves, these cached windows shift. This requires an efficient update mechanism to ensure data is always available without stalling the rendering pipeline.

Geometry Layout: Nested Rings

To render this data efficiently, the algorithm does not draw a simple grid of squares. Instead, it utilizes a **Nested Ring Structure**.

- **Level 0 (Fine):** The area immediately beneath the viewer is rendered as a solid grid.
- **Levels 1 to $L - 1$ (Coarse):** These levels are rendered as hollow rings surrounding the finer levels.

This “donut” shape ensures that high-density triangles are only used near the camera, while distant areas use larger, lower-density triangles. By nesting these rings, the screen-space size of the triangles remains roughly uniform, optimizing vertex processing.

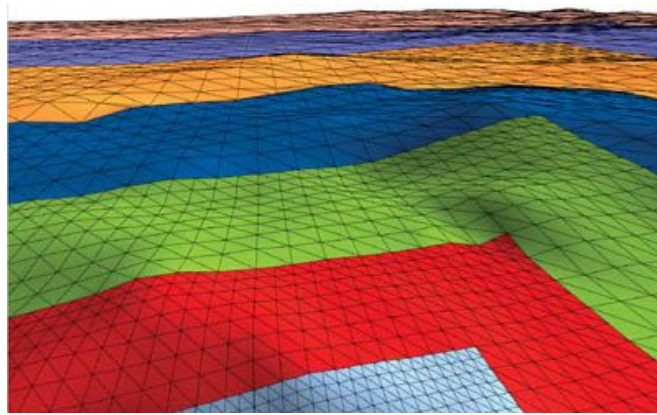


Figure 2.6: Terrain Rendering Using a Coarse Geometry Clipmap [37].

Pipeline Execution: Vertex Texture Fetch

The actual construction of the 3D terrain happens inside the GPU’s Vertex Shader. This relies on a technique known as **Vertex Texture Fetch (VTF)**. The workflow is as follows:

- (1) **Instancing:** The CPU sends a generic, flat 2D grid mesh to the GPU. This same mesh is reused (instanced) for every level of the clipmap, scaled appropriately.
- (2) **Sampling:** Inside the Vertex Shader, the grid vertices use their (x, y) coordinates to sample the **Elevation Map** texture.
- (3) **Displacement:** The shader reads the pixel value (height) from the texture and modifies the vertex’s z position.
- (4) **Visual Continuity:** To prevent gaps between the different ring levels, the shader calculates a transition blend. Vertices near the outer edge of a ring are smoothly “morphed” to match the elevation of the next coarser ring.

Memory Management: Toroidal Update

Finally, to handle infinite movement, the textures are updated using **Toroidal Addressing**. The elevation maps act as circular buffers. When the camera moves, the CPU only uploads the new “L-shaped” strip of data that enters the view. The texture coordinates wrap around, meaning the physical memory footprint remains constant regardless of how far the player travels.

Level of Detail (LOD)

In large-scale terrain rendering, rendering the entire world at maximum resolution is computationally impossible. **LOD** is the technique of reducing the complexity of a 3D model representation as it moves away from the viewer.

While traditional **LOD** techniques involve swapping discrete static meshes (e.g., LOD0, LOD1), terrain rendering requires a more fluid approach to prevent visual “popping” artifacts. Modern engines use **Continuous LOD (CLOD)** to solve this. One of the most well-known methods is **Continuous Distance-Dependent Level of Detail (CDLOD)**, introduced by Strugar [38] as an improvement over geometry clipmaps.

This technique addresses a critical limitation of the original Geometry Clipmap algorithm [Section 2.3 → p.28]: the dependency on 2D coordinates. As Strugar explains, basic clipmaps determine the **Level of Detail** based primarily on the horizontal position (x, y) of the viewer, often ignoring altitude (z) . This leads to inefficient mesh distribution; for instance, an observer flying at a high altitude would still process high-resolution geometry directly below them, despite it being far away visually.

Furthermore, previous attempts to approximate **LOD** on a per-chunk basis (applying the same detail level to a whole block of mesh) resulted in visual artifacts on uneven terrain and integration difficulties with physics engines. Since the terrain geometry would shift unpredictably between levels, objects placed on the ground could appear to “float” or intersect the mesh.

CDLOD solves these problems by selecting the **LOD** level as a function of the precise **3D distance** from the camera to every individual vertex, rather than per chunk or 2D grid. This ensures a predictable, artifact-free distribution of detail that remains consistent regardless of the camera’s height or the terrain’s roughness.

Another drawback of the original clipmap approach, is that the discontinuities between **LOD** levels require additional work to remove the gaps (T-junctions) between different levels of detail and can cause issues like artifacts and overdraw. **CDLOD** inherently avoids these problems because the algorithm used to transition between **LOD** levels completely transforms the mesh of a higher level into the lower detailed one before the actual swap occurs. This ensures a perfectly smooth transition with no seams or artifacts. In addition, the rendering itself is simpler

and more predictable, as only one rectangular regular grid mesh is needed to render everything.

CDLOD partitions the heightmap into a fixed-depth quadtree and selects nodes at runtime so on-screen triangle density remains approximately constant with distance. Rendering is performed by drawing areas covered by the selected nodes with a single regular grid; vertex positions are displaced in the vertex shader by sampling the heightmap, and finer nodes are smoothly morphed into coarser ones in the shader to avoid popping. Each quadtree node stores min / max heights for its covered region. A straightforward implementation keeps explicit node structures, while streaming variants use compressed min / max maps and implicit data generation to reduce memory usage. The quadtree is built from the source heightmap and only needs rebuilding if that heightmap changes.

Tessellation

Tessellation is a technique used in computer graphics to dynamically increase the geometric detail of 3D models by subdividing their surfaces into smaller polygons. As we can see from previous sections, the main challenge in terrain rendering is how we can represent large-scale environments efficiently while maintaining visual fidelity. We have seen so far that techniques like clipmaps and CDLOD are effective methods that can set the **LOD** levels based on the distance from the camera. However, these methods often require significant CPU intervention to manage the subdivision state. With the introduction of hardware tessellation in modern GPUs, we can shift this workload directly to the graphics card, making the process significantly more efficient.

Dupuy in “Concurrent Binary Trees (with Application to Longest Edge Bisection)” [39] present a method for real-time terrain tessellation using **Longest Edge Bisection (LEB)**. This technique takes advantage of the GPU hardware tessellation capabilities to dynamically refine terrain meshes based on the viewer’s distance and angle, utilizing a novel data structure called **Concurrent Binary Trees (CBT)** to manage the subdivision state in parallel.

Longest Edge Bisection

The **Longest Edge Bisection** algorithm is a recursive subdivision technique. As described by Dupuy, **LEB** “consists in recursively splitting triangles in two along their longest edge” [39]. This process creates a binary hierarchy of triangles that can be traversed to achieve the desired **Level of Detail**.

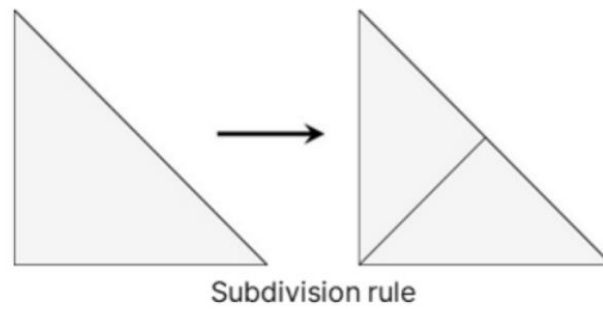


Figure 2.7: Longest Edge Bisection subdivision example [39]

A key advantage of this method is its ability to handle adaptive tessellation without artifacts. Unlike quadtree-based approaches, which often require complex "stitching" to avoid cracks between chunks of different resolutions, LEB produces *conforming tessellations* by construction. Dupuy state that:

“When applied adaptively, LEB produces conforming triangle tessellations, i.e., free of T-junctions, whenever it is constrained so as to guarantee that no neighboring triangles differ by more than one subdivision level.” [39]

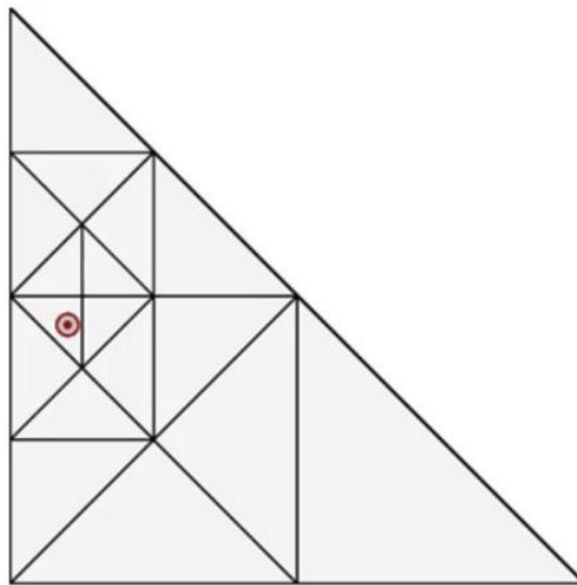


Figure 2.8: Example of recursive non-uniform adaptive subdivision [39]

This property makes LEB fundamentally suited for building multi-resolution terrains, as it ensures a watertight mesh without the need for additional geometry skirts or heavy CPU post-processing.

Link to Binary Trees

A fundamental observation in modern terrain rendering is the mapping between the subdivision process and binary trees. Dupuy note that [LEB](#) is a “*self-similar, binary process*” [\[39\]](#), which allows the problem of computing geometry to be transposed into modifying the topology of a binary tree.

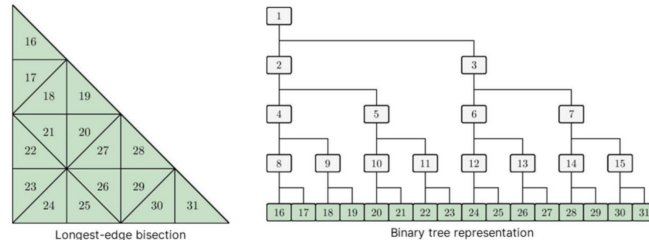


Figure 2.9: Representation of the link to Binary Tree [\[39\]](#)

In this representation:

- The root node represents the base geometry (the initial coarse triangle).
- Each node corresponds to a specific triangle produced by the subdivision.
- The two children of a node represent the two new triangles resulting from a split.
- The leaf nodes of the tree constitute the actual geometry rendered to the screen.

Dupuy leverages this link to introduce the [Concurrent Binary Trees](#). Instead of using pointer-based structures, which are inefficient on GPUs, the [CBT](#) uses a *binary heap* (a 1D array) and a bitfield to encode the tree topology. This allows threads to split and merge nodes concurrently using atomic bitwise operations, enabling massive parallelism for large-scale terrain rendering.

Occlusion Culling

[Occlusion culling](#) is a performance optimization technique used in 3D graphics to avoid rendering objects that are not visible to the camera because they are blocked (occluded) by other objects. By not rendering these hidden objects, the graphics engine can save valuable processing power and improve frame rates, especially in complex scenes with many objects.

This technique is mandatory, independently of the terrain rendering method used, to ensure that only visible parts of the terrain and scene are processed and rendered. With previous methods like clipmaps, CDLOD, and tessellation, we can partition the terrain into smaller sections or chunks. Occlusion culling can then be applied to these chunks to determine which ones are visible from the camera’s perspective.

Other methods for spatial partitioning, such as *Binary Space Partitioning (BSP) trees*, *Octrees*, and [Bounding Volume Hierarchy \(BVH\)](#), can also be used to organize the terrain data efficiently.

How this is implemented depends on the specific engine and rendering pipeline. Some engines have built-in occlusion culling systems, while others may require custom implementations. Techniques such as *Hierarchical Z-Buffering*, *Portal Culling*, and *Occlusion Queries* are commonly used to achieve effective occlusion culling in real-time applications.

2.4 Procedural Terrain Generation Methods

As described in [Section 2.2](#)^{p. 23}, procedural terrains are often generated from heightmap textures. These heightmaps are commonly created using noise algorithms or physically inspired simulation techniques. This section provides an organized overview of the main categories and methods used to generate procedural heightmaps, their characteristics, and typical applications.

[Pseudorandomness](#) is a concept worth noting in this context, as many noise algorithms rely on pseudorandom number generation to produce their results. Pseudorandomness is the property of a sequence of numbers that appears to be random but is actually generated by a deterministic process. This is why in games like *Minecraft* or *Valheim*, the same [seed](#) will always produce the same world.

Noise Basis Functions

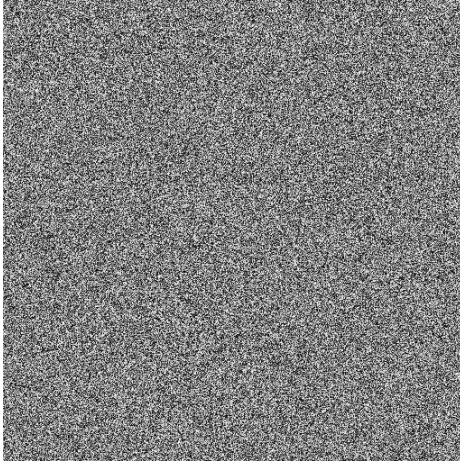
Noise basis functions serve as the fundamental building blocks for procedural terrain generation. Unlike chaotic white noise, which produces abrupt and uncorrelated values, these mathematical functions are designed to generate *coherent noise*. This property ensures that values change smoothly across space, allowing for the simulation of continuous natural phenomena such as rolling hills, cloud patterns, and marble textures.

These functions are typically computationally efficient and allow for random access, meaning the value at any point (x, y) can be calculated independently without generating the entire dataset.

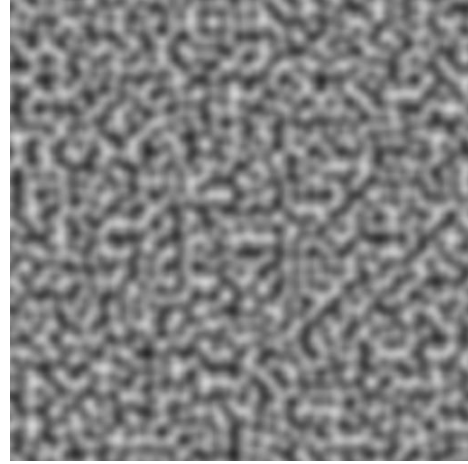
Perlin Noise

Introduction Perlin noise was developed by Perlin in 1985 [\[40\]](#). Ken Perlin created this algorithm after working at Disney's film *Tron* for the animation studio *MAGI*.

Unlike simple white noise, which is chaotic and abrupt, Perlin noise is a type of [gradient noise](#). It generates a texture that appears pseudo-random but maintains visual coherence and smoothness, mimicking the randomness found in nature.



(a) Example of white noise.



(b) Example of perlin noise.

Figure 2.10: (a) Example of white noise. (b) Example of perlin noise.

Functioning The generation of Perlin noise can be broken down into three fundamental steps:

- (1) **Grid Definition:** The algorithm begins by defining a grid of regular cells. At each node (corner) of the grid, a pseudorandom gradient vector of unit length is assigned. These vectors determine the slope of the noise at the grid intersections.

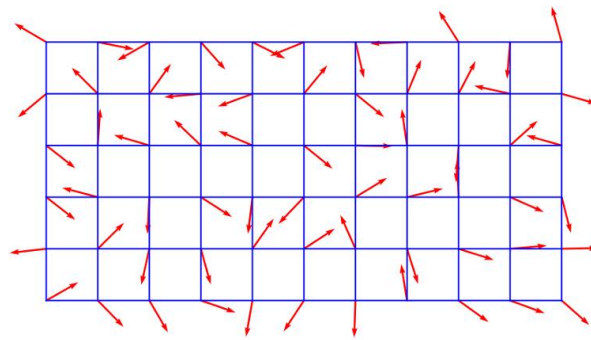


Figure 2.11: A two-dimensional grid of gradient vectors [41]

- (2) **Dot Product Calculation:** For any given point P within a grid cell, the algorithm calculates the four distance vectors from the point to the cell's corners. Then, it computes the dot product between each distance vector and the corresponding gradient vector at the corner. This projects the gradient influence onto the point's position.

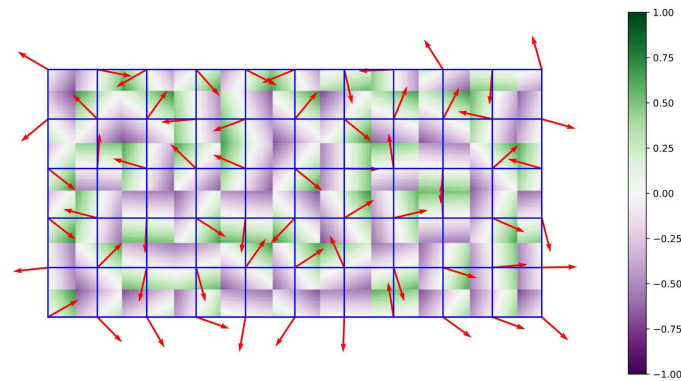


Figure 2.12: The dot product of a point with its nearest grid node gradient value. The dot product determines the ramp intensity relative to that corner [42]

(3) **Interpolation:** Finally, the four dot product values obtained in the previous step are blended together. To ensure that the transitions between grid cells are smooth and continuous (avoiding visual artifacts), a smooth interpolation function is used, typically a quintic curve ($6t^5 - 15t^4 + 10t^3$) rather than simple linear interpolation.

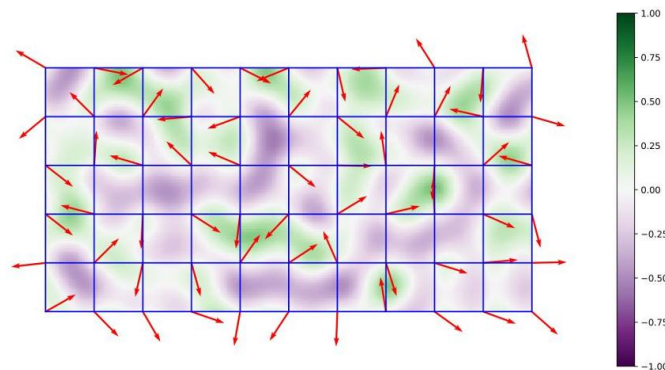


Figure 2.13: The final interpolated result showing the smooth gradient noise distribution [43]

Characteristics and Applications The most notable characteristic of Perlin noise is its *band-limited* nature; it contains details of a specific size or frequency. By combining multiple layers of Perlin noise at different frequencies and amplitudes (a technique known as [Fractal Brownian Motion](#) or *octave*), developers can generate complex structures.

It is widely used in Computer Graphics to generate:

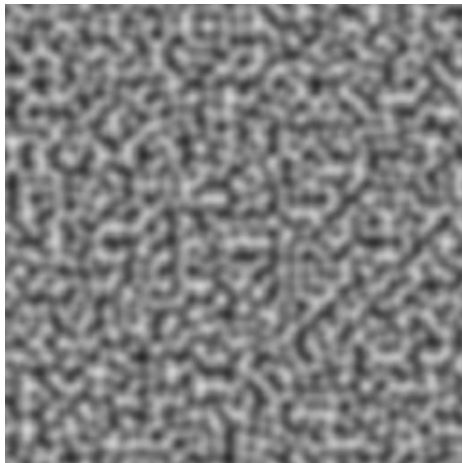
- **Natural textures:** Such as marble, wood, clouds, and fire.
- **Terrain Heightmaps:** Creating "realistic" hills and valleys rather than chaotic spikes.

- **Animation:** Adding natural movement to camera shakes or water ripples.

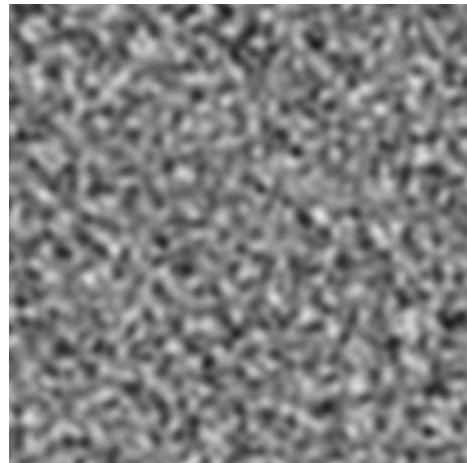
Simplex Noise

Introduction Simplex noise was also developed by Perlin in **2002** [44] as an improvement over his original Perlin noise algorithm. The motivation behind Simplex noise was to address some of the limitations of Perlin noise. The advantages of simplex noise over Perlin noise include:

- Simplex noise has lower computational cost and requires fewer multiplications.
- Simplex noise scales to higher dimensions (4D and 5D) with much less computational cost. The complexity is $O(n^2)$ compared to $O(n2^n)$ for Perlin noise.
- Simplex noise has no noticeable directional artifacts.
- Simplex noise has a continuous gradient everywhere.
- Simplex noise is easy to implement in hardware.



(a) Example of perlin noise.



(b) Example of simplex noise.

Figure 2.14: (a) Example of perlin noise. (b) Example of simplex noise.

Functioning Simplex noise operates on a different geometric structure compared to Perlin noise. Instead of using a square grid, Simplex noise uses a simplex grid, which is a generalization of triangles and tetrahedra to higher dimensions. In 2D, the simplex grid consists of equilateral triangles, while in 3D, it consists of tetrahedra.

Characteristics and Applications Simplex noise shares many of the same applications as Perlin noise, however, its improved performance and scalability make it better suited for real-time applications and higher-dimensional noise generation.

OpenSimplex Noise

Introduction OpenSimplex noise, was developed by Kurt Spencer in 2014. This algorithm was created in order to overcome the patent-related limitations of Simplex noise. This algorithm addresses the same issues as simplex noise, such as reducing directional artifacts and improving performance in higher dimensions. It shares numerous similarities with simplex noise [Section 2.4 → p.38], but it has two primary differences:

- Unlike Simplex noise, which forms its grid by "squashing" a hypercubic lattice along the main diagonal, OpenSimplex employs a different lattice construction based on a stretched hypercube. Practically, this results in a different geometric subdivision of space. For instance, while 3D Simplex relies on a purely tetrahedral subdivision, 3D OpenSimplex utilizes a *tetrahedral-octahedral honeycomb* structure to fill the space.
- OpenSimplex noise uses a larger kernel size than simplex noise. This is noticeable in smoother appearance at the cost of performance.

Functioning Conceptually, the execution pipeline of OpenSimplex mirrors that of Simplex noise.

Characteristics and Applications Its primary application is to serve as a **patent-free alternative** to Simplex noise. It is the standard choice in open-source game development, terrain generation libraries, and procedural texture tools where legal safety regarding the 2001 Simplex algorithm patent is a priority.

Voronoi Noise

Introduction A [Voronoi diagram](#) is a mathematical method for partitioning a plane into specific regions based on distance to a set of points. Within the context of procedural generation, this technique is often referred to as *Cellular Noise*. It can be classified as a type of tessellation [Section 2.3 → p.32], as it divides the space into discrete polygons known as *Voronoi cells* or *Thiessen polygons*.

Functioning The fundamental principle of the algorithm relies on a finite set of points in the space, formally called *seeds*, *sites*, or *generators*.

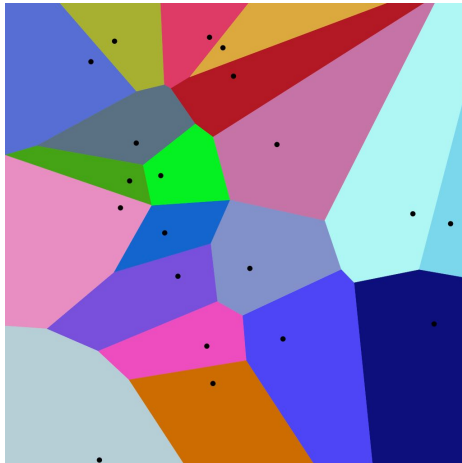
For any given point x in the Euclidean plane, the algorithm identifies the nearest seed. A specific region (Voronoi cell) R_k associated with a seed P_k is defined as the set of all points whose distance to P_k is less than or equal to their distance to any other site P_j . This relationship is formally expressed as:

$$R_k = \{x \in X \mid d(x, P_k) \leq d(x, P_j) \text{ for all } j \neq k\} \quad (2.15)$$

In a procedural rendering context (such as a shader), the "noise value" is typically the scalar distance $d(x, P_k)$ itself.

The visual result depends heavily on the distance metric (d) employed:

- **Euclidean distance:** Produces organic, rounded cellular shapes.
- **Manhattan distance:** Results in blocky, diamond-shaped, or square regions, giving a more mechanical appearance.



(a) Euclidean distance [45].



(b) Manhattan distance [46].

Figure 2.16: (a) Euclidean distance. (b) Manhattan distance.

Characteristics and Applications Unlike Perlin or Simplex noise, which produce smooth, wavelike gradients, Voronoi noise is characterized by its distinct cellular structure and hard edges (at the boundaries where the nearest seed changes).

This makes it particularly suitable for simulating natural patterns that exhibit discrete partitioning, such as:

- **Geological features:** Cracked mud, dried soil beds, and stone paving patterns.
- **Biological structures:** Reptile scales, cell tissues, and bone microarchitecture.
- **Terrain features:** While rarely used for base terrain height, it is often used to modify terrain to create sharp mountain peaks or to determine biome distribution zones.

Furthermore, in 3D applications, Voronoi partitioning is extensively used for fracturing geometry (creating breakable objects) and for optimizing point queries, such as finding the nearest resource or obstacle in navigation meshes.

Worley Noise

Introduction Worley noise, introduced by Steven Worley in 1996 as an extension of the Voronoi diagram, is a procedural texturing basis function widely referred to as *cellular noise*. Unlike Gradient Noise, which simulates continuous waves,

Worley noise models the distances to randomly distributed feature points (seeds) in space. The algorithm was specifically designed to efficiently approximate the geometric properties of Voronoi diagrams within a rendering shader, avoiding the high computational cost of traditional tessellation methods by generating feature points "on the fly".

Functioning The algorithm operates by dividing space into a regular grid, similar to the approach used in Gradient Noise. However, instead of assigning gradients to corners, a random number of feature points (seeds) are generated within each grid cell.

To determine the noise value for any sample point P in space:

- (1) The algorithm identifies the grid cell containing P .
- (2) It then calculates the distance from P to the feature points located in that cell and its immediate neighbors (in 2D, this involves checking the 3×3 block of cells).
- (3) The algorithm sorts these distances and outputs the n -th closest distance, denoted as F_n . Typically, F_1 (distance to the closest point) is used for standard cellular looks.

This grid-based optimization ensures that the complexity remains constant $O(1)$ regardless of the total number of feature points in the infinite domain.

Characteristics and Applications The visual output of Worley noise is defined by the mathematical combination of the distances (F_n). While originally intended for textures like stone or water caustics, in the context of terrain generation, it serves three critical functions:

- **Biome Distribution:** The discrete nature of the cells (using the seed ID rather than the distance) creates organic regions with hard boundaries. This is the standard method for creating biome maps, separating, for example, a desert area from a forest area.
- **Mountain Peaks:** By inverting the closest distance ($1 - F_1$), the algorithm produces cone-shaped gradients. Unlike the rolling hills of Perlin noise, this creates sharp, scattered peaks, suitable for modeling young mountain ranges or volcanic islands.
- **Canyons and Cracks:** The metric $F_2 - F_1$ (the difference between the second and first closest points) highlights the boundaries between cells. This produces a web-like structure used to simulate dried mud, tectonic plate edges, or river canyon networks.

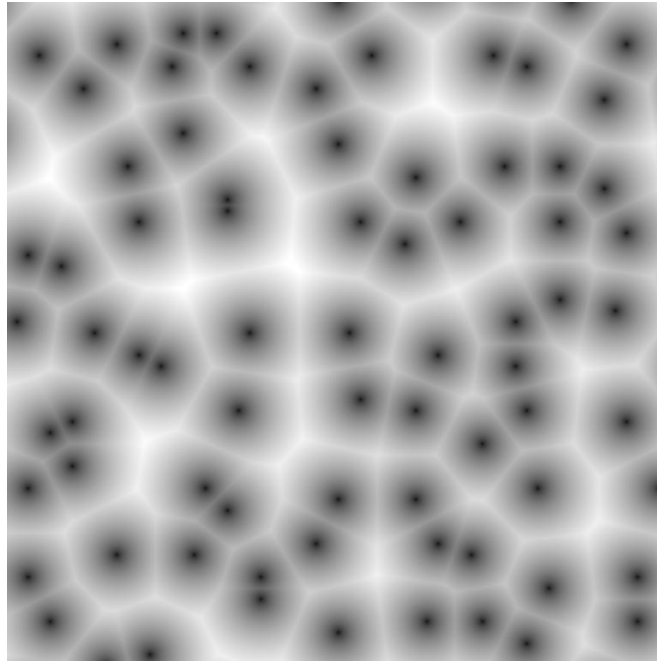


Figure 2.17: Example of Worley noise.

Subdivision Methods

While noise functions allow for random access evaluation at any point in space, subdivision methods operate through an iterative process of refinement. These algorithms typically begin with a coarse geometric shape or a low-resolution grid and recursively add detail to generate complex, fractal-like structures. Historically, these techniques were among the first used to render convincing landscapes due to their efficiency in generating self-similar topology without the need for complex basis functions.

Diamond-Square Algorithm

Introduction The Diamond-Square algorithm is a stochastic method for generating fractal heightmaps, introduced by Fournier et al. at SIGGRAPH 1982 [47].

Often referred to as the "plasma fractal" or "cloud fractal" due to its visual characteristics, it was one of the first algorithms capable of generating convincing terrain features with relatively low computational cost.

Functioning The algorithm operates on a square grid of size $(2^n + 1) \times (2^n + 1)$. This specific size is required to ensure that the recursive subdivision works correctly down to the pixel level. The process begins by initializing the four corners of the grid with seed values and then iterates through two alternating steps:

- (1) **The Diamond Step:** For each square in the grid, the algorithm locates the midpoint. The height of this midpoint is calculated by averaging the

heights of the four corner points of the square and adding a random value (perturbation). This creates a diamond shape (rotated square) centered within the original square.

- (2) **The Square Step:** For each diamond created in the previous step, the algorithm calculates the midpoint (which lies on the edge of the previous squares). This value is the average of the four surrounding points (top, bottom, left, right) plus a random value.

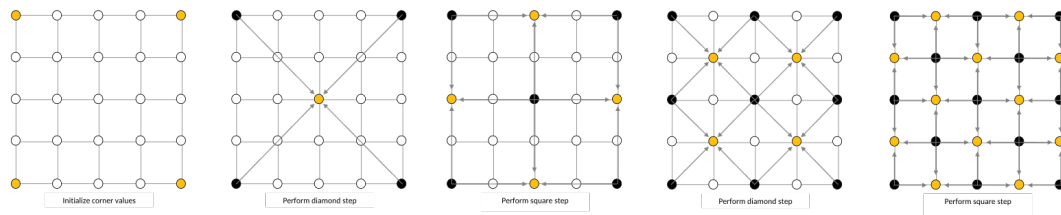


Figure 2.18: Visualization of diamond-square algorithm on a 5×5 array [48].

A complication arises during the Square Step for points located on the edges of the array, as they lack a fourth neighbor. This is commonly solved by wrapping around to the opposite side of the array (creating tilable terrain) or by averaging only the three available neighbors.

Characteristics and Applications The Diamond-Square algorithm is computationally efficient and produces terrain that exhibits self-similarity, a key property of natural landscapes. However, it has significant drawbacks compared to modern noise functions (like Perlin or Simplex):

- **Artifacts:** The algorithm tends to produce noticeable vertical and horizontal "creases" or straight lines. This happens because early random perturbations propagate through the recursion, aligning with the grid axes.
- **Non-Local Generation:** Unlike Perlin noise, which can calculate the height of any point (x, y) independently ($O(1)$), Diamond-Square requires generating the entire grid at once. This makes it difficult to use for infinite terrain streaming or GPU parallelization.

Despite these limitations, it remains a pedagogical staple and is used in applications requiring generated finite maps, such as heightmaps for strategy games or early versions of landscape software like Terragen.

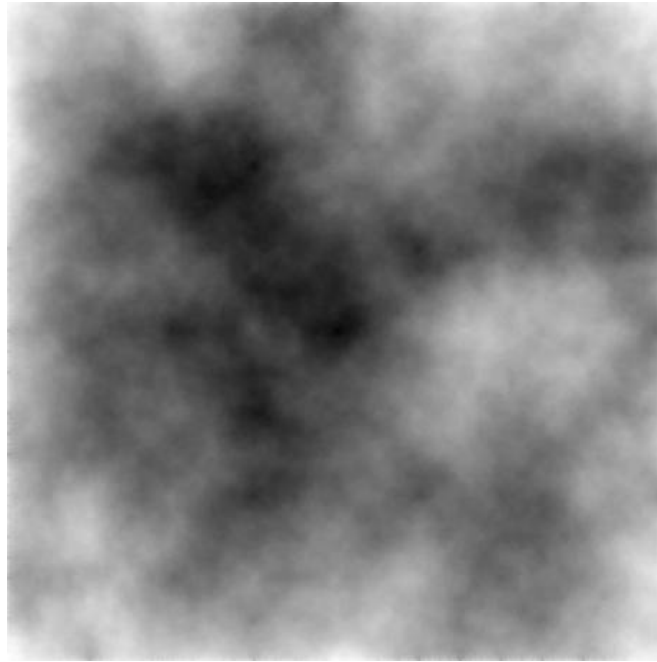


Figure 2.19: Example of diamond-square generated heightmap [49].

Noise Layering and Manipulation

The noise basis functions discussed previously provide the essential stochastic building blocks, but their raw output is rarely sufficient to generate convincing landscapes on their own. A single layer of gradient noise typically creates smooth, rolling undulations that lack the fractal roughness and multi-scale detail characteristic of real-world topology.

To bridge the gap between simple gradient generation and realistic terrain synthesis, these basis functions must be processed through layering and mathematical manipulation techniques. By combining multiple passes of noise at varying frequencies and amplitudes (a concept rooted in fractal geometry) it is possible to simulate complex structures ranging from jagged mountain peaks to fine-grain surface details. Furthermore, by manipulating the input coordinates rather than just the output values (a technique known as [domain warping](#)), developers can introduce turbulence and distort the underlying grid structure, resulting in significantly more organic and unpredictable formations.

Fractal Brownian Motion (fBm)

Basic Concept From a practical graphics programming perspective, as described by Quilez [50], [fBm](#) is not a complex mathematical formula, but rather a method of constructing self-similarity through iteration.

The core idea is to start with a basic noise signal (a "wave") and continuously add smaller and smaller detailed noise invocations to it. In nature, within a given space, there is room for only a few large shapes (mountains), but room for

thousands of tiny details (rocks, pebbles). **fBm** mimics this "Power-Law" behavior found in nature by accumulating waves with a coordinated reduction of wavelength and amplitude.

Algorithmic Implementation While mathematically represented as a sum, in a shader or procedural engine, **fBm** is implemented as a loop. The most efficient approach avoids expensive power functions *pow()* by updating the frequency and amplitude via simple multiplication in each step.

The following GLSL-style pseudocode illustrates the standard implementation used in modern terrain engines:

```
1 float fbm(vec2 x) {  
2     float total = 0.0;  
3     float amplitude = 0.5; // Starting amplitude (G)  
4     float frequency = 1.0; // Starting frequency  
5  
6     for(int i = 0; i < numOctaves; i++) {  
7         // Accumulate the noise layer  
8         total += amplitude * noise(frequency * x);  
9  
10        // Prepare for next layer (make it smaller and weaker)  
11        frequency *= 2.0; // Lacunarity  
12        amplitude *= 0.5; // Gain  
13    }  
14    return total;  
15 }
```

Figure 2.20: Standard **fBm** implementation as a loop over octaves.

Understanding "Octaves" The variable `numOctaves` determines the number of iterations in the loop. The term is borrowed from music theory, where an "octave" represents a doubling of frequency (the distance between a low C and a high C note).

In terrain generation, since we typically multiply the frequency by 2.0 in each step, we are effectively adding a new layer of detail that is half the size of the previous one. This geometric progression is incredibly efficient. As noted by Quilez, synthesizing terrain linearly would require millions of iterations, whereas using octaves allows for exponential detail. For instance, with just **24 octaves**, an algorithm can generate a heightmap that covers the entire planet Earth while still providing unique details down to a resolution of 2 meters.

Variations The code above produces standard "cloudy" terrain. However, the accumulation line (`total += ...`) can be modified to create different geological features without changing the loop structure:

- **Turbulence:** `total += amplitude * abs(noise(x));`
Taking the absolute value creates sharp creases at zero-crossings, useful for folded terrain or canyons.
- **Ridges:** `total += amplitude * (1.0 - abs(noise(x)));`
Inverting the absolute value pushes the creases upward, creating sharp mountain ridges.

Domain Warping

Concept While [Fractal Brownian Motion](#) builds complexity by adding layers of noise to the *output value* (the range), **Domain Warping** builds complexity by manipulating the *input coordinates* (the domain).

As described by Quilez [\[51\]](#), this technique involves evaluating a noise function $f(p)$, but instead of passing the raw position p , we pass a position that has been displaced by another noise function $g(p)$. Mathematically, this composition is expressed as:

$$\text{final}(p) = f(p + g(p)) \quad (2.21)$$

Visually, this creates a distortion effect similar to looking at a texture through rippled glass or heat haze. The coordinate space itself is twisted and stretched before the final pattern is generated.

Iterative Implementation The most powerful visual results are achieved when this warping is applied iteratively—using the output of one [fBm](#) pass to distort the coordinates of the next. This mimics physical fluid dynamics, where different currents interact with each other.

The following GLSL code demonstrates a "two-stage" domain warp pattern, often used to generate marble-like textures or swirling alien terrains:

```
1 float domainWarp(vec2 p) {  
2     // Stage 1: Generate a vector field 'q' using fBm  
3     // This creates the base distortion pattern  
4     vec2 q = vec2( fbm(p), fbm(p + vec2(5.2, 1.3)) );  
5  
6     // Stage 2: Generate a secondary vector 'r'  
7     // This distorts the coordinates based on 'q'  
8     // '4.0' is a scaling factor to control the warp strength  
9     vec2 r = vec2( fbm(p + 4.0*q + vec2(1.7, 9.2)),  
10                    fbm(p + 4.0*q + vec2(8.3, 2.8)) );  
11  
12     // Final Stage: Sample the noise using the  
13     // heavily distorted coordinates  
14     return fbm( p + 4.0*r );  
15 }
```

Figure 2.22: Two-stage domain warp pattern.

Visual Characteristics This technique fundamentally changes the character of the noise:

- **Organic Flow:** Unlike standard Perlin noise which looks like "lumps", domain warping creates long, curling structures that resemble smoke, clouds, or magma flows.
- **Directional Distortion:** The derivative of the warping function essentially defines a flow field, causing the terrain to look as if it has been eroded or stretched by a liquid current.

In terrain generation, Domain Warping is rarely used for the entire heightmap (as it can be too chaotic), but is often mixed with standard **fBm** to break up regularity and add "magical" or "surreal" twisting features to specific biomes.

Erosion Simulation

Purely procedural landscapes generated via noise functions often exhibit a distinctive lack of geological realism. Despite the fractal complexity provided by **fBm**, these terrains tend to miss the structural scars left by time and weather, resulting in heightmaps that appear more like random chaotic surfaces than actual landforms shaped by nature.

To remedy this, erosion simulation introduces a layer of physical interaction over the static geometry. This stage shifts the paradigm from mathematical approximation to dynamic mass transport, effectively acting as a post-processing filter that “ages” the terrain.

The fundamental mechanism relies on the conservation of mass: virtual material is dissolved from steep gradients, carried by agents such as water or thermal

stress, and deposited in depressions. Through this iterative cycle, the simulation creates defining geological characteristics impossible to achieve with noise alone, including:

- **Talus slopes:** Smooth gradients formed by material sliding down due to gravity ([thermal erosion](#)).
- **Drainage networks:** Coherent river channels and gullies carved by flowing water ([hydraulic erosion](#)).
- **Sedimentation:** The accumulation of soil in valleys and alluvial fans, softening the transition between mountains and plains.

Thermal Erosion

Concept and Physics Often referred to in geological literature as *mechanical weathering* or *mass wasting*, this form of erosion simulates the disintegration of rock due to thermal stress (expansion and contraction caused by temperature fluctuations) and the subsequent transport of debris by gravity.

In computer graphics, the simulation focuses primarily on the formation of **talus slopes**. The pivotal concept governing this process is the *angle of repose* (or critical angle). In nature, loose material (sand, soil, scree) can only pile up to a certain steepness before it becomes unstable and slides down.

Algorithmic Implementation From an implementation standpoint, thermal erosion is typically modeled using a Cellular Automata approach or a simple iterative kernel. The algorithm analyzes the local neighborhood of every vertex to ensure that the slope does not exceed a defined threshold T (the tangent of the angle of repose).

If the height difference between a vertex and its neighbor exceeds this threshold, a portion of material is moved from the higher vertex to the lower one. This process attempts to restore stability to the slope. The basic logic for a single iteration can be described as follows:

```

1 void applyThermalErosion(float[][] map, float threshold, float
   c){
2   for (int x = 0; x < width; x++) {
3     for (int y = 0; y < height; y++) {
4
5       // Check all neighbors
6       foreach (neighbor in getNeighbors(x, y)) {
7         float diff = map[x][y] - map[neighbor.x][neighbor.y];
8         // If slope is steeper than the angle of repose
9         if (diff > threshold) {
10          float transfer = c * (diff - threshold);
11          // Erode
12          map[x][y] -= transfer;
13
14          // Deposit
15          map[neighbor.x][neighbor.y] += transfer;
16        }
17      }
18    }
19  }

```

Figure 2.23: Single-iteration thermal erosion kernel.

Here, c represents an erosion rate or “viscosity” constant (typically usually divided by the number of neighbors to avoid removing more material than available).

Visual Characteristics Applying this algorithm iteratively results in a distinctive smoothing effect. Unlike Gaussian blur, which smooths everything uniformly, thermal erosion is selective:

- **Cliff reduction:** Sharp, unrealistic spikes generated by noise are worn down into conical hills.
- **Valley filling:** The material eroded from the peaks accumulates at the bottom, creating *scree slopes* or accumulating sediment in the valleys, effectively connecting disjointed pits.

This technique is essential for aging procedural terrain, transforming the “jagged” look of raw Perlin noise into a landscape that appears to have settled over millennia.

Hydraulic Erosion

Concept While thermal erosion simulates the localized smoothing of rough slopes due to gravity, **Hydraulic Erosion** simulates the global transport of material driven by flowing water. It is widely considered the most critical algorithm for generating realistic terrain, as it is responsible for carving the drainage networks, river valleys, and channels that define geologically aged landscapes.

Unlike the simple smoothing of thermal erosion, hydraulic erosion is a constructive and destructive process simultaneously. It carves deep cuts in steep areas (destructive) and builds up new land in flat areas (constructive/sedimentary).

The Water Cycle Process Conceptually, the simulation models the lifecycle of water interacting with the soil. Whether implemented via complex fluid dynamics or simplified particle systems, the fundamental logic follows a four-stage cycle:

- (1) **Precipitation:** Virtual water droplets are spawned randomly across the terrain surface or simulating rainfall patterns.
- (2) **Flow and Acceleration:** Driven by gravity, water moves towards lower elevations. As it flows down steep slopes, it gains velocity and energy.
- (3) **Erosion (Carving):** High-velocity water has the energy to dissolve terrain material. It subtracts height from the current location and carries the "dissolved" sediment in suspension. This creates a self-reinforcing loop: as water carves a path, the path becomes deeper, attracting more water in future iterations, eventually forming distinct river channels.
- (4) **Deposition (Sedimentation):** When water reaches flatter terrain or depressions (pools), it loses velocity. As energy decreases, the water can no longer carry the sediment, depositing it back onto the terrain. This typically fills valleys and creates alluvial fans at the base of mountains.
- (5) **Evaporation:** Over time or steps, the water volume decreases until the droplet vanishes.

Visual Impact The primary contribution of this technique to procedural terrain is the generation of **dendritic** (tree-like) structures. By simulating the flow of water, the algorithm naturally connects isolated noise patterns into a coherent system of ridges and valleys, providing the "skeleton" that makes a procedural landscape look like a believable geographical location.

Data-Driven Terrain Synthesis

All the methods discussed previously—from Perlin noise to hydraulic erosion—fall under the category of *procedural algorithms*. They rely on explicit, human-designed mathematical rules to approximate natural phenomena, requiring manual tuning of parameters and complex combinations of noise layers.

Data-driven approaches shift this paradigm. Instead of defining *how* to generate the terrain (rules), these methods learn *what* real terrain looks like by analyzing vast datasets of real-world topography, typically [Digital Elevation Model](#) obtained from satellite data (such as NASA's SRTM). By learning the statistical distributions and features of real geography, these systems can synthesize new terrain that retains the structural realism of Earth without explicit programming.

This is at the edge of the state of the art in terrain generation, not yet standard practice and still an active research area, but this section aims to provide an overview of what is possible with current technology.

Machine Learning / GANs

Concept The most prominent architecture in this field is the [Generative Adversarial Network \(GAN\)](#). In the context of terrain generation, [GAN](#) treat heightmaps as single-channel grayscale images, effectively repurposing powerful Computer Vision architectures for geological synthesis.

The architecture consists of two competing neural networks trained simultaneously in a zero-sum game:

- (1) **The Generator (G):** Takes an input (random noise or a user sketch) and attempts to produce a fake heightmap that looks realistic.
- (2) **The Discriminator (D):** Receives both real [DEM](#) samples from the dataset and fake samples from G . Its goal is to correctly classify them as "Real" or "Fake".

Over time, the Generator learns to map the statistical distribution of real-world topography so accurately that the Discriminator cannot distinguish its output from the training data.

Applications in Terrain Machine Learning is currently applied to terrain generation in three primary ways:

- **Interactive Sketching (cGANs):** Pioneered by Guérin et al. [\[52\]](#), this approach uses *Conditional GANs*. Instead of random generation, the network is conditioned on a user's input sketch (e.g., lines representing rivers or ridges). The AI "fills in the blanks," generating a realistic mountain range that strictly adheres to the artist's rough layout.
- **Simulation Acceleration:** As demonstrated in the paper, neural networks can be trained to approximate computationally expensive processes. A [GAN](#) can learn the result of a hydraulic erosion simulation, performing the task in milliseconds (real-time) compared to the tens of seconds required for a traditional physics-based simulation.
- **Super-Resolution (Amplification):** Similar to image upscaling, a low-resolution procedural heightmap (e.g., basic Perlin noise) is fed into a neural network, which "hallucinates" realistic high-frequency details based on learned geological patterns.

Limitations Despite their visual fidelity, data-driven methods face significant challenges that limit their widespread adoption in commercial game engines compared to procedural noise:

- **Data Dependency:** The quality of the output is strictly bound to the quality of the training data. If the [DEM](#) dataset contains noise, artifacts, or lacks variety, the generated terrain will inherit these flaws.
- **Sparse Input Artifacts:** As noted in the paper, if the user provides very sparse input (e.g., a large empty area with no sketch lines), the [GAN](#) may produce "grid artifacts" or repetitive patterns as it struggles to fill the void with meaningful data.
- **Continuity and Tiling:** GANs typically generate fixed-size images (e.g., 1024×1024). Stitching these outputs together to form an infinite, seamless world is difficult, whereas mathematical noise functions are continuous by definition.

2.5 Biome Assignment Techniques

The categorization of procedural terrain into distinct ecological zones, or "biomes," is a process with virtually infinite implementation possibilities. Since a biome is an abstract semantic concept rather than a strict geometric rule, developers have devised numerous strategies to determine whether a specific coordinate represents a desert, a forest, or a tundra.

Despite the variety of proprietary solutions, the most prevalent techniques in the industry can be broadly classified into two dominant paradigms: continuous field generation (Noise-based) and discrete partition (Voronoi-based).

Noise-Based Biome Generation

This method relies on the generation of multiple continuous scalar fields using the basis functions discussed in [Section 2.4](#) → p. 35 (typically Perlin or Simplex noise). The core concept is to simulate the physical properties of the environment, such as Temperature, Humidity, and Altitude, as separate noise maps.

By sampling these overlapping maps at a given coordinate (x, y) , the engine obtains a tuple of values (e.g., $\{Temp : 0.8, Moisture : 0.1\}$). These values are then mapped to a specific biome using a classification lookup table, often based on the **Whittaker Biome Classification** scheme.

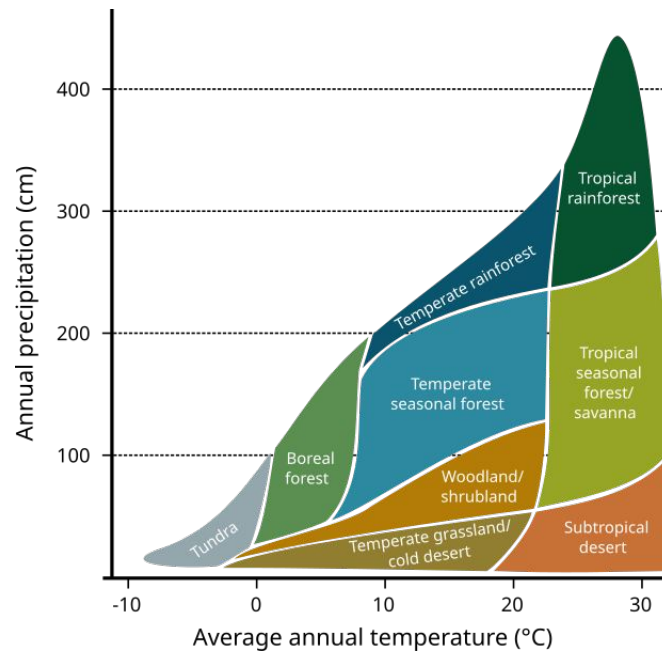


Figure 2.24: Whittaker Biome Classification diagram [53].

For example:

- **High Temp + Low Moisture** → Desert.
- **High Temp + High Moisture** → Rainforest.
- **Low Temp + High Moisture** → Boreal Forest / Taiga.

This approach results in smooth, organic transitions between biomes, as the underlying noise functions are continuous gradients. This technique is used in games like *Minecraft*.

Voronoi-Based Biome Assignment

In contrast to the gradient-based approach, this technique utilizes Voronoi diagrams (see [Section 2.4 → p. 39](#)) to partition the terrain into discrete regions or “cells” based on proximity to seed points.

In this workflow, the world is first divided into cellular zones. Each cell is then assigned a specific biome type (often randomly or semi-randomly based on neighbor rules). This creates distinct “patches” of terrain with defined boundaries.

This technique is particularly popular in strategy games or stylized world generation (like *Civilization*), where distinct political or ecological borders are preferred over realistic blending.

Comparison and Use Cases

Both techniques offer distinct advantages, and the choice largely depends on the specific requirements of the generation pipeline and the desired workflow:

Workflow Paradigm

- **Noise (Terrain First):** Typically follows a “Terrain → Biomes” workflow. The heightmap is generated first, and biomes are “painted” onto it based on the resulting height and simulated weather conditions. This is ideal for simulations aiming for geological realism.
- **Voronoi (Biomes First):** Often follows a “Biomes → Terrain” workflow. The regions are defined first (e.g., “This cell is an island, that cell is ocean”), and the terrain height is generated afterwards to fit inside those boundaries. This effectively treats biomes as tectonic plates.

Selection Guide A common heuristic for choosing between these methods is the scale and complexity of the desired ecosystem:

- **Low Complexity (2-5 Biomes):** If the goal is to create clearly defined zones (e.g., a Fire Level vs. an Ice Level), **Voronoi** is superior. It ensures regions are distinct and avoids ambiguous transition zones.
- **High Complexity (10+ Biomes):** If the goal is to simulate a complex, interconnected world, **Noise** is generally preferred. The multidimensional combinations of temperature and humidity allow for a vast variety of sub-biomes and natural transition zones (ecotones) that would be difficult to manage manually with Voronoi cells.

2.6 How does Godot handle terrain generation?

Unlike other commercial game engines such as Unity or Unreal Engine, which offer established, monolithic landscape systems out of the box, Godot Engine currently maintains a minimalist approach regarding terrain rendering.

This decision stems from a foundational design philosophy debated within the core development team. As detailed in the official proposal regarding terrain implementation initiated by lead developer Juan Linietsky [54], the primary challenge lies in the sheer diversity of requirements across different game genres. A solution optimized for a heightmap-based strategy game (RTS) is fundamentally different from one required for a spherical planetary simulator or a voxel-based cavern system.

The consensus derived from this discussion [54] highlights three key pillars for Godot’s future terrain strategy:

- **Avoid Monolithic Solutions:** Implementing a single, high-level "Terrain Node" inevitably leads to a tool that is too specific for some users and too limited for others. The core team argues that maintaining such a complex system would drain resources from other critical areas.
- **Data-Agnostic Rendering:** Instead of a rigid editor tool, the goal is to provide low-level rendering APIs (likely based on *Geometry Clipmaps* or *Meshlets*) that can efficiently render massive amounts of geometry, regardless of whether that geometry comes from a heightmap, a voxel grid, or code.
- **Plugin-First Ecosystem:** Godot encourages the community to build specialized terrain tools (like *Zylann's Voxel Tools* or *Godot Heightmap Plugin*) on top of these high-performance APIs.

Consequently, for a developer implementing a custom procedural terrain system in Godot 4, the engine does not provide a "black box" to configure. Instead, it provides a suite of modular, low-level building blocks. The following subsections analyze the specific classes and technologies that enable the construction of such a system.

GDExtension

GDExtension represents the evolution of the engine's extensibility layer in Godot 4.0, replacing the previous *GDNative* system. It is a high-level interoperability technology that allows developers to extend the engine using compiled languages—such as C++, Rust, or Swift, without the need to modify the underlying engine source code or recompile the editor binary.

Technically, GDExtension exposes a C-based interface that acts as a bridge between the engine's core and external shared libraries (*.dll* on Windows, *.so* on Linux). When the engine starts, it loads these dynamic libraries, allowing the external code to register new classes, bind methods to the scripting API, and interact with engine singletons as if they were native engine components.

For a procedural terrain generation system, the choice of GDExtension over the built-in scripting language (GDScript) is driven by computational necessity. Terrain generation involves iterating over grid chunks containing tens of thousands of vertices. For each vertex, the system must evaluate complex noise functions (*fBm*, Domain Warping) and mathematical operations (dot products, vector normalization).

While GDScript is optimized for game logic, it runs on a virtual machine. In contrast, C++ code via GDExtension runs natively on the CPU. Benchmarks typically show that heavy arithmetic loops in C++ can be orders of magnitude faster than in interpreted languages.

Unlike writing a custom C++ Engine Module (which was the standard high-performance method in Godot 3), GDExtension does not require redistributing a

custom build of the Godot Editor. The resulting terrain system can be distributed as a simple plugin folder containing the compiled libraries, making it accessible to designers and other developers using the standard version of Godot.

EditorPlugin

While `GDExtension` provides the computational backend, the **EditorPlugin** class serves as the entry point for extending the functionality of the Godot Editor itself. It allows developers to inject custom logic into the editor's lifecycle, effectively transforming the procedural generation algorithms into a usable, interactive toolset for level designers.

The plugin functions as a manager class. When enabled, the editor calls its `_enter_tree()` virtual method, and when disabled, `_exit_tree()`. For this project, this lifecycle is utilized to:

- **Register Custom Nodes:** It exposes the C++ classes created via `GDExtension` (such as the Terrain Generator node) to the "Create New Node" dialog, making them indistinguishable from native engine nodes.
- **Resource Management:** It handles the initialization of shared resources or singleton classes required for the terrain generation pipeline.

Beyond simple registration, **EditorPlugin** grants access to the editor's GUI system. This is essential for creating a terrain editor. Key capabilities leveraged include:

- **Input Handling:** The `_forward_3d_gui_input()` method allows the plugin to intercept mouse events within the 3D viewport. This is the underlying mechanism used to implement terrain sculpting brushes or biome painting tools directly on the mesh surface.
- **Custom Inspectors:** It allows the injection of custom controls into the Inspector dock. For instance, adding a "Regenerate Terrain" button or a real-time noise preview texture that updates as the user drags a frequency slider.

ArrayMesh

The `ArrayMesh` serves as the primary data container for procedurally generated geometry within the Godot Engine. Unlike static meshes imported from external modeling software (which are typically handled as `ImporterMesh` resources), an `ArrayMesh` is designed to be constructed and manipulated programmatically at runtime.

As its name implies, this class organizes geometry into a series of typed arrays. To define a valid surface for the Rendering Server, the system must provide, at a minimum, an array of vertex positions. However, for a fully featured terrain system, the generation algorithm populates typically four synchronized arrays:

- **Vertex Array (`ARRAY_VERTEX`):** Vectors defining the 3D position of each grid point, displaced by the heightmap values.
- **Normal Array (`ARRAY_NORMAL`):** Vectors perpendicular to the surface at each vertex, essential for calculating lighting and shading.
- **UV Array (`ARRAY_TEX_UV`):** 2D coordinates mapping the geometry to texture space, allowing for material application.
- **Index Array (`ARRAY_INDEX`):** Integers defining the topology (which vertices connect to form triangles). Using indexed geometry significantly reduces memory usage by allowing vertices to be shared between adjacent triangles.

In the context of this project, the `ArrayMesh` acts as the final output of the generation pipeline. The C++ code calculates the fractal noise and erosion on the CPU, packs this data into the aforementioned arrays, and then commits them to the GPU using the `add_surface_from_arrays()` method.

Once the arrays are committed, the data is uploaded to the video memory (VRAM). This separation is crucial for performance: while the construction happens on the CPU, the rendering is handled entirely by the GPU, allowing the engine to render massive terrains efficiently as long as the CPU is not constantly re-uploading the geometry every frame.

A notable capability of the `ArrayMesh` is the ability to manually define its [Axis-Aligned Bounding Box \(AABB\)](#). For procedural terrain, calculating the bounds of a chunk is a trivial mathematical operation (based on chunk size and max height). By setting this manually using `custom_aabb`, the system avoids the computational overhead of the engine iterating through every vertex to calculate the bounds, further optimizing the generation time.

MultiMeshInstance3D

In the context of terrain generation, a realistic environment requires the scattering of thousands of distinct objects, such as trees, grass blades, rocks, and debris. Attempting to represent each of these elements as an individual `MeshInstance3D` node would result in a prohibitive performance cost due to the overhead of thousands of individual draw calls sent to the GPU.

The `MultiMeshInstance3D` node solves this bottleneck by implementing *Hardware Instancing*. This technique allows the engine to render thousands of copies of the same base mesh in a single draw call, shifting the workload from the CPU to the GPU.

The node itself acts as a container for a `MultiMesh` resource. This low-level resource holds the instance data in a tightly packed buffer. Unlike standard nodes, the instances within a `MultiMesh` do not exist in the scene tree hierarchy; they are purely data entries on the GPU.

For each instance, the system stores:

- **Transform:** The position, rotation, and scale (stored as a Matrix or simplified vectors).
- **Color:** An individual color tint per instance.
- **Custom Data:** A set of 4 arbitrary floats. This is particularly powerful for procedural shaders, allowing developers to pass unique attributes per instance—such as "wind sway phase," "wetness," or "variation seed", without creating unique materials.

By utilizing `MultiMeshInstance3D`, the procedural generation system can populate a chunk of terrain with high-density foliage (e.g., 10,000 grass blades) with negligible CPU impact. The renderer treats the entire field of grass as a single object, allowing for vast, lush environments that would be impossible to render using traditional scene nodes.

HeightMapShape3D

While `ArrayMesh` handles the visual rendering of the terrain, it does not inherently provide collision data for the physics engine. To enable interactions, such as the player walking on the ground or vehicles driving over hills, the system requires a collision shape.

Godot provides the **HeightMapShape3D**, a specialized physics resource optimized specifically for terrain geometry. Unlike general-purpose mesh colliders (`ConcavePolygonShape3D`) which store arbitrary triangle soups, the `HeightMapShape3D` stores elevation data as a structured grid of scalar values (floats).

The primary motivation for using this class over a standard mesh collider is performance.

- **Mesh Collider (`ConcavePolygonShape3D`):** To test a collision, the physics engine must traverse a [BVH](#) tree to find which triangles are near the object. This is typically an $O(\log n)$ operation.
- **HeightMap Collider:** Since the data is a regular grid, the physics engine can calculate exactly which cell an object is currently standing on using simple integer arithmetic (flooring the coordinates). This turns the collision query into an $O(1)$ constant-time lookup.

In the procedural pipeline, the `HeightMapShape3D` acts as the physical counterpart to the visual mesh. The same noise values calculated in C++ to displace the `ArrayMesh` vertices are serialized into a `PackedFloat32Array` and assigned to the shape's `map_data` property. This ensures that the visible terrain and the physical terrain are perfectly synchronized, preventing visual artifacts where feet clip through the ground or float above it.

FastNoiseLite

At the core of any procedural terrain system lies the noise generation algorithm. In previous versions of the engine, developers often had to implement Perlin or Simplex noise from scratch or integrate third-party C++ libraries. Godot 4 addresses this by integrating the **FastNoiseLite** library directly into the core engine as a built-in Resource.

This class provides a high-performance, open-source collection of noise algorithms. It serves as a unified API to access the mathematical basis functions discussed in [Section 2.4](#) → p. 35. Supported noise types include:

- **Lattice Noise:** `TYPE_PERLIN`, `TYPE_SIMPLEX` and `TYPE_VALUE`.
- **Cellular Noise:** `TYPE_CELLULAR` (Voronoi/Worley), with options to change the distance metric (Euclidean, Manhattan) and the return type (Distance to center, Cell ID).

Furthermore, the class internally handles [Fractal Brownian Motion](#). By configuring properties such as `fractal_type`, `octaves`, `lacunarity`, and `gain`, the developer can obtain complex, multi-layered noise values via a single function call, without needing to implement the accumulation loop manually.

One of the most significant advantages of using **FastNoiseLite** over a custom C++ implementation is its integration with the Godot Editor. Because it is a **Resource**, it exposes all its parameters (seed, frequency, noise type) to the Inspector dock.

Additionally, it provides a real-time visual preview. As the developer adjusts the parameters, the editor renders a texture showing the resulting noise pattern. This immediate visual feedback loop is invaluable for level designers tuning the "look and feel" of the terrain biomes without needing to run the game.

In the context of a C++ GDExtension, this class can be instantiated directly. This allows the terrain system to leverage the optimized C++ implementation of the library while maintaining full compatibility with the engine's serialization system.

3 Project Management

In this section, the tools used to track the project's progress and the organizational strategies employed will be discussed. Additionally, the validation tools selected to verify that the project meets its technical and performance objectives will be detailed.

3.1 Organizational Tools

To ensure an efficient workflow and proper tracking of the tasks described in [Chapter 4](#) ^{p. 71}, the project relies on the GitHub ecosystem. This centralizes code hosting, version control, and task management in a single platform.

- **GitHub Projects:** Used as a Kanban and Gantt board to visualize the state of active tasks. The workflow follows a standard "Backlog", "In Progress", "In Review" and "Done" structure, allowing for a quick overview of the development status.

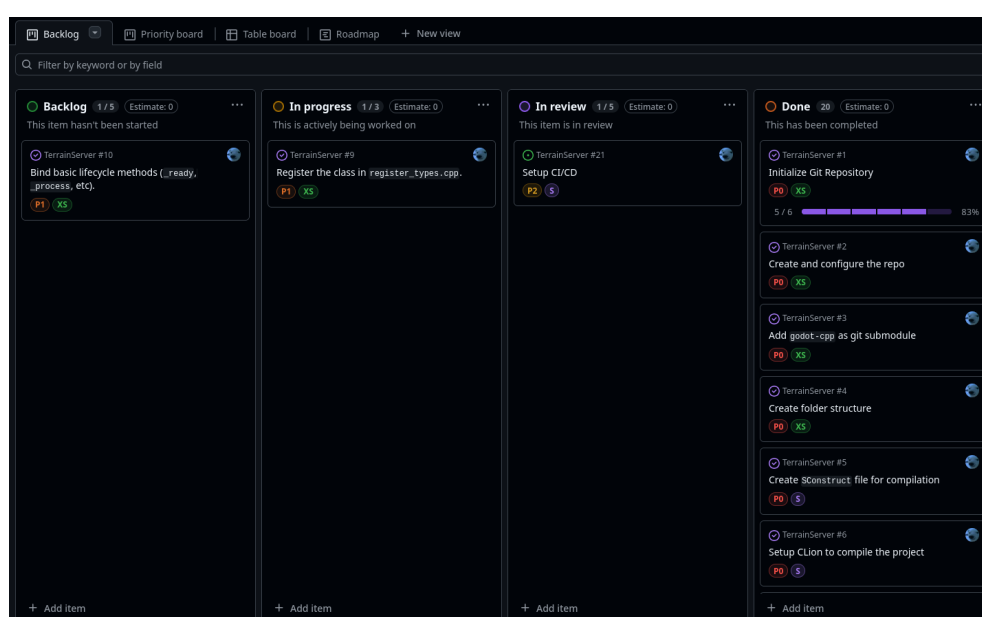


Figure 3.1: GitHub Project Board used for task management.

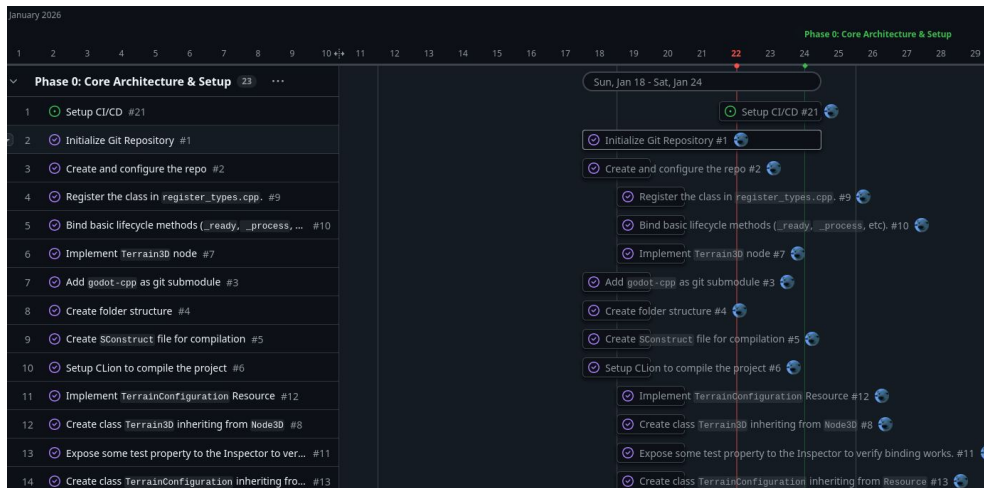


Figure 3.2: Gantt Chart view of the project timeline.

- **Milestones:** The development timeline is divided into key milestones that will be discussed in detail later in [Chapter 4 → p.71](#). These milestones act as deadlines to ensure the project stays on schedule and avoids scope creep.

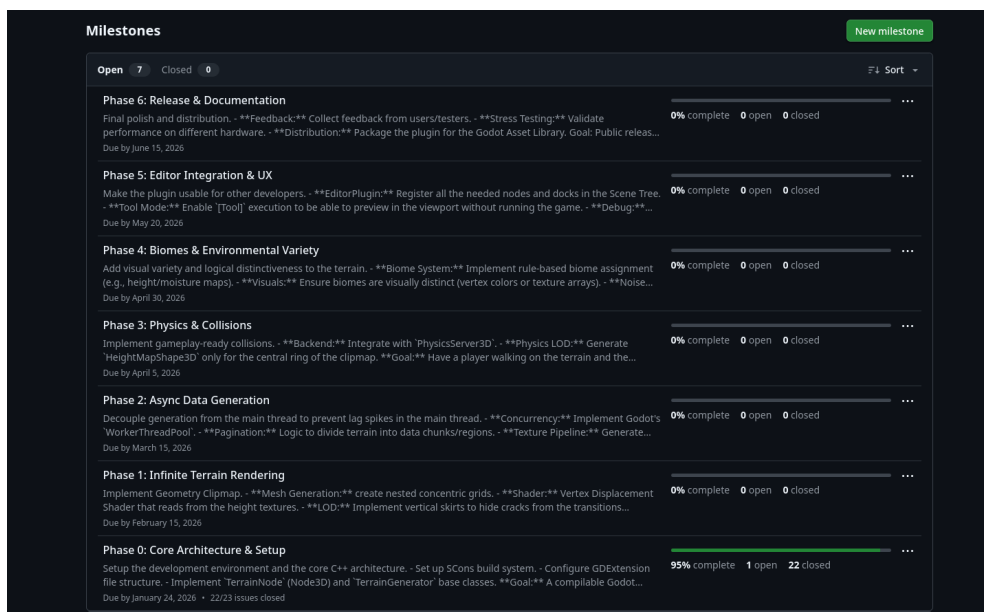


Figure 3.3: GitHub Milestones tracking project deadlines.

- **Version Control (Git):** Standard branching strategies are used to isolate new features before merging them into the main branch, ensuring the stability of the codebase at all times. The main branch is protected, requiring pull requests and code reviews before integration. The Pull Request needs to pass automated tests (build success in Linux, Windows and MacOS) before being merged. Also, another workflow is set up to automatically build release binaries when a new version is tagged.

3.2 Development Tools

The tools used for development are crucial for ensuring an efficient workflow and high-quality code. The following tools are selected for their compatibility with the project requirements and their ability to facilitate the development process:

- **Godot Engine:** The primary development environment for the project, providing the necessary tools for 3D rendering, scene management, and integration of custom C++ code through GDExtensions.
- **CLion:** The main IDE for C++ development, offering advanced code analysis, refactoring tools, and seamless integration with SCons and Git. It also provides a powerful debugging environment.
- **GitHub:** For version control and code hosting. It also serves as the platform for issue tracking and project management (see [Section 3.1 → p. 60](#)).
- **GitHub Desktop:** A graphical interface for Git, making it easier to manage repositories.
- **SCons:** The build system used to compile the C++ code for the GDExtension.
- **GitHub Actions:** For [Continuous Integration/Continuous Deployment](#), ensuring that code changes do not introduce regressions and that the project remains stable across all supported platforms.
- **RenderDoc:** A graphics debugger used to analyze and optimize the rendering pipeline, ensuring that the terrain generation system meets performance requirements.
- **Tracy Profiler:** A real-time, multi-threaded profiler for C++ applications, used to identify performance bottlenecks in the code.
- **Valgrind:** A tool for memory debugging, memory leak detection, and profiling. It helps ensure that the C++ code is efficient and free of memory-related issues.

3.3 Validation Tools

Given the high-performance requirements of a terrain generation system, ensuring code efficiency and rendering correctness is critical. The following specific tools are used to validate the technical success of the project:

- **Godot Profiler:** The built-in engine tools are the first line of defense to monitor frame rate, video memory usage, and draw calls in real-time. It may be insufficient, so it is complemented with the following tools.
- **Tracy Profiler:** Since the core logic is written in C++, the Godot Profiler cannot see inside the custom functions. Tracy is used to measure the C++

code performance with very high accuracy, allowing for detailed profiling of the noise generation and mesh construction algorithms.

- **RenderDoc:** A standalone graphics debugger used to capture frames and inspect how the terrain geometry and textures are being sent to the GPU. This is essential for debugging shaders and verifying that the [LOD](#) system is reducing vertex count as expected.
- **Manual & Visual Testing:** Due to the procedural nature of the project, unit testing is often insufficient for evaluating aesthetic quality. Rigorous manual testing involves traversing the terrain to identify visual artifacts like seams between chunks or texture stretching and verifying collision consistency.

3.4 SWOT

A SWOT analysis is used to identify and analyze the Strengths, Weaknesses, Opportunities, and Threats related to the project. It helps in understanding the internal and external factors that can impact the success of the project.

	Positive	Negative
Internal Origin	Strengths • Performance: High execution speed using C++. • Architecture: CPU-authoritative design. • Integration: Native GDExtension feel fully integrated with the engine.	Weaknesses • Complexity: Higher difficulty than GDScript. • Setup: Requires build tools (SCons). • Portability: Platform-specific binaries.
External Origin	Opportunities • Market Gap: Need for Godot 4 terrain tools. • Adoption: Growing industry interest.	Threats • API Volatility: Breaking changes in engine. • Competition: Ports of mature plugins and tools.

Figure 3.4: SWOT Analysis Diagram.

3.5 Risk and Contingency Plan

In this section, the potential risks that could impact the project are identified, along with their likelihood and impact. A contingency plan is also developed to mitigate these risks.

Table 3.5: Risks and Contingency Plan.

Risk	Probability	Impact	Solution
API changes in Godot 4	Medium	High	Regularly monitor Godot updates; allocate time for adjustments.
Delays in development	Medium	Medium	Set realistic milestones; maintain buffer time in schedule.
Hardware failures	Low	High	Regular backups; use reliable hardware; have spare parts available or ask university for temporary replacements.
Publishing problems	Medium	Medium	Familiarize with publishing guidelines early; seek help from community/forums.
Time management issues	Medium	Medium	Use project management tools; set clear priorities; avoid scope creep.

3.6 Cost Analysis

This section provides a detailed breakdown of the costs associated with the project. Although this is an open-source project developed using open-source tools, estimating the budget is essential for understanding the resources involved.

The budget is divided into three main categories: Personnel, Hardware Amortization, and Indirect Costs.

Personnel Costs

The most significant cost driver is the developer's labor. The project's calendar span was extended to approximately 11.5 months (from October 2, 2025, to September 15, 2026) by a temporary work pause, detailed in [Section 3.8](#)^{→ p. 69}; since that pause suspended development rather than expanding its scope, the underlying labor effort remains the originally planned 720 hours, at a part-time dedication of 20 hours per week. Based on a junior engineering internship rate of 8€/hour, the personnel cost is calculated as follows:

Table 3.6: Breakdown of personnel costs.

Task Category	Hours	Rate (€/h)	Cost (€)
Research and Learning	144	8.00	1,152.00
Development	432	8.00	3,456.00
Report Writing & Documentation	144	8.00	1,152.00
Total Personnel	720		5,760.00

Hardware and Software Amortization

The project utilized specific hardware resources. Unlike labor, hardware ages with calendar time regardless of the work pause discussed above, so the amortization is calculated using a standard linear depreciation over a useful life of 4 years (48 months), applied to the project's full 11.5-month calendar span.

- **High-Performance Workstation:** AMD Ryzen 7 9800X3D, RTX 5080, 32GB RAM (Value: 3,000€).
- **Laptop:** Slimbook Executive 16 (Value: 2,000€).
- **Apple Hardware:** Mac Mini (Value: 800€).
- **Software:** The project relies heavily on open-source software (Godot, Blender, VS Code, GitHub, Arch Linux). CLion was used under a free Educational License; however, for a commercial estimate, an 11.5-month subscription would cost approximately 115€.

Table 3.7: Hardware amortization costs.

Item	Value (€)	Useful Life	Project Usage	Amt (€)
Main Workstation	3,000.00	48 months	11.5 months	718.75
Laptop	2,000.00	48 months	11.5 months	479.17

Continued on next page

Table 3.7: Hardware amortization costs. (Continued)

Item	Value (€)	Useful Life	Project Usage	Amt (€)
Apple Test Device	800.00	48 months	11.5 months	191.67
Software Licenses	0.00	N/A	N/A	0.00
Total Hardware				1,389.59

Total Budget

Finally, indirect costs such as electricity and internet connection are added to form the final budget. Using an estimated cost of 30€ per month over the 11.5-month period.

Table 3.8: Final Cost Analysis.

Category	Total Cost (€)
Personnel	5,760.00
Hardware Amortization	1,389.59
Indirect Costs (Electricity & Internet)	345.00
TOTAL PROJECT COST	7,494.59

3.7 Environmental Impact and Social Responsibility

This project has been developed taking into consideration its environmental footprint and social implications. Following the standard methodology for software engineering projects, the impact analysis is divided into three distinct phases: development, usage, and end of life.

Development Phase

The development phase accounts for the direct and indirect emissions generated during the creation of the project. This includes direct energy consumption, hardware amortization, and the use of cloud-based services.

Quantitative Analysis (CO₂ Footprint)

To estimate the carbon footprint, three main sources of emissions are considered:

- (1) **Energy Consumption:** The project required approximately **720 hours** of active development. The primary equipment used was a high-performance desktop workstation. Due to the high-end specifications and the computational load of compiling C++ and rendering 3D scenes, the average power

consumption is estimated at **250 Watts** (0.25 kW). Using a conservative estimate based on the Spanish energy mix emission factor reported by Red Eléctrica de España (≈ 0.20 kg CO₂/kWh [55]), the direct emissions are calculated.

- (2) **Hardware Amortization:** Precise carbon footprint data for the specific devices used in this project are unavailable. To estimate the emissions from manufacturing, this thesis utilizes Product Carbon Footprint (PCF) reports from comparable industry-standard devices as proxies.

The estimated manufacturing impact is calculated based on the *Production/Manufacturing* phase percentage reported in the PCF documents:

- **Workstation Proxy** (Dell Precision 3660): ≈ 280 kg CO₂e (46.1% of 608 kg total mean) [56].
- **Laptop Proxy** (Dell XPS 15 9530): ≈ 365 kg CO₂e (83.8% of 435 kg total mean) [57].
- **Test Device** (Mac mini M2): ≈ 71 kg CO₂e (63% of 112 kg total) [58].

The combined production impact for the hardware suite is estimated at **716 kg CO₂e**. These emissions are amortized over a standard useful lifespan of 4 years. The thesis accounts exclusively for the fraction of the device's lifespan dedicated to this project based on the hours logged.

(3) **Cloud and AI Services:**

- **CI/CD & Storage:** The project utilizes GitHub Actions for automated Continuous Integration. The pipeline executes compilation jobs on standard Linux runners. Based on the mean runtime usage (≈ 50 runs $\times$ 15 minutes) and the carbon intensity of hyperscale data centers (≈ 0.096 kg CO₂/kWh according to the Cloud Carbon Footprint methodology [59]), the direct emissions from **CI/CD** are estimated, though a safety buffer is applied to account for storage redundancy and data transfer.
- **Generative AI:** Large Language Models (LLMs) were employed for code optimization, debugging and documentation. The impact of Generative AI includes both energy consumption from using these models and the amortized cost of model training. Recent studies estimate that complex, long-context queries can emit significantly more carbon than standard search queries [60]. Using a conservative estimate of **15g CO₂e per complex query** to account for inference and training amortization, the 150 logged queries contribute approximately 2.25 kg to the total footprint.

The resulting CO₂ budget is detailed in Table 3.9 $\rightarrow$ p. 68.

Table 3.9: Estimated CO₂ emissions during the development phase.

Source	Quantity	Factor/Formula	Total CO ₂
Direct Electricity	180.0 kWh	$(720 \text{ h} \times 0.25 \text{ kW}) \times 0.20 \text{ kg/kWh}$	36.00 kg
Hardware Amortization	720 hours	$\frac{716 \text{ kg}}{11680 \text{ h}} \times 720 \text{ h}$	44.14 kg
AI & Cloud Services	150 queries + CI/CD	$(150 \times 0.015 \text{ kg}) + 1.00 \text{ kg}_{\text{buffer}}$	3.25 kg
Total Impact			83.39 kg

Usage Phase

The usage phase analyzes the impact of the software when deployed by end-users. Unlike the development phase [Section 3.7](#)^{p.66}, this is qualitative, focusing on efficiency and social impact.

Environmental Impact: Efficiency Sustainability

The core objective of this project is optimization. By implementing terrain generation in C++ rather than GDScript, the computational cost of using the addon is significantly reduced.

- **Energy Efficiency:** Optimized code requires fewer CPU cycles to perform the same task. If a game developed with this plugin is played by thousands of users, the aggregate reduction in power consumption compared to an unoptimized software represents a net positive environmental impact.
- **Hardware Lifespan:** By utilizing efficient, low-level algorithms, the software runs better on lower-end hardware. This prevents users from needing to upgrade their systems frequently, reducing electronic waste.

End of Life Phase

The End of Life phase considers the long-term implications of the project after its active development and usage period. This includes data management and digital waste. Digital projects often contribute to data garbage, stored in servers that is never accessed again, consuming energy passively.

To mitigate this:

- **Repository Maintenance:** The code is hosted on GitHub. To ensure it does not become digital waste, the repository includes clear documentation, making it a reusable resource rather than abandoned data.

- **No Physical Waste:** As a purely digital software product, there are no physical components requiring recycling or toxic waste management.

Social Responsibility

The project adopts a permissive open-source license (GNU GPL v3). This fosters knowledge sharing and education within the Godot community. By providing a documented reference for GDExtension architecture, this project prevents other developers from "reinventing the wheel", saving collective development time and resources.

3.8 Adjustments to the Project Plan

The project has experienced a schedule deviation of approximately two to three months compared to the initial timeline. This delay is the direct result of the author assuming a new professional role as a Tool Developer at Rovio, which temporarily constrained the time available to dedicate to this thesis. Consequently, active development was paused to accommodate the onboarding process of this new position.

To address this delay without compromising the scope of the project, the milestones and the final delivery date have been recalibrated. The project's conclusion, originally targeted for June 2026, has been shifted to September 2026. The adjusted development schedule, maintaining the original sequential dependencies, is now defined as follows:

- **Phase 0: Core Architecture & Setup:** Completed by January 24, 2026. The SCons build system, the core C++ architecture, and the GDExtension file structure were successfully established and compiled.
- **Phase 1: Infinite Terrain Rendering:** Shifted from February 15, 2026, to May 15, 2026. This phase focuses on the implementation of Geometry Clipmaps, generating nested concentric grids, and utilizing vertex displacement shaders to ensure seamless Level of Detail (LOD) transitions without visual artifacts.
- **Phase 2: Async Data Generation:** Shifted from March 15, 2026, to May 22, 2026. Implements shader based procedural noise generation. Instead of using CPU-based noise generation, the project will leverage GPU shaders to generate heightmap data.
- **Phase 3: Physics & Collisions:** Shifted from April 5, 2026, to June 15, 2026. Focuses on backend integration with the PhysicsServer3D to generate HeightMapShape3D collision data exclusively for the central high-resolution clipmap rings.
- **Phase 4: Biomes & Environmental Variety:** Shifted from April

30, 2026, to July 15, 2026. Incorporates rule-based biome assignment by combining multiple noise functions (e.g., elevation, temperature, and moisture maps) to simulate organic environmental diversity.

- **Phase 5: Editor Integration & UX:** Shifted from May 20, 2026, to August 15, 2026. Ensures the plugin is fully functional within the Godot Editor by registering custom `EditorPlugin` docks, enabling `[Tool]` mode execution, and implementing custom Gizmos for debugging.
- **Phase 6: Release & Documentation:** Shifted from June 15, 2026, to September 13, 2026. Encompasses final stress testing across diverse hardware configurations, writing comprehensive API documentation, and packaging version 1.0 for the Godot Asset Library.

Despite this schedule adjustment, the original scope of the project remains intact. No planned features have been removed. The primary focus remains on delivering a robust, high-performance C++ implementation, fully integrated with the Godot Engine, that meets the technical and performance objectives outlined in the initial project proposal.

Methodology

The development of the project follows an **Iterative and Incremental** methodology. Given the scope of implementing a system of this size, the project was broken down into manageable, self-contained phases. Each phase builds upon the previous one, adding specific functionality and complexity layer by layer.

The implementation plan is structured into seven distinct phases:

4.1 Phase 0: Core Architecture & Setup

The initial phase lays the groundwork for the entire project. It focuses on establishing the core architecture and development environment necessary for building a Godot GDExtension plugin. The main tasks include:

- Setup **Git** repository for version control.
- Setup Godot Engine with GDExtension support.
- Integration of **godot-cpp** bindings as a git submodule.
- Implementation of a **CI/CD** pipeline using **GitHub Actions** for automated builds and testing in Linux, Windows, and MacOS.
- Implementation of an Action to automatically publish releases to GitHub Releases when a new version is tagged.
- Configuring the **SCons** build system for cross-platform compatibility.
- Implementing base C++ classes such as **TerrainNode** and **TerrainGenerator**.
- Achieving a compilable Godot plugin that can load a custom node into the scene with a basic terrain representation.
- Getting familiar with Godot's GDExtension API and C++ integration.

The file structure is organized to support the GDExtension workflow. The main objective of this phase is to achieve a compilable Godot plugin that can load a custom node into the scene without causing engine crashes, serving as the stable base for future development.

4.2 Phase 1: Infinite Terrain Rendering

This phase addresses the graphical core of the project. It focuses on implementing the rendering techniques required to display massive terrains efficiently using a Geometry Clipmap approach. The main tasks include:

- Implementing the mesh generation logic to create nested concentric grids.

- Developing a Vertex Displacement Shader that reads from height textures.
- Implementing **LOD** strategies, such as vertical skirts, to hide transitions between grid levels.
- Synchronizing the clipmap center with the camera position to enable infinite movement.

The main objective of this phase is to achieve a seamless infinite terrain where the camera can move freely without visual artifacts or geometry cracks.

4.3 Phase 2: Async Data Generation

To ensure optimal performance, this phase focuses on decoupling the terrain data generation from the main rendering thread to prevent execution freezes. The main tasks include:

- Implementing concurrency using Godot's `WorkerThreadPool`.
- Developing pagination logic to divide the terrain into data chunks or regions.
- Creating a pipeline to generate noise in background threads.
- Ensuring thread-safe synchronization when uploading results to VRAM.

The goal of this phase is to eliminate stuttering during player movement by generating the terrain asynchronously in the background.

4.4 Phase 3: Physics & Collisions

This phase addresses the problem of terrain collisions, ensuring that the player can interact physically with the generated terrain. The main tasks include:

- Integrating the terrain system with Godot's `PhysicsServer3D`.
- Implementing a Physics **LOD** strategy to optimize performance.
- Generating `HeightMapShape3D` collisions specifically for the central area around the player.

The objective is to have a player character capable of walking on the terrain with accurate collision responses.

4.5 Phase 4: Biomes & Environmental Variety

This phase aims to enhance the aesthetic and logical quality of the terrain, moving beyond simple noise generation to create distinct environments. The main tasks include:

- Implementing a rule-based biome assignment system such as using height or moisture maps.
- Differentiating biomes visually using vertex colors or texture arrays.
- Fine-tuning noise algorithms to produce organic, user-configurable results.

The goal is to ensure the terrain looks organic and diverse, avoiding the repetitive look of simple noise functions.

4.6 Phase 5: Editor Integration & UX

The focus of this phase is Developer Experience, making the plugin accessible and usable for other developers within the Godot Editor. The main tasks include:

- Registering custom nodes and docks via the `EditorPlugin` class.
- Enabling `[Tool]` mode to allow previewing the terrain in the viewport without running the game.
- Implementing debug gizmos to visualize clipmap boundaries and chunk states.

The objective is to allow developers to configure and preview the terrain entirely within the Godot Editor interface.

4.7 Phase 6: Release & Documentation

The final phase is dedicated to polish, validation, and preparing the project for public distribution. The main tasks include:

- Collecting feedback from initial users or testers.
- Stress testing the system to validate performance on different hardware.
- Packaging the plugin for distribution on the Godot Asset Library.

The ultimate goal is the public release of version 1.0 in the Godot Asset Library.

5 Project Development

This chapter details the project's development process, providing an in-depth analysis of the architectural decisions, technical challenges encountered, and the solutions implemented. The structure of this section is the same as the one defined in [Chapter 4](#) → p. 71, following the development phases chronologically. Each phase is described in detail, including the specific tasks performed, the technologies used, and the outcomes achieved. Furthermore, code snippets, diagrams, and screenshots are included to illustrate the development process and the results obtained. The goal of this chapter is to provide a comprehensive overview of the project development, highlighting the key decisions made and the lessons learned throughout the process.

5.1 Phase 0: Core Architecture & Setup

File Structure

This initial phase focused on establishing the foundational architecture of the project and configuring the development environment. To achieve this, a Git repository was initialized for version control, and the Godot Engine was set up with GDExtension support following the official Godot documentation [\[61\]](#). The resulting file structure is illustrated in [Figure 5.1](#) → p. 75.

This directory structure serves as the foundational root of the repository. It is architected to be highly scalable, accommodating future expansions such as comprehensive documentation, supplementary C++ classes, custom shaders, and utility scripts as new features are iteratively implemented. Currently, the most critical components within this hierarchy include the following:

- **SConstruct**: The SCons build script responsible for cross-compiling the C++ source code and generating the dynamic GDExtension library.
- **demo/**: A standalone Godot project directory designated for testing, debugging, and demonstrating the plugin's capabilities.
- **src/**: The root directory that contains the entirety of the plugin's C++ source code, internally organized to separate core logic from scene nodes.
- **godot-cpp/**: A Git submodule containing the official Godot C++ bindings required to interface with the engine's internal APIs during GDExtension development [\[62\]](#).
- **.github/**: A directory hosting GitHub-specific configurations, mostly [CI/CD](#) workflows.

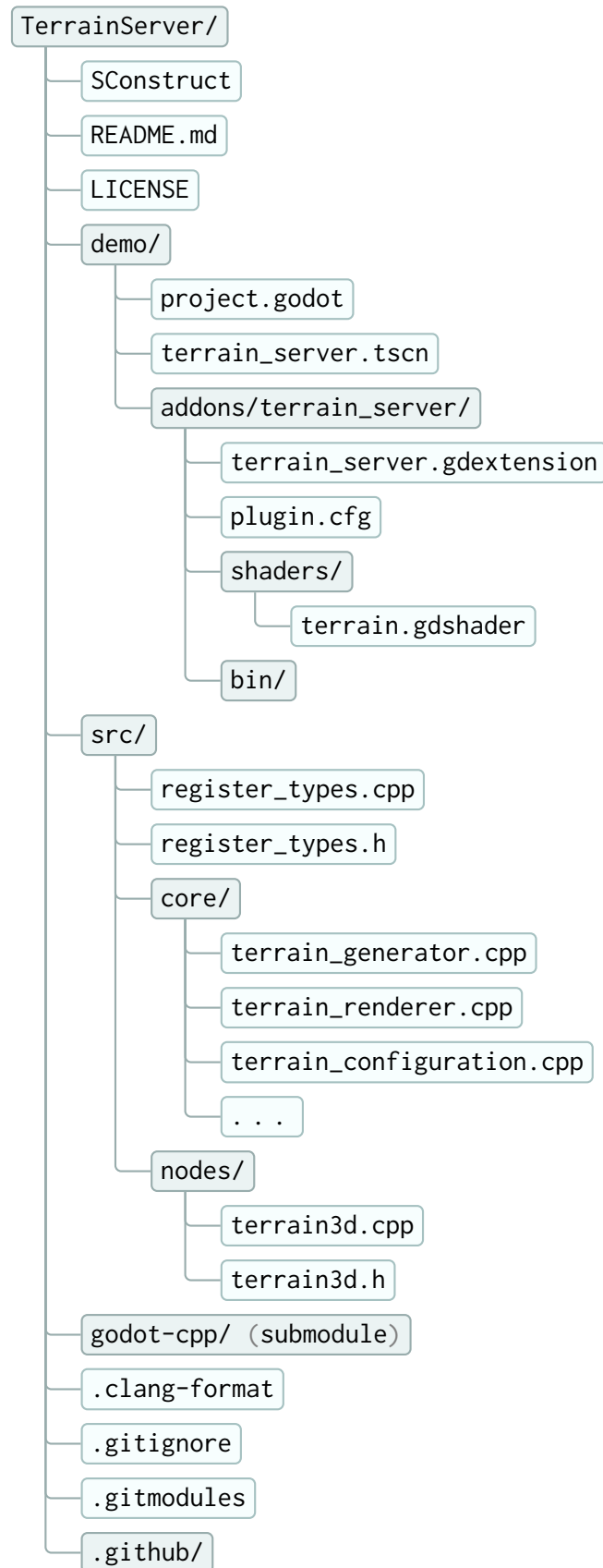


Figure 5.1: Project file structure.

CI/CD Setup

Furthermore, a [Continuous Integration/Continuous Deployment \(CI/CD\)](#) pipeline was implemented utilizing GitHub Actions to automate the plugin's build and release processes. This infrastructure includes cross-compilation for multiple target platforms (Linux, Windows, and macOS) and the automated deployment of new versions to GitHub Releases upon tag creation.

The [CI/CD](#) architecture relies on two primary workflows:

- **build.yml**: A workflow dedicated to cross-compiling the plugin. Triggered automatically on every push or pull request to the main branch, it generates the necessary build artifacts for each supported operating system. This workflow prevents integration of breaking changes. It is useful because it can be pretty common for some changes to work in the developer's local environment but fail to compile in other environments due to differences in OS, compiler versions, or library dependencies. By enforcing that every change must pass through this automated build process, it ensures that the codebase remains stable and functional for all users, regardless of their specific setup. It also eliminates the overhead of manual verification.
- **release.yml**: A deployment workflow configured to automatically package the compiled binaries and publish them to GitHub Releases whenever a new version tag is pushed to the repository.

With this initial configuration, a robust foundation for the project was established. The focus then shifted to implementing the core functionalities of the plugin, specifically architecting the primary classes responsible for terrain generation and rendering.

Proof of Concept

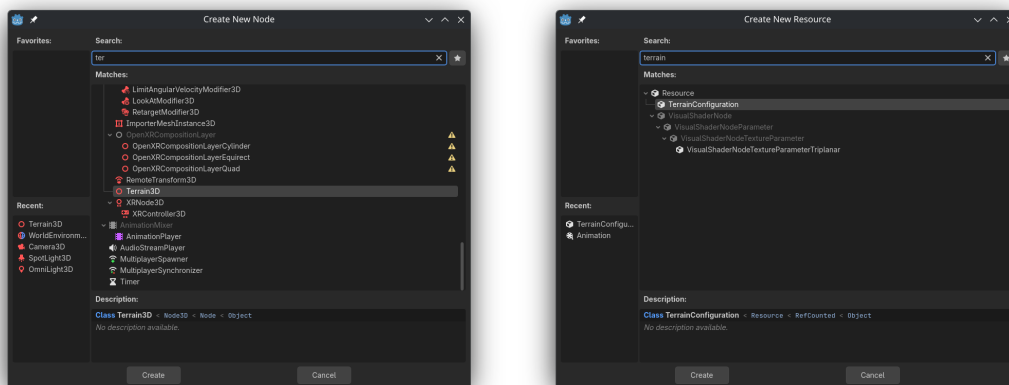
Upon successfully compiling the plugin and integrating it into the Godot Engine, development commenced on a preliminary iteration of the terrain generation system. This version served as a [Proof of Concept \(PoC\)](#) for real-time procedural generation. The initial implementation was executed entirely on the CPU. The primary objective of this stage was to solidify the foundational architecture and to assimilate the GDExtension API and Godot's node lifecycle.

The fundamental classes implemented during this architectural phase include:

- **register_types**: The standard GDExtension entry point, responsible for binding and registering the plugin's custom classes within Godot's ClassDB system.
- **Terrain3D (Orchestrator Node)**: Inheriting from Node3D, this is the primary node instantiated within the Godot scene tree. It acts as the central coordinator for the three core modules: `TerrainGenerator`, `TerrainRenderer`,

and `TerrainConfiguration`. It manages the node's lifecycle and listens to configuration changes to automatically trigger terrain regeneration.

- **TerrainConfiguration (Data Container):** A class that inherits from `Resource` and encapsulates all procedural parameters, such as `height_scale`, `mesh_resolution`, `terrain_size`, materials, and the `FastNoiseLite` resource. By exposing a custom resource to the editor, it allows developers to fine-tune the terrain visually. Upon any parameter modification, it emits a `changed` signal to notify the `Terrain3D` node.
- **TerrainGenerator (Generation Logic):** Responsible for calculating terrain heights and constructing the `ArrayMesh` using Godot's `SurfaceTool`. It retrieves the active parameters from the `TerrainConfiguration` to compute the final geometry. During this preliminary phase, the procedural generation execution is strictly CPU-bound.
- **TerrainRenderer (Visualization):** Encapsulates rendering logic by managing a `MeshInstance3D` child node named *"TerrainMesh"*. It assigns the geometry computed by the `TerrainGenerator` and safely handles memory updates or cleanups when parameters change.



(a) `Terrain3D` node in the Godot Create New Node dialog. (b) `TerrainConfiguration` resource in the Godot Create New Resource dialog.

Figure 5.2: (a) `Terrain3D` node. (b) `TerrainConfiguration` resource.

Result

With the implementation of these classes, a preliminary iteration of the system was consolidated. In this initial prototype, it is possible to instantiate a `Terrain3D` node within the Godot scene tree, bind a custom `TerrainConfiguration` resource to it, and create a basic terrain mesh generated from procedural noise. Although this early version lacks the performance optimizations and advanced features slated for subsequent stages, it fully achieves its purpose. Specifically, it provides an

architecture upon which more complex functionalities can be iteratively integrated, while assimilating the GDExtension API.

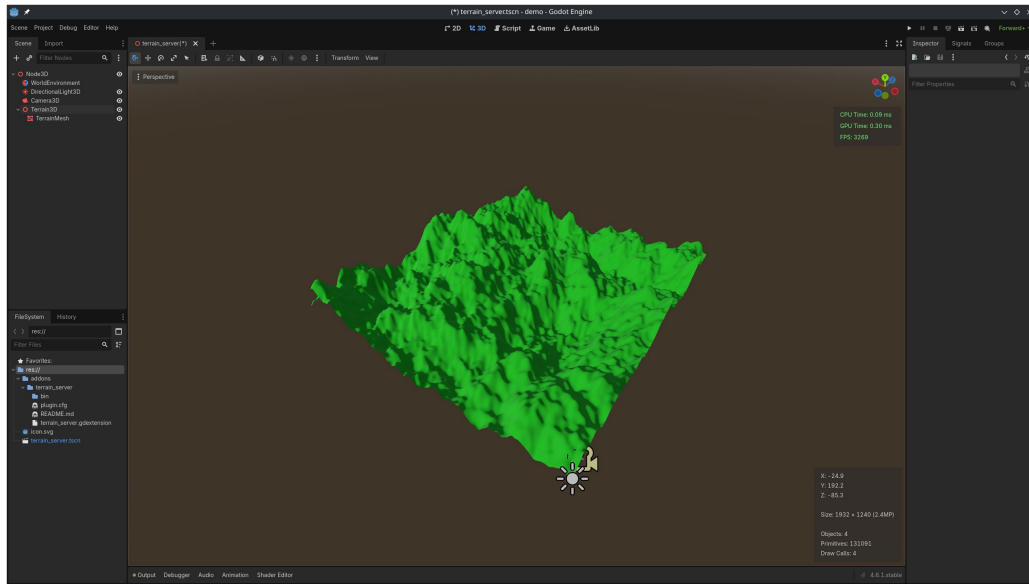


Figure 5.3: Preliminary PoC of the terrain generation system running within the Godot editor.

5.2 Phase 1: Infinite Terrain Rendering

The objective of this phase is to have an infinite terrain rendering system using the Geometry Clipmap technique. By the end of this phase, the system should be capable of rendering an infinite terrain without visual artifacts or geometry cracks and allow the camera to move freely through the generated world.

Architecture Improvements

The first step in this phase is to enhance the existing architecture to support the new requirements. This involves several key tasks:

- Adding new parameters to the `TerrainConfiguration` custom resource to support the additional features required for infinite terrain rendering, such as `_clipmap_levels`, which defines the number of LOD for the terrain, and `_noise_base_frequency`, which controls the frequency of the noise used for terrain generation.
- Refactoring the `TerrainRenderer` to use the `RenderingServer` directly, eliminating the overhead of using a `MeshInstance3D` in the scene tree. This change is expected to improve performance by reducing the overhead associated with scene tree management.

- Migrating the terrain generation process to the GPU using a Vertex Displacement Shader that reads from heightmap textures. The idea is to create a flat plane mesh in the CPU and then displace the vertices in the vertex shader using heightmaps generated using FastNoiseLite.
- Since the terrain node needs to use a `ShaderMaterial` to apply the Vertex Displacement Shader, we can't use a `StandardMaterial` for the terrain material. Since we can't have two materials on the same mesh, we will have to create a material system to be able to use the vertex displacement shader in the `ShaderMaterial` and later be able to define textures for the different biomes. This will likely be implemented in [Section 4.5 → p. 72](#).
- Modify the mesh generation logic to create the geometry clipmaps based on the paper “Geometry Clipmaps: Terrain Rendering Using Nested Regular Grids” [\[63\]](#). This technique allows for efficient rendering of large terrains by using multiple levels of detail (LOD) that adapt based on the camera's position.
- Synchronize the center of the clipmap with the camera's position to enable infinite movement through the terrain.
- Implement LOD strategies, such as vertical skirts, to hide the transitions between different levels of grid detail.

Adding the new parameters to the `TerrainConfiguration` resource is straightforward and involves defining new properties that can be edited in the Godot editor.

```
1 class TerrainConfiguration : public godot::Resource {  
2     GDCLASS(TerrainConfiguration, godot::Resource);  
3  
4     private:  
5         double _height_scale;  
6         int _mesh_resolution;  
7         float _terrain_size;  
8  
9         int _noise_octaves;  
10        float _noise_base_frequency;  
11        float _noise_lacunarity;  
12        float _noise_gain;  
13  
14        int _clipmap_levels;  
15        godot::Ref<godot::Texture2D> _albedo_texture;  
16  
17        godot::Ref<godot::FastNoiseLite> _internal_noise;  
18        godot::Ref<godot::ImageTexture> _noise_preview;  
19    };
```

Figure 5.4: `TerrainConfiguration` parameters.

Refactoring the `TerrainRenderer` to use the `RenderingServer` directly involves removing the dependency on the `MeshInstance3D` and instead creating and managing the mesh directly through the rendering server. We do not need to bind any method to the editor for this, since the `TerrainRenderer` is an internal class that is not exposed to the editor. In [Section 5.1 → p. 74](#), the `TerrainRenderer` had a function `generate_mesh()` that instances a `MeshInstance3D` and assigned the generated mesh to it. Now instead of doing that, the `TerrainRenderer` will have a function `rebuild_mesh()` that will retrieve the `RenderingServer` singleton and, using the functions from `TerrainGenerator`, creates the mesh and assigns the Vertex Displacement Shader to it.

The Vertex Displacement Shader will be responsible for displacing the vertices in the Y axis based on the heightmap texture generated by the `FastNoiseLite` resource. The shader will read the height value from the heightmap texture and multiply it by the `height_scale` parameter to determine how much to displace the vertex.

```
1  shader_type spatial;
2
3  uniform sampler2D height_map;
4  uniform float height_scale;
5  void vertex() {
6      float h = texture(height_map, UV).r;
7      VERTEX.y += h * height_scale;
8  }
9  void fragment() {
10     ALBEDO = vec3(0.5);
11     ROUGHNESS = 0.8;
12 }
```

Figure 5.5: Vertex Displacement Shader.

Also, there is a new `update_camera_position()` function that will update the position of the clipmap based on the camera's position.

Challenges Encountered & Solutions

During the implementation of the infinite terrain rendering system, several challenges were encountered:

- **FastNoiseLite Texture 8 bits Limitation:** The `FastNoiseLite` resource generates heightmap textures in 8 bits, which limits the number of height levels to 256. So this result in a low detailed terrain. Also, by passing a texture generated in the CPU to the shader, we are limited to a finite terrain that repeats itself every time the camera moves too far. This was ok for the [PoC](#), but for an infinite terrain we need to generate the noise directly in the GPU. The solution to this problem was to migrate the noise generation to the GPU, allowing us to generate heightmap textures with higher bit depth

and avoid the overhead of transferring large heightmap textures from the CPU. Since the bandwidth of the GPU is the major bottleneck, it is more efficient to generate the noise directly in the GPU which is "free" as the GPU can handle it with not much overhead. So migrating the noise generation to the GPU not only allows for a more detailed terrain but also enables an infinite terrain that does not repeat itself.

- **Screen Space Effects Artifacts:** When using the depth buffer for screen space effects, such as [Screen Space Ambient Occlusion \(SSAO\)](#) or [Screen Space Reflections \(SSR\)](#), there are some visual artifacts in the terrain, specifically in the areas where the terrain has a valley, there are darker shadows line following the mesh edges.
- **UV Swimming:** This is a common issue when using clipmaps [LOD](#) technique. As the camera moves, the UV coordinates of the terrain mesh can shift, causing textures to appear to "swim" across the surface. This can be fixed by using a world-space UV mapping technique, where the UV coordinates are calculated based on the world position of the vertices rather than the local position. This way, the textures will remain stable and will not swim as the camera moves.

Result

By the end of this phase, the system is capable of rendering an infinite terrain without visual artifacts or geometry cracks. The camera can move freely through the generated world, and the terrain is generated in real-time using the GPU-based procedural generation. The implementation of the Geometry Clipmap technique allows for efficient rendering of large terrains by using multiple levels of detail (LOD) that adapt based on the camera's position. The use of a Vertex Displacement Shader enables dynamic terrain deformation based on procedural noise, resulting in a more detailed and visually appealing environment.

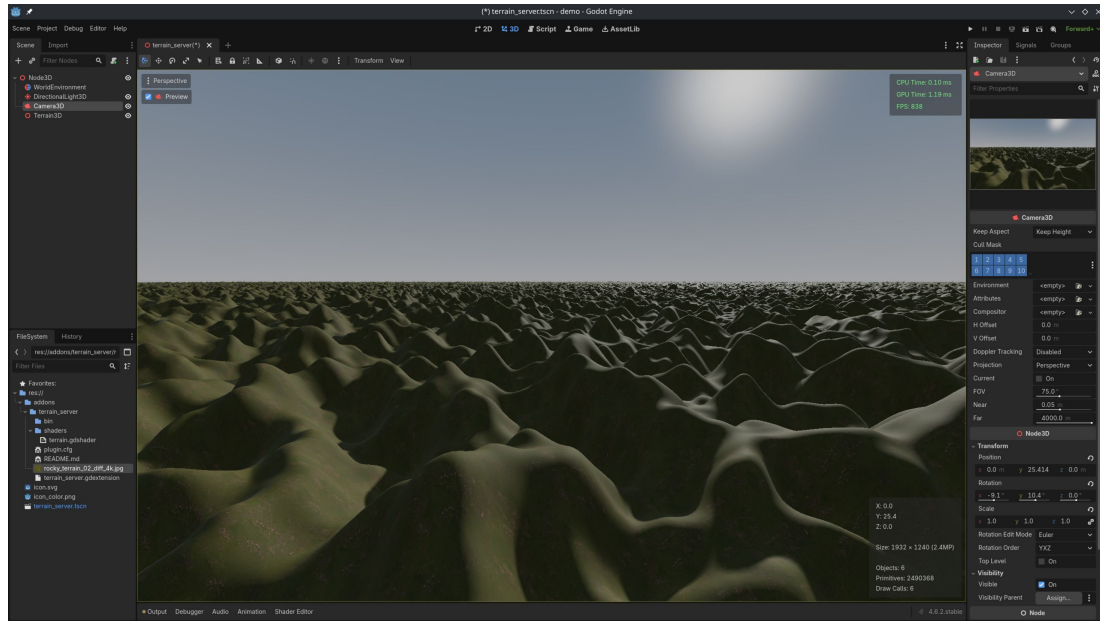


Figure 5.6: Infinite terrain using Geometry Clipmaps, Vertex Displacement Shader, and CPU-based noise.

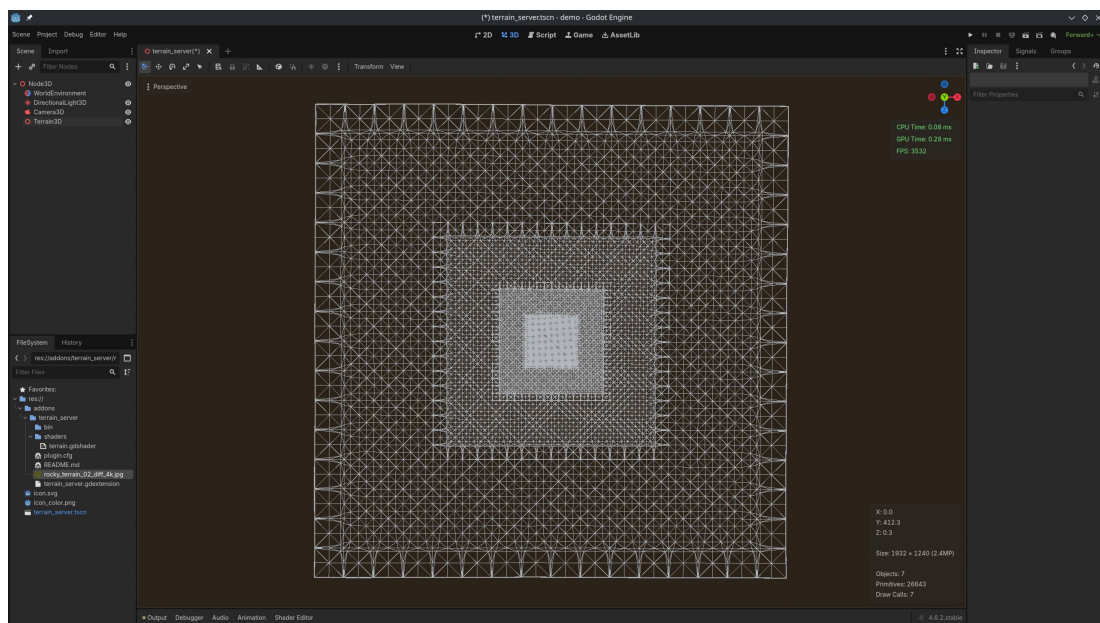


Figure 5.7: Visualization of the Geometry Clipmap levels. 4 clipmap levels, terrain size of 64, mesh resolution of 64.

5.3 Phase 2: GPU-Based Procedural Generation

Previously, the idea for this phase was to use Godot's `WorkerThreadPool` to offload the terrain generation to a separate thread. But after seeing the implementation benefits of migrating the terrain generation to the GPU, it was decided to skip the

multi-threading implementation and directly implement the GPU-based procedural generation.

The main benefits of using the GPU for procedural generation are that we're not limited to the texture size and bit depth. We can have the noise values calculated in the shader for each vertex, allowing us to have an infinite terrain with "infinite" height levels. Also, by generating the noise directly in the GPU, we can avoid the overhead of transferring large heightmap textures from the CPU to the GPU, which can be a significant bottleneck. Also if we generate the noise in the CPU, we are limited to a finite terrain that repeats itself every time the texture is repeated. By generating the noise directly in the GPU, we can have an infinite terrain that does not repeat itself.

The shader implementation

In the previous phase, the vertex shader was responsible for displacing the vertices based on a heightmap texture generated by the `FastNoiseLite` resource on the CPU. Now, the noise generation is implemented directly in the shader, eliminating the need to generate and transfer heightmap textures to the VRAM. The shader calculates the noise value for each vertex dynamically based on its world position and the procedural parameters defined in the `TerrainConfiguration` resource.

Fractal Brownian Motion (fBm) Implementation

The foundation of this system relies on a native 2D Perlin noise implementation, which is layered using `fBm` to achieve an organic, mountainous look. The following code snippet demonstrates how the `fBm` is calculated and subsequently used to displace the vertex along the Y-axis:

```

1 // Fractal Brownian Motion (fBm)
2 float get_height_at(vec2 world_xz) {
3     float h = 0.0;
4     float amplitude = 1.0;
5     float frequency = base_frequency;
6     float max_amplitude = 0.0;
7
8     // Iterate to add detail (octaves)
9     for(int i = 0; i < octaves; i++) {
10         h += noise(world_xz * frequency) * amplitude;
11         max_amplitude += amplitude;
12         amplitude *= gain;
13         frequency *= lacunarity;
14     }
15
16     // Normalize to [-1, 1] range
17     h = h / max_amplitude;
18     return h * height_scale;
19 }
20
21 void vertex() {
22     // Calculate World Space coordinates
23     vec3 world_pos = (MODEL_MATRIX * vec4(VERTEX, 1.0)).xyz;
24     v_world_xz = world_pos.xz;
25
26     // Sample procedural noise and apply displacement
27     float final_height = get_height_at(v_world_xz);
28     VERTEX.y = final_height / length(MODEL_MATRIX[1].xyz);
29
30     // Note: Normal, tangent, and binormal recalculations are
31     // performed afterwards to ensure correct lighting.
32 }

```

Figure 5.8: Vertex Displacement Shader with GPU-based procedural noise generation.

Furthermore, because displacing the geometry in the shader invalidates the original flat mesh normals, the lighting would appear incorrect. To solve this, the shader uses finite difference approximations to evaluate the height at four neighboring points around the current vertex. With these height differences, the shader analytically reconstructs the normal, tangent, and binormal vectors in world space before converting them back to local space for Godot's rendering pipeline.

Noise Function Implementation

Unlike C++ or GDScript, GLSL-based shaders do not possess a built-in random number generator. Therefore, randomness must be derived from spatial coordinates deterministically. This is achieved through a custom `hash()` function. The function

relies on the `fract(sin(x) * constant)` pattern, a standard computer graphics technique to generate pseudo-random gradients. Because it is purely deterministic, the same (x, z) world coordinate will always return the exact same gradient vector, ensuring that the terrain remains static and consistent frame after frame.

```

1 // Hash function to generate a pseudo-random gradient vector
2 // based on the input position
3 vec2 hash(vec2 p) {
4     p = vec2(dot(p, vec2(127.1, 311.7)), dot(p, vec2(269.5,
5         183.3)));
6     return -1.0 + 2.0 * fract(sin(p) * 43758.5453123);
7 }
8
9 // Perlin Noise 2D Algorithm
10 float noise(in vec2 p) {
11     vec2 i = floor(p); // Integer grid cell coordinates
12     vec2 f = fract(p); // Fractional offset within the cell
13
14     // Quintic interpolation curve for smooth derivatives
15     vec2 u = f * f * f * (f * (f * 6.0 - 15.0) + 10.0);
16
17     // Bilinear interpolation of the dot products
18     return mix(
19         mix(dot(hash(i + vec2(0.0, 0.0)), f - vec2(0.0, 0.0)),
20             dot(hash(i + vec2(1.0, 0.0)), f - vec2(1.0, 0.0)), u.x),
21         mix(dot(hash(i + vec2(0.0, 1.0)), f - vec2(0.0, 1.0)),
22             dot(hash(i + vec2(1.0, 1.0)), f - vec2(1.0, 1.0)), u.x),
23         u.y);
24 }

```

Figure 5.9: Pseudo-random hash and Perlin Noise functions implementation.

Building upon the hash function, the `noise()` implementation defines a classic 2D Perlin Noise algorithm. It divides the 2D space into a grid of unit squares, determining which grid cell the current vertex belongs to using the `floor()` function, and calculating its local offset using `fract()`.

Terrain Shaping: Ridged Multifractal, Domain Warp and Provinces

This subsection documents a later revision of `get_height_at()`, made after the biomes (Section 5.5 → p.94) and the editor and validation work (Section 5.6 → p.111) and (Section 5.7 → p.113) described later in this chapter were already in place, rather than during Phase 2's original implementation. Plain `fBm` gives every point on the terrain the same character, rounded and dune-like, with the same relief everywhere, and that stopped being good enough once biomes gave the surface material variety the underlying shape did not have. (Section 2.4 → p.46) already sketched two textbook ways to break that uniformity: folding each octave to raise

ridges, and warping the sample coordinate with a second noise field. The naive forms shown there, $1.0 - |\text{noise}(x)|$ for ridges and an unconditional iterative warp, are not something a real height function can apply everywhere at once. Ridges belong on a mountain range and look wrong on a plain, and a warp strong enough to be visible is also strong enough to corrugate flat lowland into parallel wrinkles. `get_height_at()` was extended with both techniques, plus a third that decides where they apply, gating each by a low-frequency *province* field rather than mixing them in uniformly.

A single noise sample, taken at `noise_continent_frequency` and remapped through a `smoothstep`, defines the province field: a mask $\in [0, 1]$, near 0 for lowland, near 1 for highland, with `noise_continent_contrast` controlling how wide the transition between them is. Scaled by `noise_continent_influence`, this mask drives three things: it offsets the base height by `noise_continent_elevation` around a `noise_continent_sea_level` threshold, so highlands sit measurably higher rather than merely rougher; it flattens the calmest provinces through `noise_relief_floor` without touching the mountains; and it scales how strongly the two shaping techniques below apply, so only highland provinces get ridged and warped while lowlands stay plain [fBm](#).

Ridging replaces the crude $1 - |n|$ fold with a rounded version, applied per octave rather than to the finished sum:

$$\left(\text{ridge_offset} - \sqrt{n^2 + \text{ridge_crest_rounding}^2} \right)^2$$

The square root rounds off the crease a plain absolute value would leave at $n = 0$, band-limiting it to roughly `ridge_crest_rounding` instead of injecting detail down to a single mesh edge. Each octave is further weighted by how prominent the previous one was, scaled by `ridge_weight_gain`, concentrating detail onto ridges that are already tall and leaving the valleys between them simple, closer to how real mountain ranges look than uniformly busy noise would. The ridged and plain [fBm](#) sums share the same octave loop and are `mix()`ed by `ridge_amount` times the province mask, so turning ridging off costs nothing beyond the branch it skips.

The domain warp is a single evaluation, not the iterative one in [Section 2.4](#) → p. 46: two extra noise taps at `noise_warp_frequency`, offset so they don't correlate with the elevation and climate fields, displace the sample position by up to `noise_warp_amount` world units before the octave loop runs. It is the *product* of amount and frequency that sets how much the warp stretches the terrain, so keeping the frequency well below `noise_base_frequency` keeps ridgelines meandering instead of shearing into corrugations, a failure mode found on the demo's lowlands while tuning these defaults.

```

1  float height_from_mask(vec2 world_xz, float mask) {
2      vec2 p = world_xz;
3      float warp = warp_amount * mask;
4      if (warp > 0.0) {
5          vec2 w = vec2(
6              noise((world_xz + WARP_OFFSET_X) * warp_frequency),
7              noise((world_xz + WARP_OFFSET_Y) * warp_frequency));
8          p += w * warp;
9      }
10
11     float relief = relief_at(p, ridge_amount * mask);
12     if (redistribution != 1.0) {
13         relief = sign(relief) * pow(abs(relief), redistribution);
14     }
15
16     float h_norm = continent_elevation * (mask -
continent_sea_level)
17         + relief * mix(relief_floor, 1.0, mask);
18     return clamp(h_norm, -1.5, 1.5) * height_scale;
19 }

```

Figure 5.10: Combining the domain warp, the ridged relief and the redistribution curve under the province mask into the final height.

Finally, `noise_redistribution` applies a sign-preserving power curve, $\text{sign}(h) |h|^r$, to the finished normalized height before `height_scale` is applied. Values of $r > 1$ push the common mid-range elevations toward the extremes, producing more valley floor and plateau and fewer, sharper peaks, which matches the skewed elevation distribution of real terrain more closely than noise’s naturally symmetric one.

Every one of these terms is duplicated verbatim in `TerrainNoise::get_height_at()`, so the CPU/GPU parity established in [Section 5.4](#) → p. 88 covers the shaped terrain automatically. The read-only `noise_preview` on `TerrainConfiguration`, previously rendered from a separate `FastNoiseLite` instance, now calls a `build_noise_params()` helper and runs through the exact same height function, and widens the area it samples once `noise_continent_influence` is above zero, so both the province scale and the ridges inside it stay visible in the same thumbnail.

5.4 Phase 3: Physics & Collision

In this phase, the goal is to implement physics and collision detection for the generated terrain. The expected outcome is that the terrain should have a collision shape that matches the generated geometry, allowing for realistic interactions with physics objects in the scene. Two new classes are introduced to achieve this, following the same modular split used by `TerrainGenerator`, `TerrainRenderer` and

TerrainConfiguration: **TerrainPhysics**, which owns a static heightmap collision body, and **TerrainNoise**, a CPU-side port of the noise function used by the shader.

Decoupling Collision from the Render Mesh

The first obstacle is architectural rather than physical. The clipmap render mesh, as established in [Section 5.3 → p.82](#), is a flat grid that is displaced vertically *inside the vertex shader* (see [Figure 5.8 → p.84](#)), entirely on the GPU. **PhysicsServer3D**, however, needs geometry or a heightmap, on the CPU side to build a collision shape. There is no displaced mesh anywhere to hand it. The vertices only exist in their final, displaced form for the duration of a single frame, inside the GPU pipeline.

Reading the displaced vertices back from the GPU every time the collision grid needs updating would reintroduce exactly the kind of transfer bottleneck that motivated moving noise generation to the shader in the first place (see [Section 5.3 → p.82](#)). Instead, **TerrainPhysics** independently evaluates the *same* noise function on the CPU, through the new **TerrainNoise** class, over a small grid around the player, and builds a **HeightMapShape3D** ([Section 2.6 → p.58](#)) from the resulting elevation values using `PhysicsServer3D::heightmap_shape_create` [\[13\]](#).

This is a recurring problem with any procedural technique that displaces geometry exclusively on the GPU: as soon as something other than the renderer needs to know the surface's shape, whether that is physics, a gameplay query, or pathfinding for AI navigation, the shader math has to grow a second, CPU-evaluable implementation. This gap is filled by **TerrainNoise**, and keeping it faithful to the shader turns out to be one of the hardest parts of this phase.

Achieving Deterministic CPU/GPU Noise Parity

Calculating what the shader already computes on the CPU, through **TerrainNoise**, is not enough by itself. The two implementations have to produce identical results, bit-for-bit, for every possible input. Otherwise, the collision shape will not match what the player sees. This is where the pseudo-random gradient hash from [Figure 5.9 → p.85](#) became a problem. That hash relies on the `fract(sin(x) * constant)` pattern, a common and cheap way to fabricate a gradient in GLSL, but it has two properties that only become visible once something other than the renderer starts depending on it:

- **It is not portable.** The precision of `sin()` for large arguments varies across GPU vendors and drivers, since it is implementation-defined, so the rendered terrain shape is not even guaranteed to be identical on two different machines.
- **It is not reproducible on the CPU.** Matching GLSL's `sin()` bit-for-bit in C++ is impractical, so any CPU-side reimplementations of this hash function would be different from the GPU one, one player would sink through

visibly solid ground, another would hover just above it, depending on tiny floating-point differences accumulated over many octaves.

The fix is to replace the hash's source of randomness with integer bit-mixing instead of a transcendental function, since integer arithmetic is deterministic across platforms in a way `sin()` is not. Concretely, the `hash()` function used in [Figure 5.9](#) → p.85 is replaced by a `pcg2d`-style hash, a small sequence of multiplications, additions and XOR-shifts on unsigned 32-bit integers, that indexes into a fixed 8-entry gradient table shared, entry for entry, between `terrain.gdshader` and `terrain_noise.h`:

```

1 // Fixed gradient table, defined identically on the CPU and the
  GPU
2 const vec2 GRADIENTS[8] = vec2[](
3     vec2( 1.0,  0.0), vec2( 0.707,  0.707),
4     vec2( 0.0,  1.0), vec2(-0.707,  0.707),
5     vec2(-1.0,  0.0), vec2(-0.707, -0.707),
6     vec2( 0.0, -1.0), vec2( 0.707, -0.707)
7 );
8
9 // PCG-style bit-mixing hash on 32-bit unsigned integers
10 uvec2 pcg2d(uvec2 v) {
11     v = v * 1664525u + 1013904223u;
12     v.x += v.y * 1664525u;
13     v.y += v.x * 1664525u;
14     v ^= v >> 16u;
15     v.x += v.y * 1664525u;
16     v.y += v.x * 1664525u;
17     return v;
18 }
19
20 // Drop-in replacement for the sin()/fract() hash from Fig. 4.11:
21 // selects a gradient from the fixed table using integer ops only
22 vec2 hash(vec2 cell) {
23     uvec2 h = pcg2d(uvec2(uint(cell.x), uint(cell.y)));
24     return GRADIENTS[h.x % 8u];
25 }

```

Figure 5.11: Integer-hash replacement shared between the GLSL shader and the C++ `TerrainNoise` port.

Because `hash()` keeps the same signature as before, the surrounding `fBm`/Perlin noise code from [Figure 5.8](#) → p.84 and [Figure 5.9](#) → p.85 didn't need to change on either side, only the source of randomness did.

Physics-Specific Level of Detail

Unlike the render mesh, `TerrainPhysics` does not track the full extent of the clipmap. Collision is only ever relevant close to the player, an actor never touches terrain several clipmap levels away, so `TerrainPhysics` maintains its own, independently sized square region around the focus point, controlled by a new `physics_range` property (editor range 8–512, default 64) on `TerrainConfiguration`:

```
1 // New "Physics" property group on TerrainConfiguration
2 private:
3     float _physics_range;
4     uint32_t _physics_collision_layer;
5     uint32_t _physics_collision_mask;
```

Figure 5.12: New Physics property group added to `TerrainConfiguration`.

The resolution of this region, in grid cells, is derived from `terrain_size / mesh_resolution`, matching the cell size of the finest clipmap ring, and is clamped to `[8, 256]` cells regardless of what that division yields. This bounds the size of the heightmap shape, whose cost is $(\text{grid_resolution} + 1)^2$ floats, to a fixed budget independently of how large `terrain_size` or `clipmap_levels` are configured to be, both of which can grow arbitrarily for the visual clipmap without making the collision grid any more expensive.

Framed this way, `physics_range` is a [LOD](#) strategy applied to collision rather than to rendering, a deliberate, physics-specific level of detail, distinct from the clipmap’s own [LOD](#) rings from [Section 5.2 → p. 78](#). It would be easy for a reader to assume collision reuses the same clipmap levels used for rendering; it does not, the two are sized and triggered independently, and [Section 5.4 → p. 90](#) covers how the physics grid decides when to move.

Asynchronous Regrid Pipeline

Re-evaluating `TerrainNoise` over every cell of the physics grid is not free. It costs an $O(\text{grid_resolution}^2)$ amount of work. Two separate problems follow from that cost, and `TerrainPhysics::update_focus_position()` addresses each of them independently.

The first is deciding *when* to regrid. The focus position is first snapped to the physics grid’s cell size to obtain a candidate origin, but that candidate does not immediately trigger a rebuild: a rebuild only happens if the grid has never been built yet, or if the candidate origin has moved at least 25% of `physics_range` away from the origin the grid was last built around.

```

1 void TerrainPhysics::update_focus_position(const Vector3
    &p_focus) {
2     Vector2 candidate = (Vector2(p_focus.x, p_focus.z) /
        _cell_size).floor()
3         * _cell_size;
4
5     if (_has_built && candidate.distance_to(_last_built_origin)
6         < 0.25 * _physics_range) {
7         return; // Still comfortably inside the current grid
8     }
9
10    _request_regrid(candidate);
11 }

```

Figure 5.13: Grid-snap margin guarding regrid requests.

Without this margin, the grid would regrid on almost every frame the player crosses a cell boundary, since that happens continuously while moving. The margin is a hysteresis pattern: the collision surface only needs to re-center often enough to keep the focus point comfortably inside the grid before it reaches the edge, not on every single cell transition, and it plays the same role for the physics grid as the clipmap’s own re-centering threshold does for the render mesh.

The second problem is *where* the $O(n^2)$ noise function evaluation runs. `_compute_heightmap_task` is submitted to Godot’s `WorkerThreadPool`, so it never blocks the main thread. `update_focus_position()` polls `is_task_completed()` once per frame; only once the task reports completion is the (by then non-blocking) result collected and pushed into `PhysicsServer3D` through `shape_set_data` and `body_set_state`. That last step has to run on the main thread regardless, since mutating a physics server resource from a worker thread is not safe, so the task itself only *computes* the heightmap; *applying* it is a separate step, gated on the task’s completion. At most one regrid is ever being calculated: a new one is not started while another is still running.

The trade-off this introduces is intentional and worth stating explicitly. The collision surface can lag the focus point by a frame or two while a regrid is in flight. Given that regrids only fire once movement crosses the hysteresis margin from [Figure 5.13](#)^{→ p.91}, this latency is judged an acceptable cost in exchange for never saturating the frame on an $O(n^2)$ noise loop.

Height Queries and the `focus_path` Property

Not every part of the game needs collision response, gameplay code such as actor spawning, object placement, or camera clamping typically just needs to know the terrain’s height at an arbitrary point, right now, including points outside the current physics grid or ahead of a pending regrid. Sampling the collision heightmap for this would inherit both of its limitations: the fixed `physics_range` extent

from [Section 5.4 → p.90](#) and the one-or-two-frame staleness from [Section 5.4 → p.90](#). Terrain3D instead exposes a public `get_height_at(Vector2)` method that forwards through `TerrainPhysics` directly to `TerrainNoise::get_height_at()`, evaluated on demand rather than sampled from the grid. The two height-query paths therefore share the same underlying math but serve different consumers: one is backed by a `PhysicsServer3D` shape for actual collision response, the other is a pure function call for queries that need an answer immediately, anywhere, at the cost of paying the noise evaluation every time it is called.

A related decoupling problem is *what point* both the renderer and the physics grid should center themselves on. Until this phase, both the clipmap's re-centering ([Section 5.2 → p.78](#)) and the physics regrid were driven directly by `get_viewport()->get_camera_3d()`. This phase introduces a `focus_path` `NodePath` property on `Terrain3D`: when it resolves to a `Node3D`, that node's global position drives *both* the renderer and `TerrainPhysics`, and the active camera is only used as a fallback when `focus_path` is left unset. This unblocks two concrete cases:

- **Headless or dedicated servers**, where no `Camera3D` exists at all, so the previous camera-only code path simply did nothing; a server still needs the collision grid to follow each connected player.
- **A camera that is not the gameplay-relevant point**, such as a third-person camera trailing behind a character, a free-flying spectator camera, or split-screen, where the terrain should stay centered on the character rather than wherever the camera happens to be looking from.

Equally important is a consistency guarantee this change provides almost for free: both the renderer and `TerrainPhysics` are now driven from a *single* focus position resolved once per frame, inside `Terrain3D::_process`, rather than each independently querying the viewport. This guarantees the visible clipmap and the collision grid can never disagree about which point they are centered around.

Challenges Encountered & Solutions

Two rendering bugs were detected during this phase:

- **Clipmap Culling Against a Flat AABB**: `RenderingServer` computes each clipmap block's culling bounds from the flat, undisplaced base mesh, since the actual displacement happens inside the vertex shader and is invisible to the engine's own automatic [AABB](#) computation. Once `height_scale` pushed vertices far enough vertically, blocks whose flat [AABB](#) no longer overlapped the camera frustum were culled even though their displaced geometry was still clearly on screen, causing terrain to pop out of view near the screen edges. The fix calls `instance_set_custom_aabb()` with a manually computed bound (a vertical half-height of `height_scale × 1.25`, a fixed safety margin) sized to the shader's actual displacement range, applied per clipmap-level instance inside `rebuild_mesh()`.

- **Redundant Rebuilds on Unrelated Edits:** both `TerrainRenderer` and `TerrainPhysics` react to the same, coarse-grained `TerrainConfiguration::changed` signal, but not every edit should cost the same amount of work. Naively, editing an unrelated property such as `albedo_texture` would reload and recompile the shader on the renderer's side, and force a full $O(n^2)$ heightmap recomputation on the physics side, even though neither collaborator actually depends on that property. The renderer now caches its shader RID and only reloads it from disk on a full teardown/setup, not on every edit, while `TerrainPhysics::set_configuration()` diffs the incoming noise, range and grid parameters against the previous ones and only marks the cached heightmap stale, forcing a regrid on the next `update_focus_position()` call, when a physics-relevant parameter actually changed.

Result

By the end of this phase, the terrain has a collision shape that stays synchronized with the geometry rendered on screen: the player can walk across the generated terrain, and both the visible surface and the physical one are driven by the same noise function evaluated bit-for-bit identically on the CPU and the GPU. Collision itself is resolved through a small, independently sized region that follows the focus point, regridding asynchronously and only when movement demands it, while a separate, always-accurate height query remains available to the rest of the game for anything that cannot wait on the collision grid to catch up. Two latent rendering bugs, an incorrect culling [AABB](#) and a hash function that was never actually portable, were also uncovered and fixed as a direct consequence of forcing the CPU and GPU to agree on the shape of the terrain.

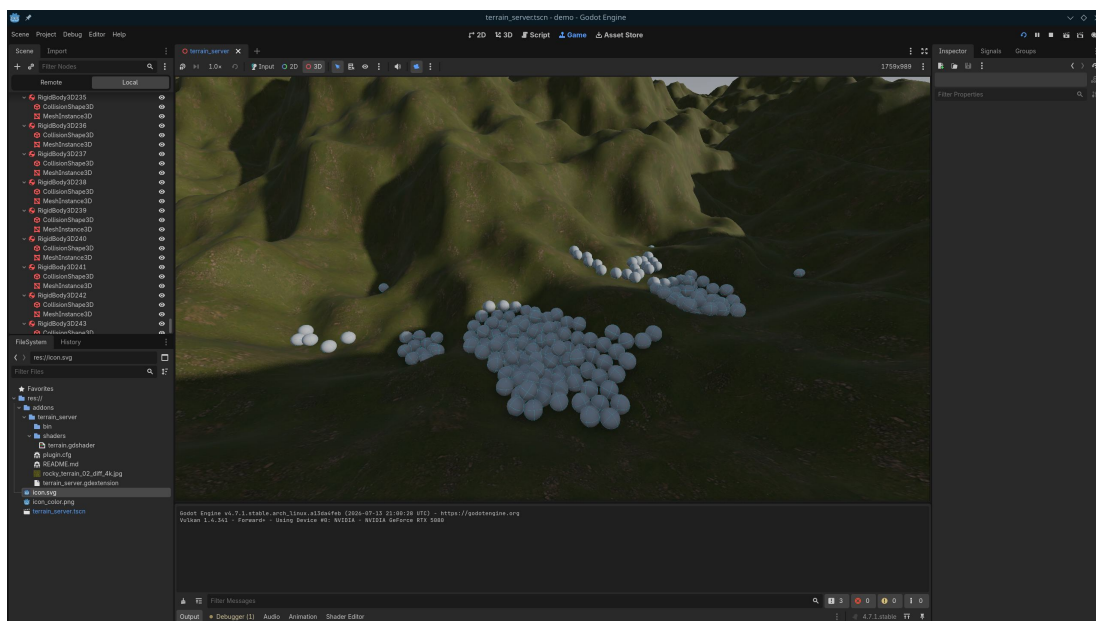


Figure 5.14: Multiple `RigidBody3D` objects interacting with the terrain.

5.5 Phase 4: Biomes & Environmental Variety

Up to this point the terrain has correct geometry and correct collision, but it is textured with a single `albedo_texture` stretched over the whole world by a fixed world-space UV scale, a placeholder left in [Section 5.3 → p.82](#) explicitly to be replaced here. The goal of this phase is to give the surface an appearance that varies depending on its location. Two new `Resource` classes are introduced for this, following the same pattern already used by `TerrainConfiguration`: `TerrainBiomeLayer`, which describes one biome as a set of textures plus the climate window it occupies, and `TerrainSlopeLayer`, which describes the rock exposed on steep faces. The bulk of the work, however, is in `terrain.gdshader`, whose `fragment()` function grows from a single texture fetch into a per-fragment classification, blend, projection and raymarch pipeline.

Modelling Climate: Temperature and Moisture Fields

[Section 2.5 → p.52](#) established that the noise-based approach to biome assignment simulates environmental properties as separate, overlapping scalar fields and then classifies the resulting tuple. This phase implements exactly that with two additional fields, temperature and moisture, evaluated from the same `noise()` function already used for elevation.

The two fields must be uncorrelated with each other and with the height field, otherwise every mountain would also be the same biome. Rather than instantiating three separately seeded noise generators, each field samples the *same* function at a different frequency and a large constant domain offset. Since the gradient hash from [Figure 5.11 → p.89](#) is derived from the coordinates of the grid cell a point falls into, shifting the input by several thousand units samples a completely different set of cells. This works as well as reseeding, and leaves a single noise implementation to maintain.

Moisture is pure noise normalized to $[0, 1]$. Temperature is not: a purely random temperature field would happily place a tropical biome on a mountain peak. Temperature is therefore driven primarily by altitude, falling as the terrain rises the way it does in the real atmosphere; noise only perturbs it, by an amount set by `temperature_noise_influence`. The result is remapped to $[-10, 30]$ °C, so that the thresholds authored on each biome layer read as real temperatures in the inspector rather than as abstract normalized values.

```

1  const float NOISE_MAX_AMPLITUDE = 0.70710678; // sqrt(2)/2, not
    1.0
2  const float TEMPERATURE_MIN_C = -10.0;
3  const float TEMPERATURE_MAX_C = 30.0;
4
5  float temperature_at(vec2 world_xz, float height) {
6      // Altitude drives the base temperature (lapse rate): high
        ground is cold
7      float altitude_norm = 1.0 - clamp(
8          (height / temperature_altitude_reference) * 0.5 + 0.5,
9          0.0, 1.0);
10
11     // Noise only perturbs it, so peaks stay cold and valleys
        stay warm
12     float n = noise(world_xz * temperature_frequency +
        temperature_offset);
13     float noise_norm = clamp(n / NOISE_MAX_AMPLITUDE, -1.0, 1.0)
        * 0.5 + 0.5;
14     float combined = clamp(altitude_norm
        + (noise_norm - 0.5) * temperature_noise_influence, 0.0,
        1.0);
15
16     return TEMPERATURE_MIN_C
17         + combined * (TEMPERATURE_MAX_C - TEMPERATURE_MIN_C);
18 }
19
20 float moisture_at(vec2 world_xz) {
21     float n = noise(world_xz * moisture_frequency +
        moisture_offset);
22     return clamp(n / NOISE_MAX_AMPLITUDE, -1.0, 1.0) * 0.5 + 0.5;
23 }

```

Figure 5.15: Temperature and moisture fields, shared between `terrain.gdshader` and `terrain_noise.h`.

The division by `NOISE_MAX_AMPLITUDE` deserves a note. The `noise()` function never actually reaches -1 or 1 : its largest possible value is $\sqrt{2}/2 \approx 0.707$, and even that requires the surrounding gradients to line up perfectly. Assuming the full $[-1, 1]$ range instead would squeeze both fields toward the middle, leaving the biomes at either extreme unreachable no matter what thresholds the user authors.

Both functions are also duplicated in `terrain_noise.h`, as a `TemperatureMoistureParams` struct next to the existing `FbmParams`, for the reason established in [Section 5.4](#) ^{→ p.88}: anything the GPU computes that gameplay also needs to know has to exist on the CPU too. This time the caller is `get_biome_at()` ([Section 5.5](#) ^{→ p.107}) rather than the collision shape, but the requirement is the same, and the integer hash from [Figure 5.11](#) ^{→ p.89} is what keeps the two versions in agreement.

Whittaker Classification as a Soft Per-Fragment Blend

With temperature and moisture available at every point, classification follows the Whittaker scheme presented in [Figure 2.24](#) → p.53. Each `TerrainBiomeLayer` declares temperature and moisture bounds, defining an axis-aligned rectangle in Whittaker space. The fragment shader then asks which rectangles contain the current fragment.

Implementing this as the literal lookup table described in [Section 2.5](#) → p.52 would be a mistake in a fragment shader. A hard classification returns exactly one biome per fragment, so the boundary between two rectangles becomes a hard, per-pixel seam cutting across the terrain, and, worse, that seam follows the noise contour lines exactly, which reads as an obvious mathematical artifact rather than as a landscape. Real biome boundaries are [ecotones](#), gradual transition zones, so the classification is instead evaluated as a *weight* per layer and every layer with a non-negligible weight contributes to the final surface.

Each of the four rectangle edges becomes a `smoothstep` rather than a comparison, and the width of that smoothstep, the softness, is derived from the layer's own range: its width times a `blend_softness` factor the layer itself exposes (a quarter by default), floored at a tiny minimum so that a degenerate window (a layer whose minimum equals its maximum) or a `blend_softness` of zero still produces a usable gradient instead of collapsing back into a hard edge.

```

1  float t = temperature_at(v_world_xz, v_height);
2  float m = moisture_at(v_world_xz);
3
4  float total_weight = 0.0;
5  for (int i = 0; i < biome_layer_count; i++) {
6      // Softness scales with the window's own width and the
7      // layer's own
8      // blend_softness, with a tiny floor so a degenerate window
9      // or a
10     // blend_softness of zero still fades instead of cutting or
11     // dividing by zero
12     float s_t = max(WHITTAKER_TEMP_SOFTNESS_EPSILON,
13         (biome_max_temperature[i] - biome_min_temperature[i]) *
14         biome_blend_softness[i]);
15     float s_m = max(WHITTAKER_MOIST_SOFTNESS_EPSILON,
16         (biome_max_moisture[i] - biome_min_moisture[i]) *
17         biome_blend_softness[i]);
18
19     // Fade in at the lower edge, fade out at the upper one
20     float wt = smoothstep(biome_min_temperature[i] - s_t,
21         biome_min_temperature[i], t)
22         * (1.0 - smoothstep(biome_max_temperature[i],
23         biome_max_temperature[i] + s_t, t));
24     float wm = smoothstep(biome_min_moisture[i] - s_m,
25         biome_min_moisture[i], m)
26         * (1.0 - smoothstep(biome_max_moisture[i],
27         biome_max_moisture[i] + s_m, m));
28
29     // The floor guarantees total_weight > 0 even outside every
30     // window
31     weights[i] = max(wt * wm, WHITTAKER_WEIGHT_FLOOR);
32     total_weight += weights[i];
33 }

```

Figure 5.16: Per-fragment Whittaker weights: four `smoothstep` edges per biome layer.

`WHITTAKER_WEIGHT_FLOOR` is not cosmetic. Nothing forces the user to author a set of rectangles that tiles Whittaker space, and a coordinate that falls outside every configured window would otherwise produce a total weight of exactly zero and a division by zero in the normalization that follows. The floor guarantees that such a fragment degrades into an even mixture of all configured layers instead of rendering as a NaN, which in practice appears as a black or white hole in the terrain.

The weights are then normalized by their sum, so the blend is a partition of unity regardless of how many windows overlap at that point. Overlap is the intended authoring model rather than an error: two layers whose rectangles share an edge

produce a smooth ecotone across it, and layers deliberately given wide, overlapping windows blend continuously everywhere.

The TerrainBiomeLayer Resource

A biome is described by a `TerrainBiomeLayer` resource, which bundles five texture channels, the four classification thresholds discussed above, a `uv_scale` controlling the tiling density of that particular material, a `pom_depth` used in [Section 5.5 → p.103](#), and a purely descriptive `biome_name` that gives the array entries readable labels in the inspector and gives gameplay code something to branch on. The five channels, albedo, normal, roughness, height and [Ambient Occlusion \(AO\)](#), are the standard [Physically Based Rendering \(PBR\)](#) channel set, which means any material from a public texture library can be dropped in unmodified; Poly Haven assets [\[64\]](#) served this role throughout development, while the shipped demo scene uses Quixel Megascan textures [\[65\]](#).

`TerrainConfiguration` gains a `TypedArray<TerrainBiomeLayer>` of these, plus a single optional `TerrainSlopeLayer`, replacing the old `albedo_texture` property. The array is capped at `MAX_BIOME_LAYERS = 32`, and `set_biome_layers()` truncates anything longer with a warning rather than silently accepting it. That ceiling is not arbitrary: [OpenGL Shading Language \(GLSL\)](#) uniform arrays must be declared with a compile-time constant size, so `biome_min_temperature[32]` and its five siblings in the shader fix the limit, and the C++ constant exists to keep the CPU side from ever handing the shader more layers than it has slots for.

Both new resources need every texture assignment to propagate a change notification up to `Terrain3D`, and both need to reject textures large enough to make the `texture array` of [Section 5.5 → p.100](#) unaffordable. Rather than repeating that logic ten times across the two classes, it is extracted into a single free function in `terrain_texture_slot.h`:

```

1  inline constexpr int32_t TEXTURE_SLOT_MAX_DIMENSION = 2048;
2
3  inline void set_texture_slot(Object *p_owner, Ref<Texture2D>
4      &r_slot,
5      const Ref<Texture2D> &p_texture, const char
6      *p_debug_kind,
7      const char *p_class_name) {
8      if (p_texture.is_valid() && (p_texture->get_width() >
9          TEXTURE_SLOT_MAX_DIMENSION
10             || p_texture->get_height() >
11             TEXTURE_SLOT_MAX_DIMENSION)) {
12          // Reject and keep the old value, rather than accepting
13          // a texture the
14          // array builder would have to reject later
15          UtilityFunctions::push_warning(/* ... size cap message
16          ... */);
17          return;
18      }
19
20      if (r_slot != p_texture) {
21          if (r_slot.is_valid()) {
22              r_slot->disconnect("changed", Callable(p_owner,
23                  "emit_changed"));
24          }
25          r_slot = p_texture;
26          if (r_slot.is_valid()) {
27              r_slot->connect("changed", Callable(p_owner,
28                  "emit_changed"));
29          }
30          p_owner->call("emit_changed");
31      }
32  }

```

Figure 5.17: Shared texture-slot setter used by `TerrainBiomeLayer` and `TerrainSlopeLayer`.

The `connect/disconnect` pair builds a chain of `changed` signals: editing a texture notifies its layer, the layer notifies `TerrainConfiguration`, and the configuration notifies `Terrain3D`, which is what makes editing a biome in the inspector update the viewport immediately. `set_biome_layers()` performs the same rewiring at array level, disconnecting from every outgoing layer before connecting to every incoming one, so that a layer removed from the array stops driving rebuilds.

The 2048×2048 cap is a memory decision. A single `PBR` channel at that resolution is 16 MiB uncompressed, five channels per layer is 80 MiB, and at the 32-layer ceiling the arrays alone would reach roughly 2.5 GiB before mipmaps, over 3 GiB with them. Rejecting the texture at assignment time, in the inspector, where the

user can see the warning and act on it, is considerably friendlier than allocating it and failing later.

Uploading Layers as Texture Arrays

The fragment shader has to sample layer i inside a loop whose bound is a uniform, and that rules out an array of `sampler2D` uniforms: the GLSL version Godot targets cannot index a sampler array by a value it does not know at compile time. A [texture array](#) [66] can. It is a single sampler holding N equally sized slices, picked by a third texture coordinate, so the loop index selects the layer and the whole set costs one bind. `TerrainRenderer` builds five of them, one per PBR channel, and uploads them through `RenderingServer::texture_2d_layered_create` [67].

Every slice of a texture array must share dimensions and format. User-authored layers will not, so `_build_biome_texture_arrays()` normalizes them: for each channel it takes the first configured texture's size as the reference, then pushes every layer through `_prepare_layer_image()`, which decompresses, converts to RGBA8 and regenerates mipmaps.

Two details in that function are worth calling out. First, it duplicates the image before touching it: `Texture2D::get_image()` hands back the imported resource's own image, and resizing or converting that in place would corrupt the asset everywhere else in the project. Second, the placeholders for a channel with no texture assigned. Albedo falls back to mid-grey, normal to a flat (0.5, 0.5, 1.0), roughness to 0.5 and AO to white, the neutral value for each. Height falls back to *black* instead: the raymarch of [Section 5.5](#) → p.103 reads a height map as depth below the surface, so mid-grey would sink a layer without one half a depth range underground and make it step down visibly against its neighbours; black reads as flat.

Mipmaps have to be generated explicitly, because `texture_2d_layered_create` uploads the images exactly as given. Without them a clipmap terrain aliases badly: the outer rings compress a 2048^2 texture into a handful of pixels, and every frame of camera motion resamples a different set of texels.

Dimension mismatches are handled, but not silently. A layer whose texture does not match the reference size is resized with Lanczos filtering, and a warning names the offending layer and channel. If any mismatch occurred at all, the whole build is abandoned and zero layers are reported to the shader. That deliberately trips the zero-layer path, which renders the terrain flat magenta: a resized-but-blended terrain only looks slightly wrong, whereas magenta is unmistakably a configuration error and sends the user straight to the warning in the output panel.

Finally, rebuilding these arrays is expensive, since it decompresses, converts and mipmaps every texture of every layer, and `rebuild_mesh()` runs on every `changed` signal. This is the same trap as in [Section 5.4](#) → p.92, solved the same way. A *signature*, the instance IDs of all five textures of all layers, is compared against

the previous one, and the arrays are rebuilt only when it differs. A separate `_biome_texture_arrays_built` flag is needed alongside it: an empty signature cannot distinguish “never built” from “built, with zero layers”, and without it a zero-layer configuration would rebuild on every edit.

Triplanar Projection

The clipmap mesh has no meaningful [UV](#) coordinates. It is a flat grid whose vertices are displaced in the vertex shader ([Figure 5.8](#) → p. 84), so the only parameterization available to the fragment shader is the world position itself, and the obvious choice, mapping $UV = (x, z)$, is exactly what the placeholder material did. That projection is correct for flat ground and degenerate for steep ground: as a face approaches vertical, its projected footprint on the XZ plane approaches zero, and the texture is smeared into infinite vertical streaks. The demo configurations authored for this phase use `height_scale` values large enough for near-vertical faces to be common, which makes this failure the dominant visual artifact.

The standard remedy is [triplanar mapping](#) [\[68\]](#): sample the material three times, once projected on each world axis plane, and blend the three results by how much the surface normal faces each axis. Weights are $|\vec{n}|$ raised to a user-controlled `triplanar_sharpness` exponent and normalized, so a higher exponent narrows the transition band between projections.

```

1  vec2 axis_uv(vec3 p_v, int p_axis) {
2      return p_axis == 0 ? p_v.zy : (p_axis == 1 ? p_v.xz :
3          p_v.xy);
4  }
5
6  vec3 triplanar_weights(vec3 p_normal) {
7      vec3 w = pow(abs(p_normal), vec3(triplanar_sharpness));
8      return w / max(w.x + w.y + w.z, TRIPLANAR_WEIGHT_SUM_MIN);
9  }
10
11 vec4 triplanar_array(sampler2DArray p_tex, int p_layer, vec3
12     p_pos,
13     vec3 p_ddx, vec3 p_ddy, float p_scale, vec3 p_w) {
14     vec4 acc = vec4(0.0);
15     for (int a = 0; a < 3; a++) {
16         float w = axis_weight(p_w, a);
17         // Skip planes that contribute nothing: flat ground pays
18         for one
19         if (w > TRIPLANAR_WEIGHT_EPSILON) {
20             acc += w * textureGrad(p_tex,
21                 vec3(axis_uv(p_pos, a) * p_scale,
22                     float(p_layer)),
23                 axis_uv(p_ddx, a) * p_scale, axis_uv(p_ddy, a) *
24                 p_scale);
25         }
26     }
27     return acc;
28 }

```

Figure 5.18: Triplanar weighting and sampling, with per-axis early-out and explicit derivatives.

Triplanar projection triples the texture fetches per channel. That is why the weight test sits inside the loop rather than blending at the end: on flat terrain two of the three weights are effectively zero, so only one plane is sampled and the cost falls back to that of a planar projection. The three-plane cost is paid only on the cliffs that need it.

Two further consequences are less obvious. The first is why `textureGrad` appears instead of a plain `texture` call. The hardware normally picks the [mipmap](#) level by comparing the texture coordinate across neighbouring fragments. That comparison is meaningless here: each plane takes a different pair of world coordinates, and once the raymarch of [Section 5.5](#)^{→ p.103} is layered on, neighbouring fragments can terminate at different depths, which shows up as visible mip-level seams. The shader therefore computes the derivatives of the world position once, with `dFdx/dFdy`, and takes the same pair from them as from the coordinates, so each plane gets the gradient that matches its own projection.

The second is normal mapping. Each normal map is stored relative to the tangent frame of its own projection, and the three projections have three different frames, so averaging the sampled vectors gives a normal that is wrong for all three. The shader uses *whiteout* blending [69] instead, which reorients each sampled normal against the geometric normal, componentwise and in that plane's own component order, before accumulating it.

The blended result has no single tangent frame either, so the `TANGENT` and `BINORMAL` outputs from [Section 5.3 → p.83] no longer mean anything. They are removed, and `fragment()` writes `NORMAL` in view space directly rather than routing a tangent-space normal through Godot's `NORMAL_MAP` slot. The finite-difference tangent and binormal of [Figure 5.8 → p.84] are deleted outright; both were built from the same four neighbouring height samples the normal still needs, so this changes nothing about the geometry and saves two normalizations per vertex.

Parallax Occlusion Mapping over a Blended Height Field

Normal maps fake surface detail through shading alone, so a pebbled riverbed lights convincingly but stays visibly flat once the camera looks along it at a shallow angle, and never casts detail over itself. `POM` [70] fixes that, and is the occlusion-aware successor to Kaneko's original parallax mapping [71]. It marches a ray along the view direction through the height map, then moves the texture lookup to the first point where the ray drops below the surface.

Two things make this harder than the textbook case. Textbook `POM` marches in tangent space, and [Section 5.5 → p.101] has just established that there is no tangent frame here. It also marches through a single height texture, whereas this height field is whatever the Whittaker blend of [Section 5.5 → p.96] produces at that fragment: several biome height maps mixed together, plus the rock overlay of [Section 5.5 → p.106].

The first is solved by marching in world space instead. The view vector is converted to world space and its component along the geometric normal removed, which leaves a direction lying in the surface plane. Dividing that by $|\vec{n} \cdot \vec{v}|$ lengthens the sweep as the view flattens out, exactly as the tangent-space formulation does. The divisor is clamped, since it approaches zero when the view is almost parallel to the surface and the sweep would otherwise run away.

The second is solved by `blended_height()`, which rebuilds the same mix at any point along the ray. Doing that in full at every step would be far too expensive, so the raymarch is deliberately given a cheaper version than the shading pass: only the two highest-weighted layers take part, picked out by the same loop that normalizes the weights and mixed by their relative proportion. Shading still blends every layer above the weight epsilon; only the height field the ray travels through is simplified. The two dominant layers carry almost all of the weight almost everywhere, so the silhouette this produces is indistinguishable from the exact one, at a fixed two texture fetches per step instead of up to thirty-two.

```

1 // Sweep lies in the surface plane and lengthens at shallow
  angles,
2 // where the divisor is clamped to keep it from diverging
3 vec3 view_ws = mat3(INV_VIEW_MATRIX) * normalize(-VERTEX);
4 float n_dot_v = dot(view_ws, geo_normal_ws);
5 vec3 sweep = (view_ws - geo_normal_ws * n_dot_v)
6   / max(abs(n_dot_v), POM_GRAZING_CLAMP) * pom_depth *
  pom_fade;
7
8 // Spend more steps at shallow angles; the distance fade scales
  both
9 // the step budget and the depth continuously down to zero
10 float step_count = max(1.0, floor(pom_fade *
11   mix(float(pom_max_steps), float(pom_min_steps),
12     abs(n_dot_v))));
13 float layer_step = 1.0 / step_count;
14
15 // Linear search: walk down until the ray drops below the height
  field
16 for (int s = 0; s <= int(step_count); s++) {
17   cur_surface = 1.0 - blended_height(base_pos - ray_depth *
18     sweep, ...);
19   if (ray_depth >= cur_surface) { hit = true; break; }
20
21   prev_surface = cur_surface;
22   prev_ray_depth = ray_depth;
23   ray_depth += layer_step;
24 }

```

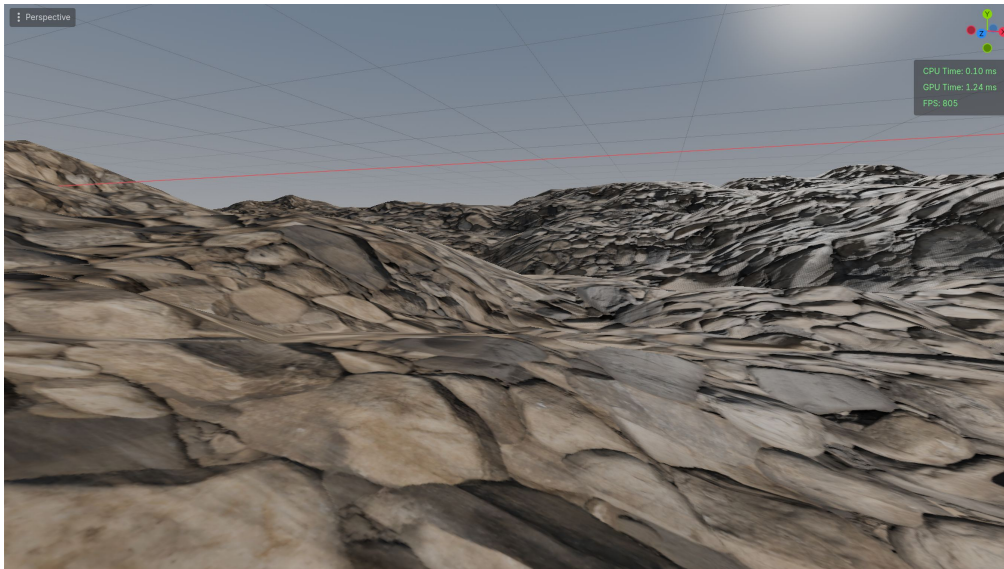
Figure 5.19: World-space parallax sweep and adaptive linear search; the refinement that follows is described below.

The step count adapts in two independent ways. It interpolates between `pom_max_steps` and `pom_min_steps` by $|\vec{n} \cdot \vec{v}|$, spending the budget where the sweep is longest and the artifacts worst. It is also scaled by a distance fade set by `pom_fade_start` and `pom_fade_end`. That fade multiplies the depth as well as the step count, so the effect does not switch off abruptly at some distance but tapers to no displacement and no cost, which avoids a visible ring of popping in the terrain. The outer clipmap levels cover most of the screen and pay almost nothing for it.

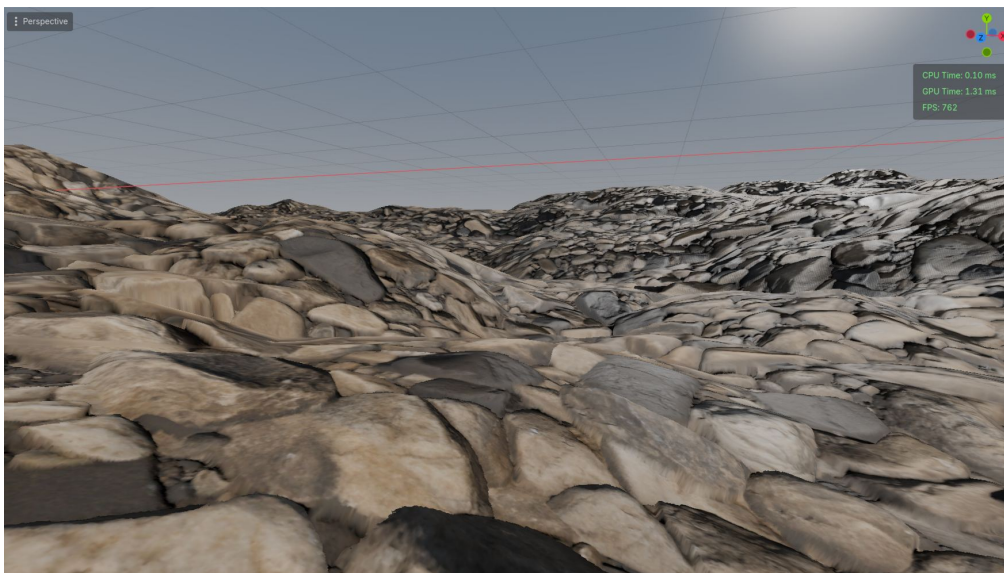
Once the linear search brackets a hit, the intersection is refined rather than accepted as it stands. Three bisection steps narrow the bracket, and a final secant step solves for where the ray and the height field actually cross inside it. The usual single interpolation between the last two samples is cheaper, but it assumes the height field runs straight across a whole coarse step. That is exactly the assumption that fails on the fine detail [POM](#) exists to show, and it leaves stair-stepped silhouettes at low step counts. Bisecting first shrinks the interval

until the assumption holds.

[Figure 5.20](#) → p. 106 shows both techniques disabled and enabled from the same viewpoint in the demo scene. With them off, the rock reads as a texture painted onto the surface; with them on, individual stones gain depth and the gaps between them darken into recesses rather than staying flat lines. The profiler overlay in each screenshot gives the cost: GPU time rises from 1.24 ms to 1.31 ms, roughly 6 %, while CPU time is unchanged at 0.10 ms, as expected for an effect that lives entirely in the fragment shader. That the penalty stays this small on a view dominated by rock, where the parallax is doing the most work, is the combined result of the early-outs and fades collected in [Section 5.5](#) → p. 108.



(a) Triplanar projection and POM disabled.



(b) Both enabled.

Figure 5.20: Triplanar projection and parallax occlusion mapping, disabled (a) and enabled (b), with the profiler overlay reporting CPU time, GPU time and frame rate for each.

Slope-Aware Rock Overlay

Temperature and moisture describe climate, and climate describes what grows on the ground, not what the ground is made of. A cliff face is bare rock in every biome, because nothing settles on it. No combination of Whittaker thresholds can express that: slope is not one of the two axes.

Adding slope as a third classification axis would multiply the number of layers a user has to author, and it still would not capture the “regardless of climate” part. Steepness is handled by a `TerrainSlopeLayer` composited *on top of* the

finished biome blend instead. Its coverage is a `smoothstep` of how far the surface tilts away from flat, $1 - (\vec{n} \cdot \vec{up})$, around a user-set `slope_threshold` with a `slope_blend_range` transition band, and the resulting alpha mixes all four shaded channels toward the rock material.

A single rock look for the whole terrain, however, is itself a climate assumption: bare granite reads as wrong on a cliff inside a snow biome, which should expose frost-crusting rock instead of the same warm-toned stone as a desert canyon. This limitation was only addressed in a later revision, after the rest of this phase and the editor and validation work in [Section 5.6 → p.111](#) and [Section 5.7 → p.113](#) were already in place: rather than adding a second, climate-keyed array of slope layers alongside the biome one, each `TerrainBiomeLayer` can now carry its own optional `slope_layer`, `slope_override`, `slope_threshold` and `slope_blend_range`, falling back to `TerrainConfiguration`'s single `rock_layer` whenever a biome does not set its own. `TerrainRenderer` deduplicates the resulting set of distinct rock materials, the shared fallback plus whichever biomes override it, into one `texture array`, the same technique [Section 5.5 → p.100](#) already used for the five biome channels, and looks each biome's slice up by index instead of allocating one texture slot per biome. The fragment shader reuses the same `pom_layers` pair the biome blend already resolved to pick a fragment's rock slice, and blends between the two dominant biomes' rock materials across an ecotone exactly as it blends their grass or dirt, so a cliff straddling a biome boundary does not snap between two unrelated rock looks.

Two details make this more than a final mix. First, the rock's alpha is folded into `pom_depth` *before* the raymarch, and the rock's height map takes part in `blended_height()`. A cliff is therefore displaced by the rock's relief rather than by the grass's, which would be wrong exactly where parallax shows up most. Second, the overlay is applied after the zero-biome-layer fallback too, so a terrain with a rock layer but no biomes still renders its cliffs correctly over the magenta debug surface. That keeps the failure diagnosable rather than total.

CPU-Side Biome Queries

[Section 5.4 → p.91](#) established `Terrain3D::get_height_at()` as the gameplay-facing counterpart to the collision shape. Biomes need the same treatment, for the same reason: spawn tables, footstep sounds, ambient audio and vegetation placement all need to know which biome a coordinate belongs to, and none of them can read that back from a shader. `Terrain3D::get_biome_at(Vector2)` forwards through `TerrainPhysics` to a CPU version of [Figure 5.16 → p.97](#). It returns the `Ref<TerrainBiomeLayer>` itself, so callers can read `biome_name`, or any other authored property, straight off the resource.

The two implementations agree on the weights, sharing the two softness constants verbatim, but differ deliberately in what they do with them. The shader normalizes and blends every layer; the CPU takes the single highest-weighted one. A renderer

wants a continuous mixture, because that is what makes an ecotone look like a gradient rather than a border. Gameplay wants one answer, because “spawn 60 % of a pine and 40 % of a cactus” is not a usable instruction. So the two ask different questions of the same fields, and agree on the only thing that has to match: which layer dominates at a given point.

Challenges Encountered & Solutions

The dominant challenge of this phase was cost. A naive reading of the requirements, blend every biome layer, project each one triplanarly, sample five channels for each, and raymarch the blended height field, multiplies out to $32 \times 3 \times 5$ texture fetches per fragment for shading alone, before the raymarch repeats a subset of that work at every step. On a clipmap that fills the screen this is not a shader that runs at interactive rates. The mitigations are individually small and were applied in several passes over the shader:

- **Weight-based early-out.** Layers below `BIOME_WEIGHT_EPSILON` are skipped entirely rather than multiplied by a near-zero weight, and the accumulated weight of the layers actually sampled is used to renormalize the result, so skipping does not darken the surface. Because the Whittaker windows are narrow relative to the full climate range, the number of layers actually sampled at any fragment is typically two or three regardless of how many are configured.
- **Axis-based early-out.** As described in [Section 5.5 \$\rightarrow\$ p.101](#), projections whose weight is negligible are not sampled, which returns flat ground to single-projection cost.
- **A cheaper height field than shading field.** The raymarch traverses only the two dominant layers ([Section 5.5 \$\rightarrow\$ p.103](#)), bounding its per-step cost independently of the layer count.
- **Distance fade.** Both the parallax depth and the step count fade to zero over a configurable range, so the outer clipmap levels, which occupy most of the framebuffer, cost nothing beyond the shading blend.
- **Explicit mip selection.** Passing per-plane derivatives to `textureGrad` is not only a correctness fix; it also ensures distant fragments read from small mip levels, which is what keeps the texture cache useful under a projection that changes per fragment.

Two further problems surfaced during implementation and are worth recording:

- **The mid-grey height placeholder.** The placeholder colours were initially uniform, mid-grey for every unassigned channel, following the reasoning that grey is the neutral value. For height it is not: the raymarch reads the height map as depth below the reference surface, so layers with no height map assigned sat half a depth range lower than their neighbours and produced a

visible step wherever such a layer blended against one that had a real height map. Black is the correct neutral for that channel, and the fallback was changed accordingly.

- **The vestigial tangent frame.** The vertex shader had computed TANGENT and BINORMAL by finite differences since [Section 5.3](#) → p. 82. Once triplanar whiteout blending replaced the NORMAL_MAP path, that frame was not merely unused but actively misleading, since it described a projection the fragment shader no longer used. Removing it costs nothing: it shared the four height samples the normal is still reconstructed from.

Result

By the end of this phase the terrain is textured by what it is rather than by a single global material. Temperature and moisture fields, derived from the same noise implementation as the elevation and evaluated identically on both the CPU and the GPU, place every point in Whittaker space; each configured biome layer contributes to the surface in proportion to how well that point falls inside its climate window, so transitions between biomes are gradients rather than seams. The materials themselves are projected triplanarly, which removes the vertical smearing that the previous world-space planar mapping produced on every cliff, and given genuine relief by a parallax raymarch over the blended height field. Steep faces are overlaid with exposed rock independently of climate, and gameplay code can ask which biome any coordinate belongs to through `get_biome_at()` without touching the renderer. The cost of all of this is bounded by a set of early-outs and fades that make the expensive path apply only to the fragments that visibly need it.

[Figure 5.21](#) → p. 110 shows the demo scene at the end of the phase. Grass, dirt and rock each hold the ground where their climate window puts them, and the boundaries between them are gradients: no seam anywhere in the frame follows a straight line or a noise contour. The rock overlay is visible on the ridge in the background, exposed on the steep faces regardless of which biome surrounds them.

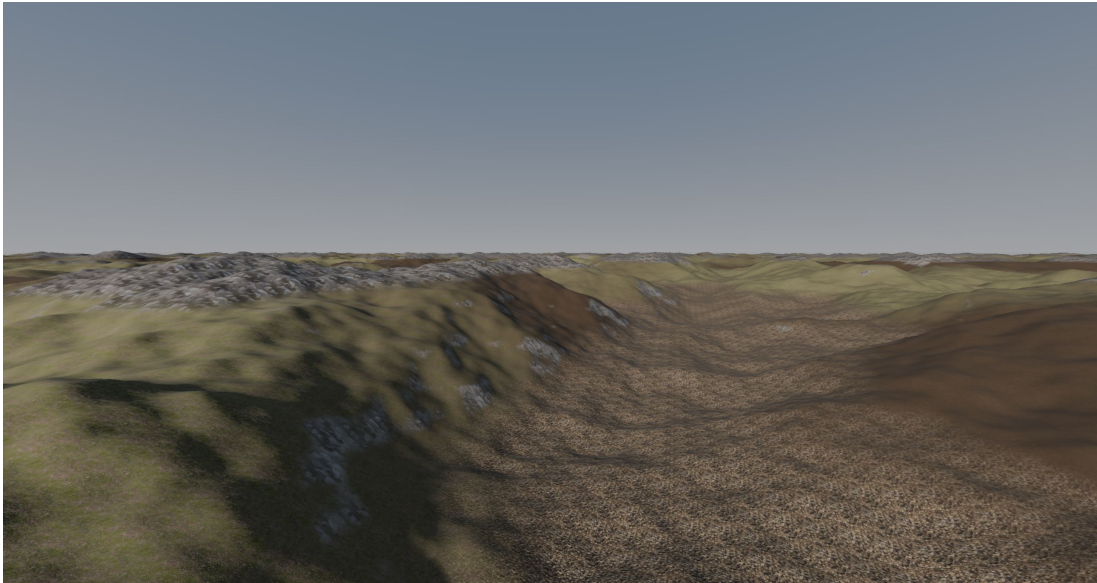


Figure 5.21: The demo scene textured by biome, with the slope-aware rock overlay on the ridge.

Seen from directly above (Figure 5.22 $\rightarrow$ p. 110), the same scene shows the distribution the Whittaker classification produces: organic patches with soft edges, rather than the cells a Voronoi assignment would give (Section 2.5 $\rightarrow$ p. 53) or the hard boundaries a lookup table would. Every patch border is an *ecotone* several metres wide.

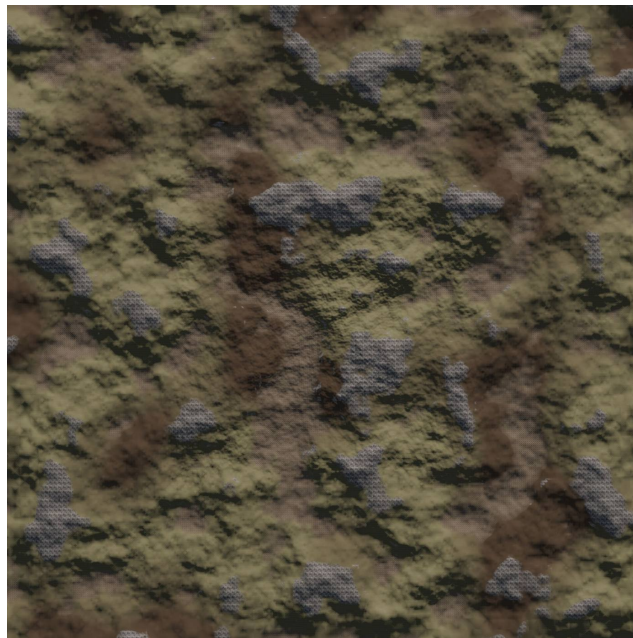


Figure 5.22: Top-down view of the biome distribution produced by the temperature and moisture fields.

Figure 5.23 $\rightarrow$ p. 111 closes in on one of those borders. Grass gives way to loose pebbles across a band rather than at a line, with both materials keeping their

own parallax relief through the transition, since [Section 5.5](#) → p. 103 raymarches the blended height field rather than either layer's alone.



Figure 5.23: Close-up of a biome transition: grass blending into loose pebbles across an ecotone, each keeping its own surface relief.

5.6 Phase 5: Editor Integration & Tooling

With the core terrain features complete, the focus of this phase is now on improving the developer experience. A procedural terrain system is inherently opaque. The goal was to build a suite of editor tools that expose the internal state of the terrain, making it observable and easier to author.

Editor Initialization & Introspection API

To support editor-specific features, the module's initialization level was elevated to `MODULE_INITIALIZATION_LEVEL_EDITOR`, enabling the registration of a `TerrainServerEditorPlugin` and several UI classes.

With the editor plugin in place, the `Terrain3D` node was expanded with a comprehensive introspection API. Methods such as `get_clipmap_level_extent()`, `get_collision_built_range()`, and `get_temperature_at()` were exposed to C++ and GDScript, allowing both editor tools and gameplay scripts to query the exact state of the renderer and physics grid.

Editor Preview & Camera Integration

Previously, the terrain centered itself around a designated focus node or the active scene camera. In the editor, this meant the terrain would either remain static or

follow a node instead of the developer's viewport camera, making editing large landscapes frustrating.

To solve this, an `editor_focus_override` property was introduced. The `Terrain ServerEditorPlugin` intercepts the editor's camera movement and continuously updates this override. Consequently, the terrain clipmap and collision grid dynamically re-center around the editor camera while editing, providing a seamless preview of the infinite terrain. Also, these previews were gated behind toggles (`editor_preview` and `editor_preview_physics`), allowing developers to disable expensive generation passes when working on unrelated parts of a scene. Menu items were also added to force a mesh rebuild or hot-reload the shader on demand.

Gizmos & Visual Debugging

Exposing the internal state through an API is useful, but rendering it directly in the 3D viewport provides immediate context. A modular gizmo system was implemented using `EditorNode3DGizmoPlugin`, registering several specialized gizmos:

- **Clipmap Gizmo:** Draws concentric wireframe rings representing the exact bounds of each [LOD](#) level, verifying that the culling margins and scaling are correct.
- **Collision Gizmo:** Renders the bounding box of the active physics grid, making it immediately obvious when the asynchronous regrid ([Section 5.4 → p. 96](#)) fires and where the collision boundaries lie.
- **Focus Gizmo:** Places a marker at the snapped focus origin.

All of them can be enabled and disabled independently from the native Godot gizmo menu, allowing developers to focus on the information they need without cluttering the viewport.

Terrain Dock & Climate Chart

Finally, a custom `TerrainDock` was added to the Godot interface. This dock acts as a dashboard, polling the introspection API to display live statistics such as the current focus position, temperature, moisture, and the active biome at the camera's location.

To assist in authoring the Whittaker biome parameters ([Section 5.5 → p. 96](#)), a custom `WhittakerChart` Control node was integrated into the dock. This chart visualizes the climate distribution of the biomes, giving artists an intuitive way to understand how the configured temperature and moisture thresholds map to the generated terrain.

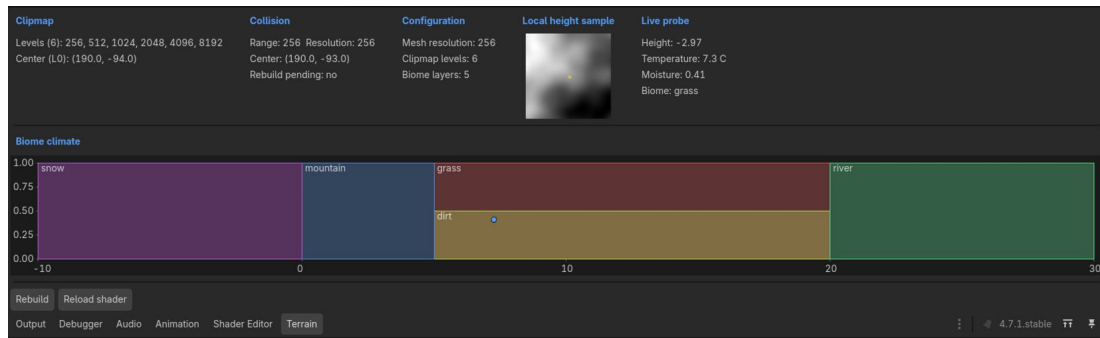


Figure 5.24: The custom Terrain Dock displaying real-time statistics and the Whittaker biome chart.

Result

By the end of this phase, the TerrainServer plugin evolved from a black-box generator into an observable, developer-friendly tool. The integration with the editor camera allows fluid navigation of the infinite terrain, while the custom gizmos and dock provide real-time, visual confirmation of the system's internal state.

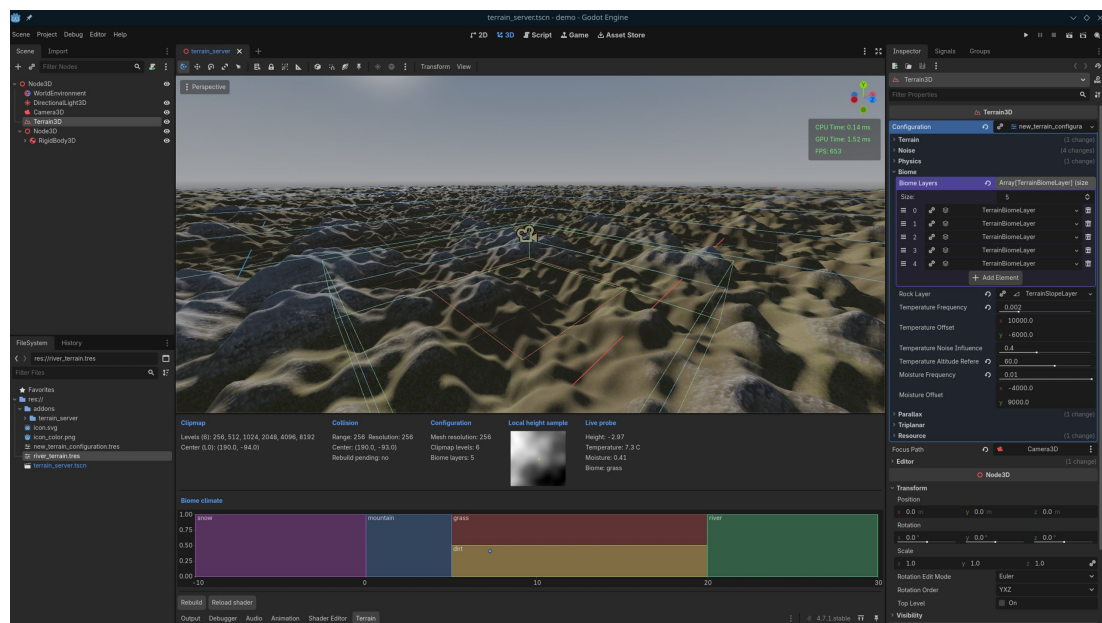


Figure 5.25: Editor viewport showing the terrain with clipmap and collision gizmos enabled.

5.7 Phase 6: Release & Documentation

The final phase of the project focused on preparing the TerrainServer plugin for public release. This involved comprehensive documentation, code cleanup, stress testing, and packaging the module for distribution. The primary objective was to

ensure that the plugin is stable, user-friendly, and accessible to developers looking to integrate procedural terrain generation into their own projects.

Documentation & Code Cleanup

Before releasing the plugin, the codebase was reviewed to ensure consistency, readability, and adherence to Godot's C++ coding guidelines. Inline comments were expanded, and the public API was documented to provide clear instructions on how to use the `Terrain3D` node, configure biome layers, and interact with the introspection methods.

A five-page manual under `docs/manual/` now covers this ground end to end: installing a release build or building from source on every supported platform, a quick start that gets a `Terrain3D` node on screen in four steps, a walkthrough of the demo scene's five-biome and rock-layer setup, the runtime query API (Section 5.4 → p.91 and Section 5.5 → p.107) with a worked spawn-on-surface example, and the editor tooling from Section 5.6 → p.111. Alongside it, the class reference already cited throughout this chapter (`doc_classes/*.xml`) is compiled directly into the plugin binary, so every documented property and method also appears in Godot's own built-in help panel, searchable next to the engine's own classes rather than only in an external document.

Testing & Validation

To ensure robustness, the system was subjected to an automated benchmark suite across two physical machines (a desktop and a laptop, each dual-booted into Linux and Windows), sweeping every major `TerrainConfiguration` axis independently plus a combined worst-case stress scenario, 304 runs in total. These tests were crucial for validating the GPU-based procedural generation and verifying that the asynchronous physics regrid (Section 5.4 → p.98) could maintain acceptable performance in demanding scenarios; none of the 304 runs failed to complete.

This testing phase also helped identify remaining edge cases and collect feedback, ensuring the system behaved predictably regardless of the user's specific setup.

The full methodology, figures, and discussion are given in Chapter 6 → p.119.

Distribution & Future Work

The original goal for the plugin's distribution was to publish it directly on the Godot Asset Library, making it instantly accessible to the community from within the Godot editor. However, due to time constraints and development delays, publishing to the Asset Library was deferred.

Instead, the plugin is currently distributed via GitHub Releases. The CI/CD pipeline established in Section 5.1 → p.76 automatically cross-compiles the plugin and packages it for all supported operating systems, streamlining the distribution

process. Users can download the compiled binaries and the accompanying demo project directly from the repository. Submitting the plugin to the Godot Asset Library remains a priority for future updates.

The repository is public at github.com/mdoradom/TerrainServer, also linked from this thesis' title page, and the installation page of the manual just described covers both paths end to end: downloading a tagged release's precompiled binaries, or building from source with `scons`, on Linux, Windows and macOS.

Result

By the end of this final phase, the TerrainServer plugin reached a stable and usable state. What began as a proof-of-concept for CPU-based terrain generation evolved into a robust, GPU-accelerated infinite terrain system with physics, biome blending, and specialized editor tooling.

The province-gated ridging and per-biome slope override from [Section 5.3 → p. 85](#) and [Section 5.5 → p. 106](#) let the same `TerrainConfiguration` that produced the gentle, dune-like terrain in [Figure 5.21 → p. 110](#) also author a genuine mountain range, without a second height function or a second, hand-placed rock material.

[Figure 5.26 → p. 116](#) looks straight down on a highland province: the dendritic ridgelines the folded octaves produce read as a mountain range rather than as noise, and it is the snow biome's own frost-crusting rock, not the demo's default granite, that shows through on every exposed slope.



Figure 5.26: Top-down view of a highland province: ridged, snow-capped terrain with the snow biome’s own rock exposed on its slopes.

Figure 5.27^{→ p. 117} is a ground-level view from inside that same range. Peaks stay sharp near the camera without the mesh-scale creasing an unrounded fold would leave, since `ridge_crest_rounding` band-limits it (Section 5.3^{→ p. 85}).

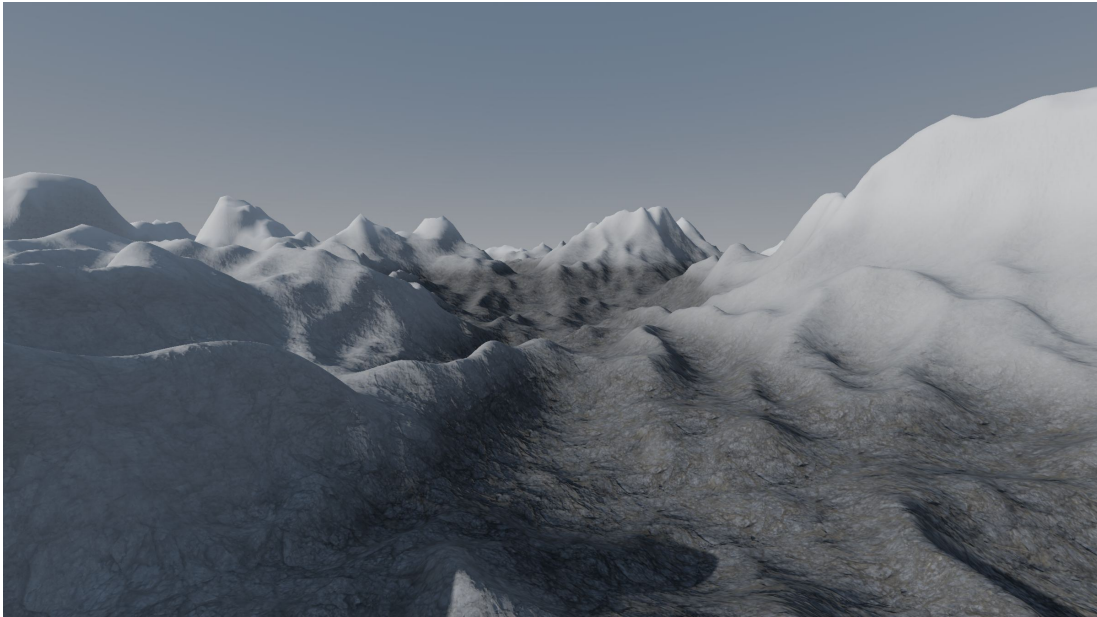


Figure 5.27: Ground-level view inside the mountain range: sharp ridgelines without mesh-scale creasing.

Pulled back to the surrounding lowland, [Figure 5.28 → p. 117](#), [Figure 5.29 → p. 118](#) and [Figure 5.30 → p. 118](#) approach the same range from across the plain and show what the province mask is actually for: the terrain stays calm, grassy [fBm](#) right up to the province boundary, and only turns ridged and snow-capped past it, exactly the “mountains here, plains there” split [Section 5.3 → p. 85](#) set out to make possible.

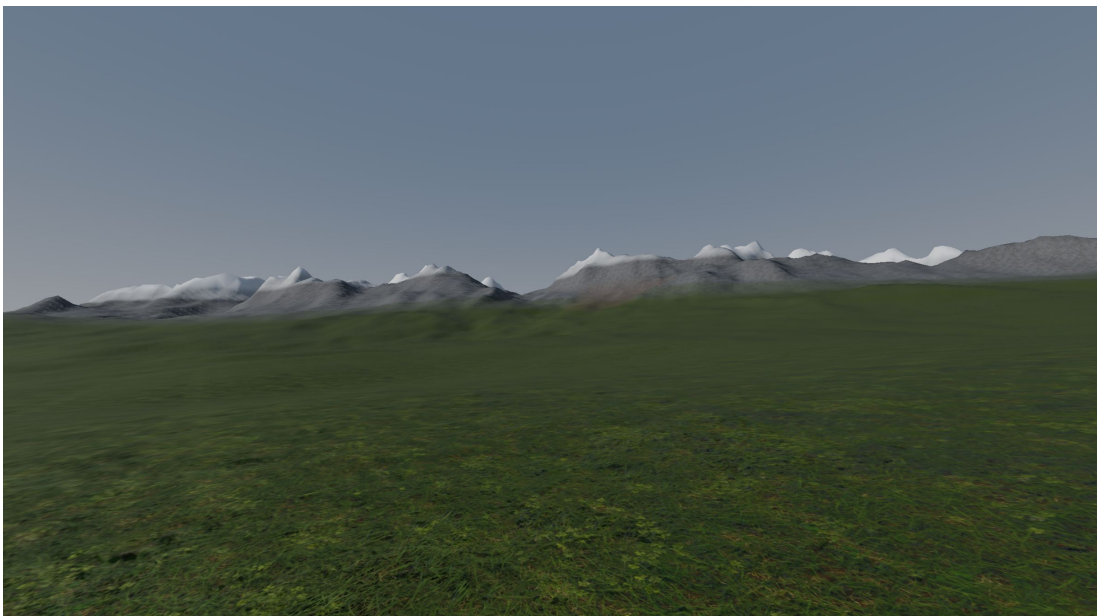


Figure 5.28: The mountain province seen from across the lowland: flat, grassy [fBm](#) in the foreground, ridged terrain on the horizon.

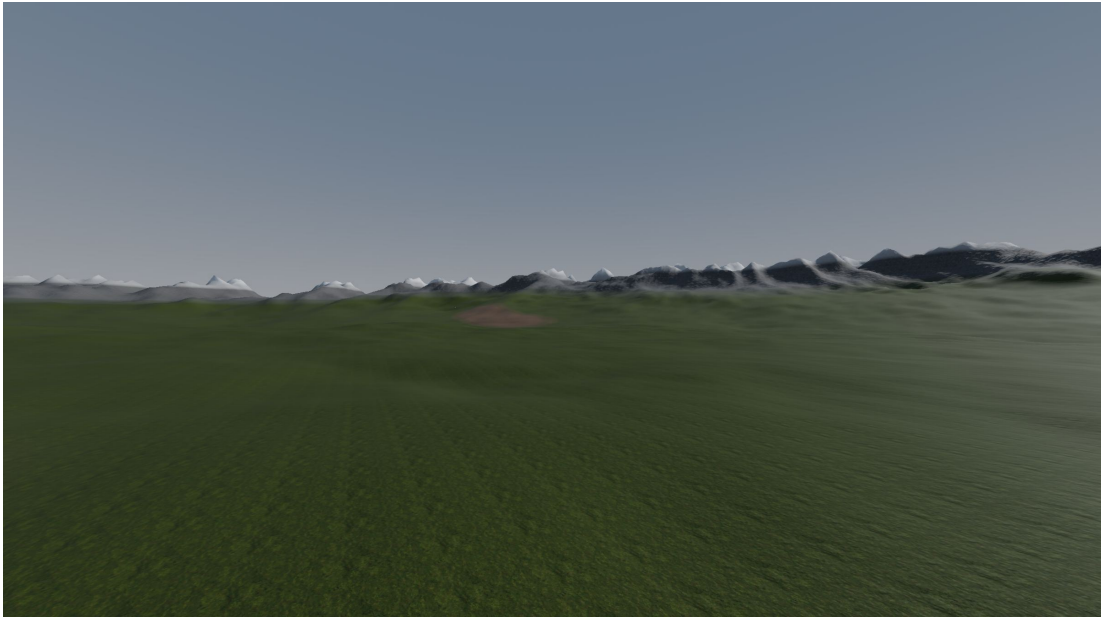


Figure 5.29: Approaching the province boundary: the transition band set by `noise_continent_contrast`.

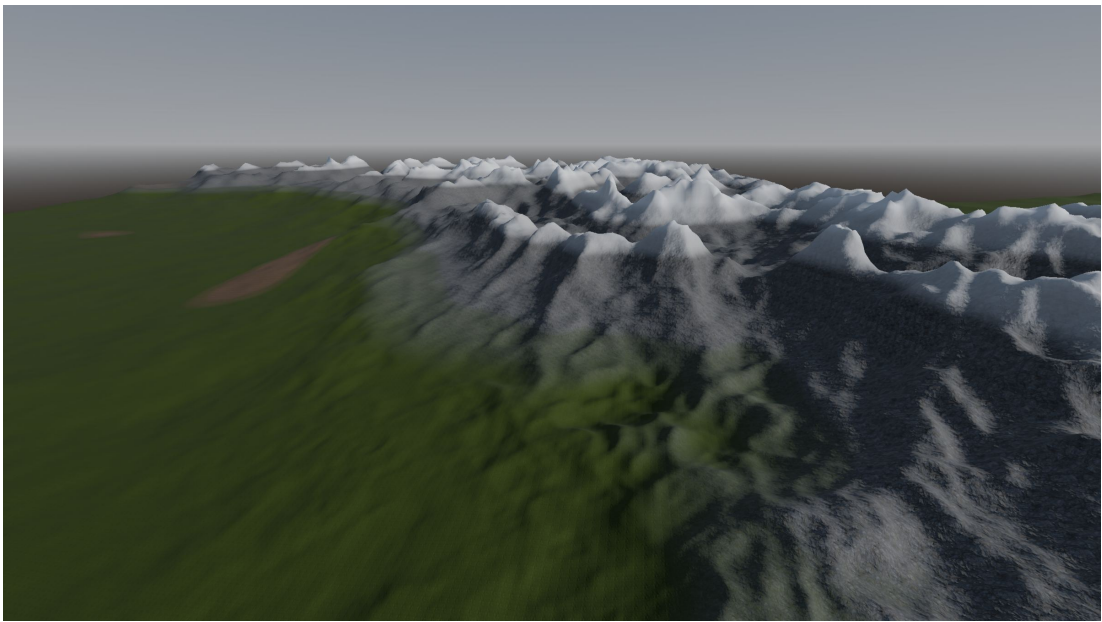


Figure 5.30: Crossing into the highland province: grass gives way to the ridged, rock-covered slopes beyond it.

Project Validation

The preceding chapter described the design decisions behind TerrainServer’s clipmap-based rendering (Section 5.2 → p.78), its GPU-based procedural generation (Section 5.3 → p.82), its asynchronous physics regrid pipeline (Section 5.4 → p.90), and its parallax-mapped biome shading (Section 5.5 → p.103). Each of those sections justified its design qualitatively. This chapter validates them quantitatively: it measures how the plugin actually performs across hardware and operating systems, and checks that the performance characteristics assumed during development hold up under measurement.

6.1 Methodology

Performance was measured with a dedicated benchmark harness shipped alongside the plugin’s demo project, rather than the demo scene itself, so that later edits to the demo’s props or camera could never silently move the numbers. Each measurement point runs as its own headless Godot process: it loads the shipped terrain configuration, applies a single named override (for example `clipmap_levels_8`, which sets `clipmap_levels` to 8 and leaves everything else at its default), and drives a camera along a fixed, frame-indexed path, 2 m per frame, 20 m above the surface. Indexing the camera by frame number rather than elapsed time means every device walks the identical path and triggers the identical sequence of collision rebuilds regardless of how fast it renders, which is what makes the per-device rows comparable at all.

Each run discards a 90-frame warm-up window, which also absorbs shader compilation, and then samples 240 frames of GPU and CPU render time through Godot’s own `RenderingServer` viewport timers. Collision-rebuild latency is timed separately, polled once per frame from `TerrainPhysics::is_collision_rebuild_pending()` (Section 5.4 → p.90), so its granularity is one frame: a rebuild that completes within a single frame is not observed at all, and is recorded as no data rather than as zero. A watchdog quits any run stuck past 600 frames and flags it, so a stalled run fails visibly instead of hanging silently. Every scenario was run twice (interleaved rounds), at 1920×1080 with vsync off, on the Vulkan backend.

Four device/OS combinations were measured: two physical machines, each dual-booted, so that hardware and operating system vary independently.

Table 6.1: Devices used for benchmarking.

Device	CPU	GPU	OS	Godot
desktop-linux	AMD Ryzen 7 9800X3D	NVIDIA GeForce RTX 5080	Linux	4.7.1
desktop-win	AMD Ryzen 7 9800X3D	NVIDIA GeForce RTX 5080	Windows	4.7.1
laptop-linux	Intel Core i7-13700H	NVIDIA GeForce RTX 4060 Laptop	Linux	4.7.2
laptop-win	Intel Core i7-13700H	NVIDIA GeForce RTX 4060 Laptop	Windows	4.7.1

laptop-linux ran a point release ahead of the other three (Godot 4.7.2 rather than 4.7.1); the gap is small enough not to explain the effects reported below, but is noted for completeness.

Six scenario families were swept, each varying one `TerrainConfiguration` property away from the shared baseline (`clipmap_levels = 6`, `mesh_resolution = 256`, `noise_octaves = 6`, `physics_range = 256`, POM steps = 8/16).

The swept axes were:

- `clipmap_levels`, from 1 to 10;
- `mesh_resolution`, from 32 to 512;
- `noise_octaves`, from 1 to 10;
- `physics_range`, from 8 to 512;
- the POM step budget, from off through 64/128;

plus a `stress` scenario that pushes every sweepable axis to the top of its declared range simultaneously. Together with `baseline` itself, this is 38 scenarios $\times$ 2 rounds $\times$ 4 devices = 304 runs. Not one of them triggered the watchdog: every run completed within its 600-frame budget on every device, which is itself a basic robustness result before any timing is even considered.

6.2 Cross-Platform Baseline

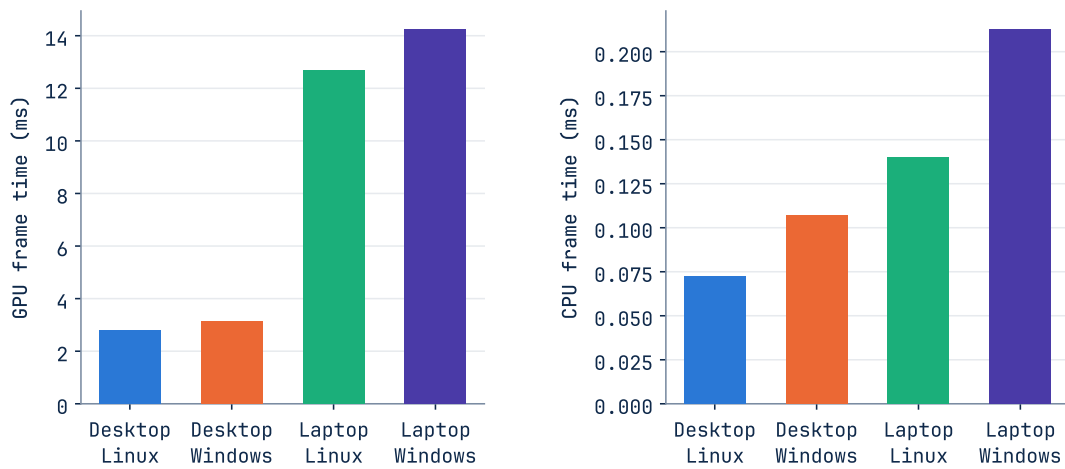


Figure 6.2: GPU and CPU frame time at the baseline configuration, median of 240 sampled frames, across the four measured device/OS combinations.

Figure 6.2 [→ p. 121](#) isolates the effect of the operating system from the effect of the hardware. On the desktop, Windows costs 3.13 ms of GPU time against Linux’s 2.80 ms at the baseline configuration, an 11.8 % overhead; on the laptop the same comparison is 14.23 ms against 12.69 ms, a 12.1 % overhead. The gap is more pronounced on the CPU side: 0.107 ms versus 0.073 ms on the desktop (+46.6 %) and 0.212 ms versus 0.140 ms on the laptop (+51.4 %). The same hardware, running the same plugin, is consistently and substantially more expensive on Windows than on Linux, on both machines.

6.3 Clipmap Level-of-Detail Scaling

Section 5.2 [→ p. 78](#) introduced the Geometry Clipmap as the mechanism that lets the visible terrain extend arbitrarily far without every ring paying full-resolution cost.

Figure 6.3 [→ p. 122](#) sweeps `clipmap_levels` from 1 to 10 and confirms the shape that design predicts: GPU cost grows with the number of rings, since each added level is another band of geometry to rasterize, but it grows sub-linearly on the desktop, where GPU time only rises from 1.5 ms to 4.1–4.4 ms across the full range.

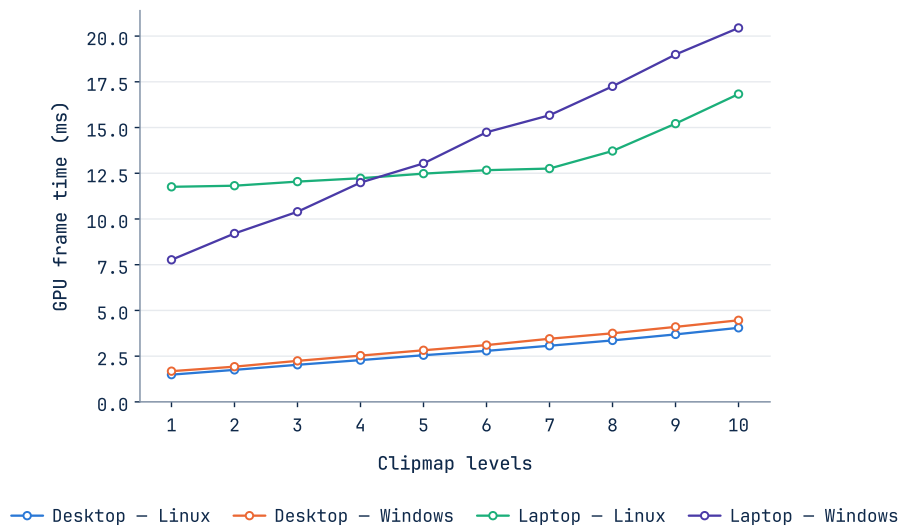


Figure 6.3: GPU frame time against clipmap ring count, median of 240 sampled frames, one line per device.

The laptop tells a more interesting story than the desktop does. `laptop-win` starts *above* `laptop-linux` at every level past 4, but at 1–3 levels it is actually *cheaper*: 7.8ms against `laptop-linux`’s 11.8ms at a single clipmap ring. The two lines cross around level 4–5 and diverge steadily afterward, reaching 20.5ms against 16.8ms at 10 levels. The desktop pair shows no such crossover; Windows sits a small, roughly constant margin above Linux at every level. Whatever is behind the Windows overhead measured in [Section 6.2](#) ^{p.121}, it interacts with clipmap ring count differently on the two machines, and only becomes the dominant cost on the laptop once enough rings are added.

6.4 Mesh Resolution Scaling

`mesh_resolution` controls the vertex density of each clipmap ring, independently of how many rings there are. [Figure 6.4](#) ^{p.123} sweeps it from 32 to 512 vertices per side.

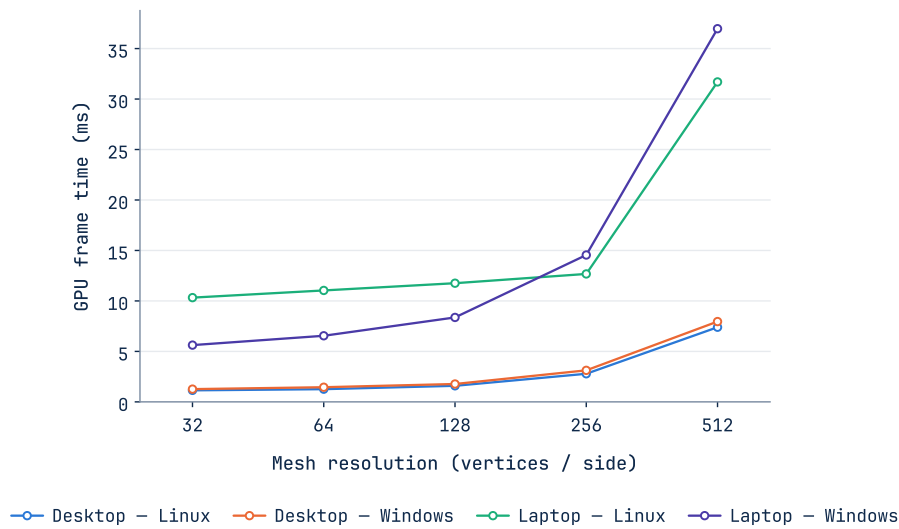


Figure 6.4: GPU frame time against mesh resolution, median of 240 sampled frames, one line per device.

Primitive count grows with the square of `mesh_resolution`, and GPU time tracks that: the curve is close to flat through 128 and then bends sharply upward, roughly doubling between 256 and 512 on every device (desktop: 2.8 → 7.4–8.1 ms; laptop: 12.7–14.6 → 31.7–36.9 ms). This is the expected cost of the vertex-displacement approach from [Section 5.3](#) ^{→ p.82}: every vertex runs the full noise evaluation in the vertex shader, so doubling the grid in each dimension quadruples the work. The default of 256 sits just before that curve steepens, which is consistent with it being chosen as a practical default rather than an arbitrary one.

6.5 Procedural Noise Cost

`noise_octaves` controls how many layers of fractal noise `TerrainNoise` sums per sample. Because the collision heightmap is recomputed by evaluating the same noise function used for rendering ([Section 5.4](#) ^{→ p.88}), its cost also appears in regrid latency, not only in GPU time, and it is regrid latency that shows the clearest signal.

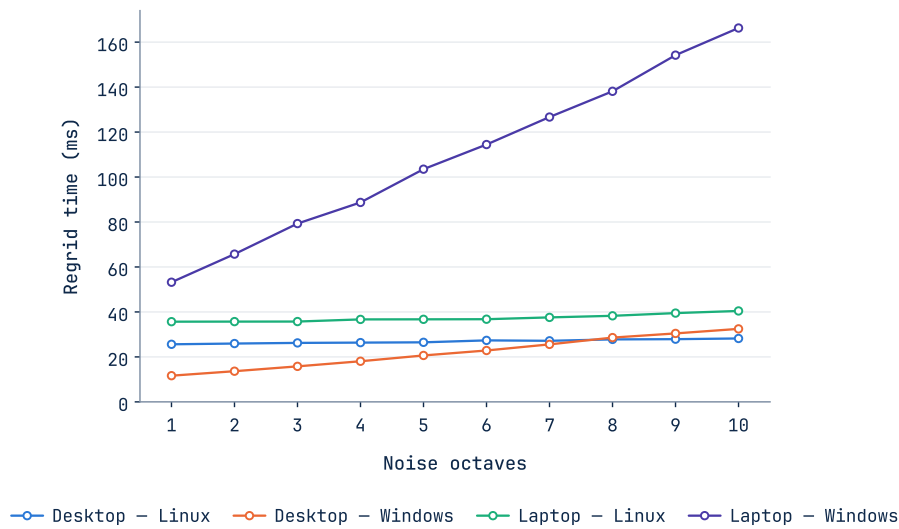


Figure 6.5: Collision regrid latency against noise octave count, mean of the regrids observed per run, one line per device.

On `laptop-linux` and both desktop configurations, regrid time is nearly flat across the full octave range: extra octaves add work, but the grid being recomputed is small enough that the added cost stays under 10–15 ms even at 10 octaves. `laptop-win` is the exception: regrid latency climbs almost linearly from 53 ms at 1 octave to 166 ms at 10, a $3.1\times$ increase, and is already the most expensive line in the chart by 2 octaves. This is the sharpest cross-platform divergence in the whole benchmark suite; whatever makes the `WorkerThreadPool` task behind a regrid (Section 5.4 $\rightarrow$ p. 90) more expensive on Windows, it scales with the per-cell noise cost rather than being a fixed overhead, which a flat gap between the two laptop lines would have suggested instead.

6.6 Physics Range and the Asynchronous Regrid Pipeline

`physics_range` sets the world-space extent of the collision grid maintained by `TerrainPhysics` (Section 5.4 $\rightarrow$ p. 90). Its resolution in cells is

$$\text{grid_resolution} = \text{clamp}\left(\left\lceil \frac{\text{physics_range}}{\text{visual_cell_size}} \right\rceil, 8, 256\right),$$

where `visual_cell_size` is `terrain_size / mesh_resolution`, and regrid cost is $O(\text{grid_resolution}^2)$. At the baseline configuration `visual_cell_size` is exactly 1, so the desired resolution equals `physics_range` directly, which makes the quadratic relationship easy to see in Figure 6.6 $\rightarrow$ p. 125.

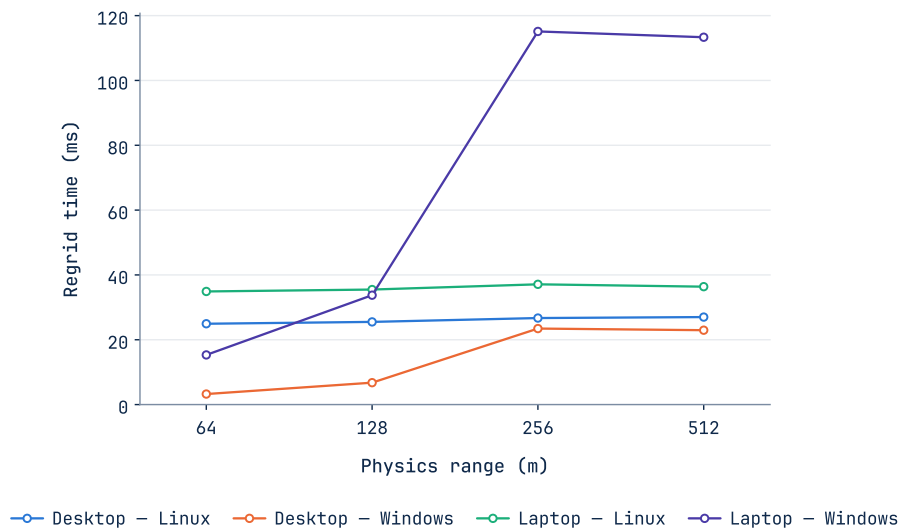


Figure 6.6: Collision regrid latency against physics range, mean of the regrids observed per run, one line per device. `physics_range = 8` is omitted: at that setting the grid clamps to its 8-cell minimum and recomputes within a single frame, so no device recorded a single observable regrid.

Three devices stay close to flat, since a 256-cell grid is still cheap enough to recompute in a few tens of milliseconds regardless of platform. `laptop-win` again stands apart: regrid time jumps from 15 ms at a 64-cell grid to 115 ms at 256 cells, a $7.6\times$ increase over the same range where every other device barely moves, then plateaus between 256 and 512 because both settings hit the same 256-cell clamp. That plateau is itself a small confirmation that the clamp documented in [Section 5.4](#) ^{→ p. 90} behaves as designed: pushing `physics_range` past 256 does not keep increasing the collision cost. The steep rise below the clamp, and how much steeper it is specifically on Windows, is the more consequential finding: a project that raises `physics_range` for gameplay reasons should expect that cost to fall unevenly across platforms.

6.7 Parallax Occlusion Mapping Cost

[Section 5.5](#) ^{→ p. 103} measured `POM`'s cost at a single close-up viewpoint in the demo scene, with the default 8/16 step configuration: GPU time rose from 1.24 ms to 1.31 ms, about 6%, with CPU time unchanged. That single data point is confirmed and extended here across the full step-count axis and averaged over the benchmark flythrough rather than one frame.

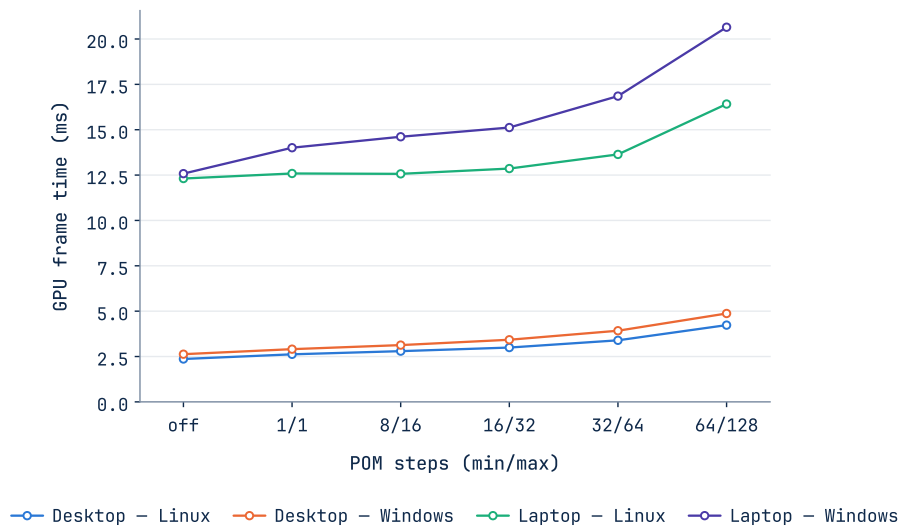


Figure 6.7: GPU frame time against **POM** step configuration, median of 240 sampled frames, one line per device.

The cost is monotonic and grows steadily with the step budget on every device, exactly as expected for an effect that lives entirely in the fragment shader: on **desktop-linux**, GPU time rises from 2.37ms with **POM** off to 2.80ms at the default 8/16 steps (+18.3%) and to 4.21ms at 64/128 steps (+77.5%); the same comparison on **laptop-win** is 12.6 → 14.6ms (+16.3%) and 12.6 → 20.5ms (+62.5%). The flythrough average is a larger relative cost than the single close-up screenshot in [Section 5.5 → p.103](#) reported, which makes sense: that screenshot framed a view dominated by rock, where the early-outs from [Section 5.5 → p.108](#) apply most aggressively, while the benchmark path also crosses stretches where **POM** is active over a larger fraction of the frame. The two measurements do not contradict each other; they characterize the same effect at different points along its cost curve, and the flythrough number is the more representative one for a level a player actually walks through.

6.8 Stress Test

The **stress** scenario sets every sweepable axis to the top of its declared range simultaneously (10 clipmap levels, 512 mesh resolution, 10 noise octaves, 512 physics range, 64/128 POM steps), a configuration well beyond anything the defaults suggest for normal use. It is not a scenario any of the individual sweeps above cover, since each of those isolates one axis while holding the rest at baseline.

Table 6.8: Stress scenario results, median and 90th percentile of 240 sampled frames.

Device	GPU p50 (ms)	GPU p90 (ms)	CPU p50 (ms)	Regrid mean (ms)
desktop-linux	19.96	21.87	0.085	43.92
desktop-win	21.12	23.61	0.131	40.29
laptop-linux	89.15	98.26	0.151	111.81
laptop-win	97.35	106.19	0.311	187.96

Every device still completed all 240 samples inside the 600-frame watchdog cap, so nothing hangs or crashes even at the extreme of the configuration space. The desktop stays close to real-time even here, at roughly 20ms of GPU time; the laptop does not, at 89–97ms median GPU time, close to 10–11 frames per second. The cross-platform gap widens under stress rather than shrinking: `laptop-win`’s regrid mean of 188ms is 68% higher than `laptop-linux`’s 112ms, a larger relative gap than either device showed at baseline. Pushing every axis at once does not surface a new failure mode; it amplifies the same Windows-specific regrid cost already visible in [Section 6.5 → p. 123](#) and [Section 6.6 → p. 124](#).

6.9 Discussion

Across all six sweeps, the same pattern repeats: the two desktop lines (Linux and Windows) stay close together, while the two laptop lines diverge, sometimes sharply, with Windows consistently the more expensive of the pair once a scenario pushes hard enough. That divergence is driven almost entirely by `regrid_ms`, the asynchronous collision-heightmap rebuild from [Section 5.4 → p. 90](#), rather than by GPU frame time, which stays within a roughly constant, platform-independent ratio of the baseline across every sweep. `WorkerThreadPool` task scheduling, not shading or geometry throughput, is the more platform-sensitive part of the plugin, and the one a user targeting Windows should budget for more conservatively than the Linux numbers alone would suggest.

The individual design decisions from [Chapter 5 → p. 74](#) hold up under measurement. The clipmap ([Section 5.2 → p. 78](#)) keeps GPU cost sub-linear in ring count on the desktop; the physics-specific LOD and grid clamp ([Section 5.4 → p. 90](#)) visibly caps regrid cost past a 256-cell grid exactly where the source says it should; and `POM` ([Section 5.5 → p. 103](#)) costs what the single-viewpoint measurement in Phase 4 suggested, scaled up consistently as more steps are spent. None of the 304 benchmark runs triggered the watchdog, so none of this scaling came at the cost of robustness.

Two limitations are worth stating plainly. First, every measured GPU is NVIDIA; the same comparison on an AMD or Intel GPU, or a non-Vulkan backend such as

Metal, is untested. Second, the macOS and mobile (Android) testing originally planned for this phase was not carried out given time and device availability constraints, as noted in [Section 5.7](#)^{→ p. 114}; the cross-platform claims in this chapter are limited to Linux and Windows on x86-64.

7 Conclusions

This chapter summarizes the key findings and outcomes of the project, reviewing the objectives set forth at the beginning and evaluating how well they were achieved. It also reflects on the challenges faced during development, the lessons learned, and potential areas for future work.

7.1 Summary of Achievements

The primary goal of this project was to develop a robust and scalable procedural terrain generation system integrated seamlessly into the Godot Engine. Looking back at the development phases, several key achievements stand out:

- **C++ Godot Plugin Integration:** The project successfully delivered a working plugin for Godot, written entirely in C++ through the GDExtension interface. This established a highly performant and scalable foundation that integrates natively with the engine's ecosystem.
- **Real-Time Infinite Terrain Rendering:** By implementing Geometry Clipmaps and a Vertex Displacement Shader, the system achieves GPU-accelerated infinite terrain rendering, completely eliminating the bottleneck of CPU-side mesh generation and data transfer.
- **Deterministic CPU/GPU Parity:** A significant technical achievement was ensuring that the procedural noise generated on the GPU for rendering matched identically with the noise evaluated on the CPU. This parity was essential for generating an accurate, asynchronous physics collision grid that dynamically follows the player.
- **Advanced Biome Blending:** The terrain is textured using an advanced Whittaker biome classification model. By implementing Triplanar projection, Parallax Occlusion Mapping (POM), and slope-aware overlays, the system provides a high degree of environmental variety with smooth, natural transitions.
- **Robust Developer Experience:** Beyond the underlying algorithms, the plugin provides a polished user experience. Custom editor tools, UI docks, climate charts, and visual debugging gizmos empower developers to easily configure and observe the system's internal state.

7.2 Challenges and Lessons Learned

The development of the TerrainServer plugin was not without its hurdles. Navigating these challenges provided valuable lessons in software architecture, graphics

programming, and Godot engine internals, but also personal growth and resilience through the long development process.

- **Navigating Multiple Implementation Techniques:** One of the broader challenges was researching and evaluating various techniques for terrain generation and rendering. Determining the most suitable approach for each problem, and successfully bringing those theoretical concepts to life, required extensive research and experimentation.
- **Migrating to GPU-Based Generation:** The initial proof of concept highlighted the strict limitations of CPU-based procedural generation. Shifting the heightmap generation to a GPU displacement shader required a paradigm shift in how vertex data and normals were computed, but it yielded massive performance and usability improvements.
- **Achieving Noise Parity:** Synchronizing the CPU and GPU required abandoning standard GLSL transcendental hash functions in favor of an integer-based bit-mixing hash. This ensured that the collision mesh and the rendered geometry were perfectly aligned across all platforms.
- **Fragment Shader Optimization:** The fragment shader responsible for biome blending quickly became a performance bottleneck due to the sheer number of texture samples required for Triplanar projection and POM. Implementing strategic early-outs, axis weight tests, and distance fades was crucial for maintaining interactive frame rates without sacrificing visual fidelity.
- **GDExtension Lifecycle Management:** Managing the memory and lifecycle of custom nodes and resources in C++ exposed several edge cases, such as teardown crashes when the editor closed. This reinforced the importance of understanding Godot's internal initialization levels and class registration order.

7.3 Future Work and Recommendations

While the current state of the TerrainServer plugin fulfills its core objectives, procedural generation is an expansive field with numerous avenues for improvement. The following areas are recommended for future development:

- **Procedural Foliage and Vegetation:** Implementing a scattering system tied to the biome data would allow trees, grass, and rocks to spawn dynamically based on climate, greatly enhancing the realism of the generated worlds.
- **Compute Shaders for Physics and Noise:** Migrating the CPU-side noise evaluation and physics grid regridding to Compute Shaders could yield further performance gains, freeing up the main CPU thread entirely.

- **Dynamic NavMesh Generation:** Adding support for runtime navigation mesh generation would allow AI agents and NPCs to pathfind across the procedurally generated terrain, making it viable for a wider variety of game genres.
- **3D Noise and Voxel Terrain:** Expanding the terrain capabilities beyond a heightmap (2.5D) to support true 3D noise (e.g., caves, overhangs, and arches) using algorithms like Marching Cubes or Dual Contouring.
- **Asset Library Publication:** Once the documentation and edge cases are fully polished, submitting the plugin to the official Godot Asset Library remains a priority to maximize accessibility and foster community contributions.

References

Back-references to the pages where the publication was cited are given by .

- [1] Jonáš Dujava. Jdujava/TeXtured. **2025**.
URL: <https://github.com/jdujava/TeXtured> (visited on 11/08/2025) 2
- [2] Godot Engine. Introducing GDNative's Successor, GDExtension. Godot Engine, **2021**.
URL: <https://godotengine.org/article/introducing-gd-extensions/> (visited on 11/10/2025) 15, 17
- [3] World Machine: The Leading 3D Terrain Generation Software.
URL: <https://www.world-machine.com/> (visited on 11/16/2025) 16, 21, 22
- [4] Stefan Kraus. World Creator - The Elite Realtime Terrain and Landscape Generator.
URL: <https://www.world-creator.com/en/index.phtml> (visited on 11/16/2025) 16, 21, 22
- [5] Gaia | Procedural Worlds.
URL: <https://www.procedural-worlds.com/products/indie/gaia/> (visited on 11/11/2025) 16
- [6] Godot Asset Library.
URL: <https://godotengine.org/asset-library/asset> (visited on 11/11/2025) 16, 17, 18
- [7] The GNU General Public License v3.0 - GNU Project - Free Software Foundation. **2007**.
URL: <https://www.gnu.org/licenses/gpl-3.0.html> (visited on 11/13/2025) 17
- [8] FastNoiseLite. Godot Engine documentation.
URL: https://docs.godotengine.org/en/stable/classes/class_fastnoiselite.html (visited on 11/13/2025) 17
- [9] ArrayMesh. Godot Engine documentation.
URL: https://docs.godotengine.org/en/stable/classes/class_arraymesh.html (visited on 11/13/2025) 17
- [10] MultiMeshInstance3D. Godot Engine documentation.
URL:
https://docs.godotengine.org/en/stable/classes/class_multimeshinstance3d.html (visited on 11/13/2025) 17
- [11] William Deegan. SCons: A Software Construction Tool. SCons.
URL: <https://scons.org/> (visited on 11/13/2025) 18

- [12] Configuring an IDE. Godot Engine documentation.
URL: https://docs.godotengine.org/en/stable/engine_details/development/configuring_an_ide/index.html (visited on 11/13/2025) 18
- [13] HeightMapShape3D. Godot Engine documentation.
URL: https://docs.godotengine.org/en/stable/classes/class_heightmapshape3d.html (visited on 11/13/2025) 18, 88
- [14] EditorPlugin. Godot Engine documentation.
URL: https://docs.godotengine.org/en/stable/classes/class_editorplugin.html (visited on 11/13/2025) 18
- [15] Bartosz Taudul. Wolfpld/Tracy. **2025**.
URL: <https://github.com/wolfpld/tracy> (visited on 11/13/2025) 18
- [16] QuadSpinner. QuadSpinner / Home. QuadSpinner.com.
URL: <https://quadspinner.com/> (visited on 11/16/2025) 21, 22
- [17] Katherine Philbin. Download E-on Software Products for Free | Bentley Systems. **2024**.
URL: <https://www.bentley.com/software/e-on-software-free-downloads/> (visited on 11/16/2025) 21
- [18] Planetside Software. Planetside Software, **2025**.
URL: <https://planetside.co.uk/> (visited on 11/16/2025) 21
- [19] True-VFX Homepage.
URL: <https://true-vfx.xyz> (visited on 11/16/2025) 21
- [20] Blender Foundation. A.N.T.Landscape. Blender Extensions.
URL: <https://extensions.blender.org/add-ons/antlandscape/> (visited on 11/16/2025) 21, 23
- [21] Domlysz/BlenderGIS: Blender Addons to Make the Bridge between Blender and Geographic Data. **2014**.
URL: <https://github.com/domlysz/BlenderGIS/tree/master> (visited on 11/16/2025) 22, 23
- [22] Terrain3D - Godot Asset Library.
URL: <https://godotengine.org/asset-library/asset/3892> (visited on 11/11/2025) 22, 23
- [23] TerraBrush - Godot Asset Library.
URL: <https://godotengine.org/asset-library/asset/2700> (visited on 11/11/2025) 22, 23
- [24] Princeton-vl/Infinigen. Princeton Vision & Learning Lab, **2025**.
URL: <https://github.com/princeton-vl/infinigen> (visited on 11/16/2025) 22
- [25] Jaysmito101/TerraForge3D: Cross Platform Professional Procedural Terrain Generation & Texturing Tool. **2021**.

- URL: <https://github.com/Jaysmito101/TerraForge3D/tree/gen3> (visited on 11/16/2025) 22
- [26] Unity Technologies. **Unity - Manual: Terrain**.
URL: <https://docs.unity3d.com/6000.2/Documentation/Manual/script-Terrain.html>
(visited on 11/16/2025) 23, 25
- [27] **Creating Landscapes in Unreal Engine | Unreal Engine 5.7 Documentation | Epic Developer Community**. Epic Games Developer.
URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-landscapes-in-unreal-engine> (visited on 11/16/2025) 23, 25
- [28] **16 Bit Grayscale PNG Heightmap Generator for Unreal Engine**.
URL: <https://manticorp.github.io/unrealheightmap/index.html> (visited on 11/19/2025) 24
- [29] Ninja Theory (director). **01 Iceland - Development Diary | Senua's Saga: Hellblade II. 2023**.
URL: <https://www.youtube.com/watch?v=pYZVKI-j0YM> (visited on 11/18/2025) 25
- [30] Vossman; M. W. Toews. **English: A Series of Voxels in a Stack with a Single Voxel Shaded. 2006**.
URL: <https://commons.wikimedia.org/w/index.php?curid=1313585> (visited on 11/30/2025) 26
- [31] The original uploader was Thetawave at German Wikipedia. **Deutsch: Modellhafte Veranschaulichung Des Voxelgitters Einer de:Volumengrafik. –Thetawave 21:08, 23. Dez 2005 (CET). 2005**.
URL: <https://commons.wikimedia.org/w/index.php?curid=3891715> (visited on 11/30/2025) 26
- [32] GDC Festival of Gaming (director). **Continuous World Generation in No Man's Sky. 2017**.
URL: <https://www.youtube.com/watch?v=sCRzxEEc02Y> (visited on 11/16/2025) 26
- [33] Christopher C. Tanner, Christopher J. Migdal, and Michael T. Jones. **The Clipmap: A Virtual Mipmap**. IN: *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*. Association for Computing Machinery, **1998**.
DOI: [10.1145/280814.280855](https://doi.org/10.1145/280814.280855) (visited on 11/22/2025) 28
- [34] "The Clipmap: A Virtual Mipmap" by Tanner, Migdal and Jones. **ACM SIGGRAPH HISTORY ARCHIVES, 1998**.
URL: <https://history.siggraph.org/learning/the-clipmap-a-virtual-mipmap-by-tanner-migdal-and-jones/> (visited on 11/22/2025) 28
- [35] Frank Losasso and Hugues Hoppe. **Geometry Clipmaps: Terrain Rendering Using Nested Regular Grids**.
URL: <https://hhoppe.com/proj/geomclipmap/> (visited on 11/16/2025) 28

- [36] Hugues Hoppe (director). Geometry Clipmaps: Terrain Rendering Using Nested Regular Grids (SIGGRAPH 2004). **2020**.
URL: <https://www.youtube.com/watch?v=yoUQRT-Hmcc> (visited on 11/16/2025) 29
- [37] Chapter 2. Terrain Rendering Using GPU-Based Geometry Clipmaps. NVIDIA Developer.
URL: <https://developer.nvidia.com/gpugems/gpugems2/part-i-geometric-complexity/chapter-2-terrain-rendering-using-gpu-based-geometry> (visited on 11/16/2025) 29, 30
- [38] Filip Strugar. Continuous Distance-Dependent Level of Detail for Rendering Heightmaps (CDLOD). *Journal of Graphics, GPU, and Game Tools*, **2010**.
DOI: [10.1080/2151237X.2009.10129287](https://doi.org/10.1080/2151237X.2009.10129287) 31
- [39] Jonathan Dupuy. Concurrent Binary Trees (with Application to Longest Edge Bisection). *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, **2020**.
DOI: [10.1145/3406186](https://doi.org/10.1145/3406186) (visited on 11/24/2025) 32, 33, 34
- [40] Ken Perlin. An Image Synthesizer. *ACM SIGGRAPH Computer Graphics*, **1985**.
DOI: [10.1145/325165.325247](https://doi.org/10.1145/325165.325247) (visited on 11/26/2025) 35
- [41] Matthewslf. English: A Grid of 2D Gradient Vectors for Generating Perlin Noise. **2019**.
URL: <https://commons.wikimedia.org/w/index.php?curid=77691624> (visited on 11/26/2025) 36
- [42] Matthewslf. English: Dot Product of the Gradient with the Distance to the Nearest Node for Generating Perlin Noise. **2019**.
URL: <https://commons.wikimedia.org/wiki/File:PerlinNoiseDotProducts.svg> (visited on 11/26/2025) 37
- [43] Matthewslf. English: The Interpolated Result of 2d Perlin Noise. **2019**.
URL: <https://commons.wikimedia.org/wiki/File:PerlinNoiseInterpolated.svg> (visited on 11/26/2025) 37
- [44] Ken Perlin. Chapter 2: Noise Hardware. *SIGGRAPH 2002 Course 36: Real-Time Shading*, **2002** 38
- [45] Balu Ertl. English: Redrawn by Hand Based on This File: File:Manhattan Voronoi Diagram.Png. This File Contains Layers and Other Helpful Information for Inkscape Editor. **2015**.
URL: <https://commons.wikimedia.org/w/index.php?curid=38535751> (visited on 11/27/2025) 40
- [46] Balu Ertl. English: Redrawn by Hand Based on This File: File:Euclidean Voronoi Diagram.Png. The First Revision (62 Kb) of This File Contains Layers and Other Helpful Information for Inkscape Editor. The Second (Latest, 4 Kb) Revision

- Is Optimized for Transmitting via Network.Magyar: Kézzel Átrajzolva Ezen Fájl Alapján: File:Euclidean Voronoi Diagram.Png. A Fájl Elsőként Feltöltött Változata (62 Kb) Tartalmazza Az Inkscape Szerkesztő Hasznos Adatait, a Második (4 Kb) Pedig Hálózaton Való Továbbításra Optimalizált. **2015**.
URL: <https://commons.wikimedia.org/w/index.php?curid=38534275> (visited on 11/27/2025) 40
- [47] Alain Fournier, Don Fussell, and Loren Carpenter. Computer Rendering of Stochastic Models. *Commun. ACM*, **1982**.
DOI: [10.1145/358523.358553](https://doi.org/10.1145/358523.358553) (visited on 11/27/2025) 42
- [48] Christopher Ewin. English: Visualization of the Diamond Square Algorithm. **2015**.
URL: <https://commons.wikimedia.org/w/index.php?curid=42510593> (visited on 11/27/2025) 43
- [49] danielbeard. Terrain Generation – Diamond Square Algorithm. Daniel Beard’s Programming Blog, **2010**.
URL:
<https://danielbeard.wordpress.com/2010/08/07/terrain-generation-and-smoothing/>
(visited on 11/27/2025) 44
- [50] Inigo Quilez. Inigo Quilez :: fBm.
URL: <https://iquilezles.org/articles/fbm/> (visited on 11/28/2025) 44
- [51] Inigo Quilez. Inigo Quilez :: Domain Warp.
URL: <https://iquilezles.org/articles/warp/> (visited on 11/28/2025) 46
- [52] Éric Guérin, Julie Digne, Éric Galin, Adrien Peytavie, Christian Wolf, Bedrich Benes, and Benoît Martinez. Interactive Example-Based Terrain Authoring with Conditional Generative Adversarial Networks. *ACM Transactions on Graphics*, **2017**.
DOI: [10.1145/3130800.3130804](https://doi.org/10.1145/3130800.3130804) (visited on 11/29/2025) 51
- [53] Navarras. English: Average Annual Temperature and Precipitation Needed for Different Categories of Common Terrestrial Biomes. **2017**.
URL: <https://commons.wikimedia.org/w/index.php?curid=61120531> (visited on 11/29/2025) 53
- [54] Juan Linietsky. Implement Terrain #6121. godotengine/godot-proposals, GitHub, **2023**.
URL: <https://github.com/godotengine/godot-proposals/issues/6121> (visited on 09/11/2026) 54
- [55] Red Eléctrica Redeia. Emisiones y factor de emisión de CO2. Red Eléctrica.
URL: <https://www.ree.es/es/datos/generacion/no-renovables-detalle-emisiones-CO2>
(visited on 01/22/2026) 67

- [56] Dell Technologies. Product Carbon Footprint Report: Dell Precision 3660 Tower. Product Carbon Footprint Report. Dell Inc., **2022**.
URL: <https://www.delltechnologies.com/asset/en-hk/products/desktops-and-all-in-ones/technical-support/dell-precision-3660-tower-pcf-datasheet.pdf> (visited on 01/22/2026) 67
- [57] Dell Technologies. Product Carbon Footprint Report: Dell XPS 15 9530. Product Carbon Footprint Report. Dell Inc., **2023**.
URL: <https://www.delltechnologies.com/asset/en-us/products/laptops-and-2-in-1s/technical-support/dell-xps-15-9530-pcf-datasheet.pdf> (visited on 01/22/2026) 67
- [58] Apple. Product Environmental Report: Mac Mini. Product Environmental Report. Apple Inc., **2023**.
URL: https://www.apple.com/environment/pdf/products/desktops/Mac_mini_PER_Jan2023.pdf (visited on 01/22/2026) 67
- [59] Methodology | Cloud Carbon Footprint.
URL: <https://cloud-carbon-footprint.github.io/docs/methodology/> (visited on 01/22/2026) 67
- [60] Alexandra Sasha Luccioni, Sylvain Viguier, and Anne-Laure Ligozat. Estimating the Carbon Footprint of BLOOM, a 176B Parameter Language Model. **2022**.
DOI: [10.48550/arXiv.2211.02001](https://doi.org/10.48550/arXiv.2211.02001) ARXIV: [2211.02001](https://arxiv.org/abs/2211.02001) [cs] (visited on 01/22/2026) pre-published 67
- [61] GDExtension C++ Example. Godot Engine documentation.
URL: https://docs.godotengine.org/en/4.4/tutorials/scripting/gdextension/gdextension_cpp_example.html (visited on 11/08/2025) 74
- [62] Godotengine/Godot-Cpp. Godot Engine, **2026**.
URL: <https://github.com/godotengine/godot-cpp> (visited on 03/15/2026) 74
- [63] Frank Losasso and Hugues Hoppe. Geometry Clipmaps: Terrain Rendering Using Nested Regular Grids. *ACM Transactions on Graphics*, **2004**.
DOI: [10.1145/1015706.1015799](https://doi.org/10.1145/1015706.1015799) 79
- [64] Textures. Poly Haven.
URL: <https://polyhaven.com/textures> (visited on 09/11/2026) 98
- [65] Megascans. Quixel.
URL: <https://quixel.com/megascans/home> (visited on 09/11/2026) 98
- [66] Texture2DArray. Godot Engine documentation.
URL: https://docs.godotengine.org/en/stable/classes/class_texture2darray.html (visited on 09/11/2026) 100
- [67] RenderingServer. Godot Engine documentation.

- URL: https://docs.godotengine.org/en/stable/classes/class_renderingserver.html
(visited on 09/11/2026) 100
- [68] Ryan Geiss. Chapter 1. Generating Complex Procedural Terrains Using the GPU. Chapter 1 of *GPU Gems 3*, ed. Hubert Nguyen (Addison-Wesley, 2007). NVIDIA Developer, **2007**.
URL: <https://developer.nvidia.com/gpugems/gpugems3/part-i-geometry/chapter-1-generating-complex-procedural-terrains-using-gpu> (visited on 09/11/2026) 101
- [69] Ben Golus. Normal Mapping for a Triplanar Shader. Medium, **2017**.
URL:
<https://bgolus.medium.com/normal-mapping-for-a-triplanar-shader-10bf39dca05a>
(visited on 09/11/2026) 103
- [70] Natalya Tatarchuk. Dynamic Parallax Occlusion Mapping with Approximate Soft Shadows. IN: *Proceedings of the 2006 Symposium on Interactive 3D Graphics and Games*. Association for Computing Machinery, **2006**.
DOI: [10.1145/1111411.1111423](https://doi.org/10.1145/1111411.1111423) (visited on 09/11/2026) 103
- [71] Tomomichi Kaneko, Toshiyuki Takahei, Masahiko Inami, Naoki Kawakami, Yasuyuki Yanagida, Taro Maeda, and Susumu Tachi. Detailed Shape Representation with Parallax Mapping. IN: *Proceedings of ICAT 2001*. **2001** 103